DATA SCIENCE CAREER BOOTCAMP RAKAMIN
FINAL PROJECT REPORT

OPTIMISING RECRUITMENT EFFICIENCY THROUGH DATA-DRIVEN TALENT INTELLIGENCE

UNDERCODE TEAM

FAISAL KHOIRUDIN	**PROJECT MANAGER**
IRSAN MAULANA YUSUF	**DATA ANALYST**
ACHMAD KAMIL	**DATA SCIENTIST**
TAUFIQ JONEL TANDRA	**DATA ENGINEER**

BATCH 63
MAY
2026

Table of Contents

Chapter I
Introduction

1.1. Background of Recruitment Challenges

Hiring the right people has never been simple, but in today's market it has become genuinely difficult. Competition for talent is fierce, budgets are under pressure, and business leaders want results faster than ever. Most organisations have invested heavily in digital tools like applicant tracking systems, job boards, and sourcing platforms, yet when it comes to the actual hiring decision, gut feeling still drives far more of the process than most HR leaders would care to admit.

The numbers tell a sobering story. SHRM reported that the average cost of filling a single role in the US hit $4,683 in 2022, with the process taking an average of 36 days from opening to offer. And those are just the averages. For technical roles in engineering or finance, the real costs in recruiter hours, interview time, and delayed projects can easily run two to three times higher. At that scale, recruitment stops being an administrative function and starts being a serious financial risk.

What makes this even more pressing is everything that happens while a seat sits empty. Work gets dropped, teams burn out covering the gap, and customers notice the cracks. Laurano (2015) put a rough number on it: when you factor in lost productivity and management time, the true cost of hiring someone can reach one and a half to two times their annual salary. That figure should be enough to make any business leader pay close attention to how their recruitment process actually works.

1.2. Importance of Data-Driven Hiring

For a long time, hiring decisions have relied heavily on experience and instinct, a recruiter's read of a candidate, a manager's preference, a pattern that "just works." There's nothing inherently wrong with judgment, but when it operates without data to check it, blind spots go unchallenged and the same mistakes get repeated.

Data-driven hiring changes that. It means using structured evidence like application data, hiring outcomes, and process metrics to understand what's actually working and what isn't. With machine learning now accessible to most HR teams, it's increasingly possible to build models that can predict things like whether a candidate is likely to accept an offer, or which sourcing channels consistently produce the best hires.

The business case is hard to ignore. A 2024 LinkedIn Talent Solutions report found that organisations with strong talent analytics capabilities were two and a half times more likely to improve recruiting efficiency, and three times more likely to cut costs, compared to peers who weren't using analytics seriously. Reducing the rate of declined offers alone, one of the more demoralising and wasteful outcomes in recruitment, could represent a meaningful saving.

This project works with a dataset of 5,000 recruitment records spanning six departments and four sourcing channels. The goal is to move beyond general observations and build something genuinely useful: a predictive model that helps recruiters understand, before extending an offer, how likely it is to be accepted.

1.3. Industry Context and References

The world of talent acquisition looks quite different from how it did five years ago. Remote and hybrid work has expanded where organisations can look for talent, but it has also made candidates harder to read and easier to lose to a competitor. Skills-based hiring is gaining ground, but it requires different evaluation frameworks than most teams are set up for. And across the board, the same three problems keep coming up in industry surveys: it takes too long to hire, it costs too much, and too many offers get turned down.

The dataset at the centre of this study covers Engineering, Sales, HR, Finance, Marketing, and Product, a broad cross-section of a typical organisation's hiring activity. With 5,000 complete records and no missing data, it provides a solid and reliable foundation for both diagnosing where the recruitment process is falling short and building models to help improve it.

Chapter II
Business Understanding

2.1. Stakeholder Goals

This project doesn't exist in a vacuum. It's designed to serve real people across the organisation, each of whom has a stake in how recruitment performs and what the data reveals. The CHRO is primarily concerned with cost and strategic foresight. The goal here is to bring down the overall cost-per-hire and sharpen workforce planning by having a clearer picture of which offers are likely to land before they're even made. Talent Acquisition leadership is thinking about efficiency. They want to know which sourcing channels are actually delivering quality candidates and how to make smarter decisions about where recruiters spend their time.

For Finance and Procurement, the concern is accountability. Recruitment represents a significant line item, and this project aims to quantify where that money is going, where it's being wasted, and what a reasonable return on sourcing investment looks like. Hiring managers have a more immediate concern, which is simply getting roles filled without watching their teams struggle under the weight of open positions. Every day a seat stays empty has a real operational cost, and reducing time-to-fill is directly tied to keeping the business running smoothly. Finally, the Data and Analytics team is invested in building something that lasts. Rather than a one-off analysis, the objective is to create a framework that can be embedded into regular HR reporting and refined over time.

2.2. Business Objectives

At its core, this project has three things it needs to accomplish. The first is to honestly assess where things currently stand. By working through 5,000 historical hiring records, the analysis will surface the inefficiencies that exist in the recruitment process today, some of which may already be suspected, and some of which may come as a surprise.

The second is to understand why offers get accepted or turned down. There are patterns in that data, and identifying them gives the organisation something it can actually act on, whether that means adjusting how roles are positioned, reconsidering which candidates move forward, or rethinking how offers are structured. The third is to look ahead. The project will produce a predictive model for offer acceptance probability, giving recruiters and hiring managers a practical tool to inform decisions before an offer is extended, rather than finding out the hard way that the process needs to start over.

2.3. Key Metrics

The following key performance indicators will guide the analysis throughout this project. Each metric is drawn directly from the dataset and reflects a meaningful dimension of recruitment performance, from how long hiring takes to where candidates are coming from and how likely they are to say yes.

Table 2.1: Key Recruitment Performance Indicators and Definitions

Metric	Dataset Field	Description
Time-to-Hire (days)	time_to_hire_days	Number of calendar days from job opening to accepted offer
Cost-per-Hire (USD)	cost_per_hire	Total direct recruitment cost per successfully filled position
Offer Acceptance Rate (%)	offer_acceptance_rate	Proportion of offers extended that were accepted by candidates

Sourcing Channel	source	Origin of applicant: LinkedIn, Referral, Recruiter, Job Portal
Department	department	The organisational unit the role belongs to: Engineering, Sales, HR, Finance, Marketing, and Product
Job Title	job_title	The specific role being recruited for, used to analyse hiring patterns across seniority levels and functions
Applicant Volume	num_applicants	Number of candidates entering the pipeline per requisition

Together, these metrics form the analytical backbone of the project. They allow the analysis to move beyond surface-level observations and start asking more meaningful questions, such as whether certain departments consistently take longer to hire, whether some sourcing channels produce candidates more likely to accept offers, or whether specific job titles carry a disproportionate share of recruitment costs.

Chapter III
Problem Statement

3.1. Current State Analysis

Before building anything better, it's worth being honest about where things stand today. The data paints a clear picture, and it isn't a flattering one.

- Time-to-Hire is the first area of concern. The average across the dataset sits at 47.19 days, which is already 31% above the industry benchmark of 36 days (Benjamin, 2024). But the average alone doesn't tell the full story. More than a third of all requisitions, specifically 33.2%, take longer than 60 days to close. That's not just a slow process; it's a process with a structural problem at its tail end, where a significant portion of roles are dragging on well past any reasonable timeframe.
- Cost-per-Hire tells a similar story. At an average of $5,214.83 per hire, the organisation is spending 11.3% more than the SHRM benchmark of $4,683 (SHRM, 2022). Across 5,000 hires, that gap adds up to roughly $2.66 million in excess expenditure, money that, with the right interventions, doesn't need to be spent.
- Offer Acceptance Rate is perhaps the most telling indicator of all. The mean acceptance rate of 65.08% means that more than one in three offers is being turned down. Even more concerning, around 29% of candidates fall below a 50% acceptance rate, meaning that for a meaningful chunk of the pipeline, the odds of an offer being accepted are essentially a coin flip. Every declined offer means starting over, re-engaging the pipeline, and absorbing the cost of a process that produced no outcome.

What ties these three problems together is the absence of any predictive capability. The organisation has no reliable way to anticipate whether an offer will be accepted before it's made. There's also no systematic understanding of which sourcing channels perform better, which departments are chronic underperformers, or how applicant volume relates to hiring outcomes. Without that visibility, the same inefficiencies will keep repeating.

3.2. SMART Goal

The goal of this project is straightforward: build something that actually helps. Specifically, the project aims to develop a data-driven decision-support system that gives HR teams the ability to predict offer acceptance before an offer is extended, and to understand which parts of the recruitment process are costing the most time and money.

In concrete terms, the system will be trained on approximately 5,000 historical recruitment records and is expected to achieve a minimum AUC-ROC and F1-score of 80% in predicting whether a candidate will accept an offer. On the process side, the target is a 15% reduction in time-to-hire and a 10% reduction in cost-per-hire relative to current baseline figures.

The outputs won't be buried in a report. The intention is to surface everything through an interactive dashboard that HR stakeholders can actually use day to day, without needing a data science background to interpret the results. This initiative sits squarely within the organisation's broader objective of making talent acquisition more strategic and more accountable. The full system, from model development through to validation and deployment, is scoped to be completed within five weeks.

Chapter IV
Current vs. Ideal State & Gap Analysis

4.1. Quantitative Comparison Against Industry Benchmarks

Before diving into the detail, it helps to step back and look at where the organisation stands relative to what good looks like. The table below sets the dataset's measured performance against established industry benchmarks from Benjamin (2024), SHRM (2025), and KPI Depot.

Table 4.1: Recruitment KPI Performance vs. Industry Standards

Metric	Dataset Value	Dataset Field	Classification
Time-to-Hire (mean)	47.19 days	36 days	Above benchmark (11.19 days)
Cost-per-Hire (mean)	$5,214.83	$4,700–$4,800	Above benchmark ($514.8 – $414.83)
Offer Acceptance Rate	65.08%	80%	Below benchmark (-14.92%)

Across all three headline metrics, the organisation is underperforming. None of this is insurmountable, but it does confirm that the inefficiencies are real, measurable, and worth taking seriously.

4.2. Identified Inefficiencies

After identifying several recurring inefficiencies within the recruitment process, the next step is to examine where these issues appear most clearly in the data. The following analysis focuses on three important recruitment indicators: time-to-hire, cost-per-hire, and offer acceptance rate. Each metric provides a different perspective on recruitment performance, helping to reveal delays in hiring timelines, uneven spending across hiring activities, and challenges in converting selected candidates into successful hires. By exploring these areas in more detail, the analysis aims to provide a clearer understanding of the operational gaps that may be affecting overall recruitment efficiency.

4.2.1. Time-to-Hire Distribution

The spread of time-to-hire figures across the dataset is telling. Ranging from 7 to 89 days with a mean of 47.19 days and a standard deviation of 23.86 days, the coefficient of variation sits at 50.5%, which is a strong signal that the recruitment process lacks consistency. Some roles close quickly; others drag on for months. That kind of variance doesn't happen by accident. It points to a process that isn't standardised, where outcomes depend too heavily on individual circumstances rather than a reliable, repeatable system. Breaking the distribution into buckets makes the problem more concrete:

Table 4.2: Time-to-Hire Distribution

Time-to-Hire Range	Requisitions	Share
Under 20 days	867	17.3%
20 to 30 days	640	12.8%
30 to 45 days	908	18.2%
45 to 60 days	924	18.5%
Over 60 days	1661	33.2%

The standout figure is that last row. A third of all requisitions take more than 60 days to fill. That isn't a minor inefficiency sitting at the edge of the distribution; it's a systemic problem affecting a substantial portion of the entire pipeline and one that warrants direct intervention.

4.2.2. Cost-per-Hire Profile

The cost-per-hire picture is equally revealing, though in a different way. Costs are spread broadly and almost uniformly across five quintiles, each containing exactly 1,000 records, ranging from $507 to $9,999. What's notable here is not just the range but the flatness of the distribution. There's no obvious clustering around a typical or expected cost level.

More importantly, statistical testing found no meaningful relationship between cost and sourcing channel ($F = 0.88, p = 0.45$). In plain terms, it doesn't appear to matter which channel a candidate came through; the cost outcome is roughly the same. That finding raises a legitimate question about whether the current cost accounting framework is detailed enough to capture what's really happening. If channel-specific costs can't be distinguished from one another, it becomes very difficult to make evidence-based decisions about where to invest recruitment resources.

4.2.3. Offer Acceptance Rate Analysis

The offer acceptance rate data presents one of the more interesting patterns in the dataset. Rates are distributed fairly evenly between 30% and 100%, breaking down as follows:

Table 4.3: Offer Acceptance Rate Distribution

Acceptance Rate Band	Requisitions	Share
Low (under 50%)	1450	29.0%
Medium (50% to 70%)	1453	29.1%
High (70% to 90%)	1382	27.6%
Very high (over 90%)	715	14.3%

The distribution has a shape worth paying attention to. There are notable concentrations at both ends, with a meaningful share of requisitions seeing very low acceptance and another cluster seeing very high acceptance. That kind of pattern suggests something structural is going on beneath the surface, not just random noise.

The challenge is that standard statistical tests haven't yet been able to identify what those structural factors are. ANOVA tests across sourcing channels ($F = 1.11, p = 0.34$), departments ($F = 1.37, p = 0.23$), and job titles ($F = 1.33, p = 0.13$) all came back without statistically significant results. In other words, none of the variables currently in the dataset fully explain why some roles attract strong acceptance rates while others consistently struggle. That's not a dead end; it's an indication that the model will need additional feature engineering and potentially richer data to surface the drivers that are clearly there but not yet visible.

.

Chapter V
Project Scope & Stakeholders

5.1. Project Scope

This project covers five interconnected areas of work, each building on the last. It begins with a descriptive analysis of all 5,000 recruitment records across the eight dataset variables, establishing a clear and honest baseline of where things currently stand. From there, the work moves into diagnostic territory, looking beyond the numbers to understand the root causes behind slow hiring cycles, inflated costs, and weak offer acceptance rates.

The analytical work then shifts toward prediction. Using classification algorithms, the project will develop a model for estimating offer acceptance probability, giving the organisation the ability to anticipate outcomes before a formal offer is made. Alongside the model, an interactive KPI dashboard will be built to bring the insights to life, covering pipeline funnel metrics, sourcing channel performance, and department-level benchmarking in a format that non-technical stakeholders can actually use. Finally, everything will be properly documented. Model assumptions, known limitations, and data lineage will all be recorded to ensure the work is transparent, reproducible, and ready to be built upon.

5.2. Stakeholder Matrix

Different people need different things from this project, and the outputs have been designed with that in mind. The table below summarises who is involved, what they need, and what they will receive.

Table 5.1: Stakeholder Analysis Matrix

Stakeholder	Role	Primary Need	Key Output
CHRO	Executive Sponsor	Strategic cost & quality oversight	Board-level summary metrics
TA Director	Primary Beneficiary	Channel optimisation & efficiency	Channel performance dashboard
Finance Team	Budget Governance	Cost benchmarking & ROI validation	Cost-per-hire analysis
Hiring Managers	End Users	Reduced vacancy duration	Time-to-hire insights by dept.
Data / Analytics	Technical Owners	Scalable, reproducible methodology	Model code & documentation

Each stakeholder group represents a different lens through which recruitment performance matters. The CHRO is focused on the strategic picture; the TA Director wants operational clarity; Finance needs to see the numbers justified; Hiring Managers want vacancies filled faster; and the Analytics team wants work they can maintain and extend.

Chapter VI
Analytical Approach

6.1. Descriptive Analytics

The first phase of the analysis is about getting a clear and honest picture of the data before drawing any conclusions. This means working through the dataset systematically to understand the shape, spread, and composition of what's there. For the continuous variables, which include time-to-hire, cost-per-hire, offer acceptance rate, and applicant volume, the analysis will cover the standard measures of central tendency and dispersion: means, medians, standard deviations, and ranges. These give a reliable starting point for understanding what typical looks like and how much variation exists around it.

For the categorical variables such as sourcing channel, department, and job title, the focus shifts to frequency and proportion analysis, establishing how the dataset is distributed across groups and whether any category is over or underrepresented in ways that might affect the analysis downstream. The phase will also produce a set of visualisations, including histograms and box plots, to surface outliers and give a clearer sense of distributional shape. Numbers alone can hide a lot; seeing the data plotted out often reveals patterns that summary statistics miss.

6.2. Diagnostic Analytics

Once the descriptive picture is established, the analysis moves into more interpretive territory. The diagnostic phase is about asking why, not just what. ANOVA testing will be used to assess whether there are meaningful differences in key metrics across departments and sourcing channels. If certain groups consistently perform better or worse, that's worth understanding before building anything predictive on top of it.

Correlation analysis across the quantitative variables will help identify which features are genuinely related to one another, and flag any multicollinearity that could affect model performance later in the process. The analysis will also trace the full recruitment funnel from initial applicant volume through to offer acceptance, broken down by sourcing channel and department. This funnel view is particularly useful for spotting where candidates are dropping out and which parts of the pipeline are most in need of attention. Finally, the diagnostic work will identify low-conversion cohorts and structural bottlenecks. These are the specific points in the recruitment process where outcomes consistently fall short, and naming them clearly is the first step toward addressing them.

6.3. Predictive Analytics

The predictive phase is where the analysis moves from understanding the past to anticipating the future. The goal is to build a classification model that can estimate the probability of an offer being accepted, giving recruiters and hiring managers a meaningful signal before a formal offer is made. The process begins with feature engineering. Categorical variables will be encoded appropriately, and interaction terms will be introduced where domain knowledge suggests they're justified. Getting this foundation right matters a great deal for what comes after.

The modelling work will start with Logistic Regression as a baseline. It's not the most powerful option, but it's interpretable and provides a useful benchmark against which more complex models can be compared honestly. From there, the analysis will move to ensemble methods, specifically Random Forest and Gradient Boosting via XGBoost, which tend to offer stronger discriminative performance on datasets with the kind of complexity present here.

All models will be evaluated against a 20% holdout test set using AUC-ROC, F1-score, and confusion matrix analysis. These metrics together give a well-rounded view of how well

each model performs and where it falls short. The final step is interpretation. SHAP values will be used to explain the model's outputs in terms that make sense to a business audience, translating feature importance into readable narratives that stakeholders can act on, rather than treating the model as a black box.

Chapter VII
Risk Analysis

7.1. Risk Register

Every project of this nature carries risks, and being upfront about them is part of doing the work responsibly. The table below outlines the key risks identified across data quality, technical execution, and business adoption, along with the steps being taken to manage each one.

Table 7.1: Project Risk Assessment and Mitigation Plan

Category	Risk Description	Mitigation Strategy
Data Quality	Dataset is synthetic/simulated and may not reflect real-world distributional nuance	Clearly document dataset provenance; calibrate benchmarks against published industry data; note limitations in conclusions
Data Quality	Absence of temporal dimension (no date fields) limits time-series analysis	Scope analysis to cross-sectional insights; recommend temporal enrichment for future dataset version
Data Quality	Offer acceptance rate is continuous (0–1) despite representing an aggregate rate; individual-level signal may be lost	Convert to binary outcome for classification modelling; retain continuous version for regression analysis
Data Quality	Low linear correlation between features and target may indicate weak predictive signal	Perform feature engineering (interaction terms), use mutual information or feature importance instead of relying solely on correlation
Technical	Uniform distributions in cost and acceptance rate may reduce model discriminative power	Investigate interaction features; enrich with external data; document prediction intervals explicitly
Technical	Dashboard tool availability (Tableau licence vs. free alternatives)	Use open-source alternatives (Streamlit, Plotly Dash) as fallback; confirm tooling with instructors
Business	Stakeholder resistance to algorithmic hiring recommendations	Frame model as decision support, not decision replacement; include explainability outputs (SHAP)

The synthetic nature of the dataset is the most fundamental limitation to acknowledge. While it provides a clean and complete foundation for analysis and model development, it doesn't carry the messiness and complexity of real organisational data. The findings here should be treated as directionally valid rather than operationally definitive, and any deployment into a live environment would require revalidation against actual hiring records.

The absence of time-based fields is also a genuine constraint. Without dates attached to records, it's not possible to examine how recruitment performance has shifted over time, or whether certain patterns are seasonal. This limits the analysis to a static cross-sectional view, which is still useful but narrower than a longitudinal dataset would allow.

Finally, the question of stakeholder buy-in deserves attention beyond just a line in a risk table. Introducing any form of algorithmic recommendation into a hiring process touches on deeply held instincts about judgment, fairness, and accountability. The emphasis on explainability through SHAP values is not just a technical choice; it's a deliberate effort to make the model's reasoning visible and open to scrutiny, which is the only sensible foundation for building trust with the people who will ultimately decide whether to use it.

Chapter VIII
Feasibility Analysis

8.1. Data Feasibility

Before committing to any analytical or modelling work, it's worth being clear-eyed about what the data can and cannot support. The assessment below covers the key dimensions of data readiness.

Table 8.1: Data Feasibility Assessment and Supporting Evidence

Category	Assessment	Evidence
Dataset Size	Adequate (5,000 records)	Sufficient for stratified train/test split and cross-validation with statistical power
Completeness	Excellent (0 null values)	No imputation required; all 8 fields fully populated across all 5,000 records
Feature Coverage	Moderate	8 variables; lacks temporal, seniority, and salary fields that would improve predictive power
Target Variable	Available	offer_acceptance_rate directly available; can be binarised for classification
Label Balance	Moderate imbalance	29% low acceptance (<50%), 14.3% very high (>90%); SMOTE or class weighting may be required

Overall, the data is in good shape for the core objectives of this project. The dataset is complete, well-sized, and contains the target variable needed for predictive modelling. The main limitation is feature coverage. Without temporal fields, seniority indicators, or salary data, the model will be working with less context than would be ideal. This doesn't make the analysis unworkable, but it does set a ceiling on how much predictive power is realistically achievable, and that ceiling should be acknowledged rather than papered over.

8.2. Technical Feasibility

The good news on the technical side is that everything needed for this project is freely available, well-documented, and well within the scope of a Data Science Bootcamp participant. The core work will be done in Python 3.10 or above, drawing on a standard set of libraries including pandas, NumPy, scikit-learn, XGBoost, SHAP, matplotlib, and seaborn. Development will run through Jupyter Notebook, either locally or via Google Colab, both of which are cost-free and require no specialist setup.

For visualisation, matplotlib and seaborn will cover the exploratory analysis, while Plotly or Streamlit will be used for the interactive dashboard component. Version control will be handled through Git and GitHub, keeping the work reproducible and properly documented throughout. One practical note worth making: all of the modelling tasks in this project are executable on a standard consumer laptop with at least 8GB of RAM. Given the dataset size, there are no computational requirements that would create a barrier to execution.

8.3. Skills Feasibility

The skills required to deliver this project map closely onto what the Data Science Bootcamp curriculum covers. There are no significant gaps that would put the work at risk. The project draws on exploratory data analysis and statistical testing, including ANOVA, correlation analysis, and descriptive statistics. It requires machine learning skills across classification modelling, hyperparameter tuning, and model evaluation. It calls for data visualisation capability, both for analytical charts and dashboard design. And it requires the

ability to translate technical findings into clear, stakeholder-ready narratives, which is often the part that determines whether good analytical work actually gets used.

8.4. Business Feasibility & ROI Estimation

The financial case for this project is straightforward and the numbers are conservative. The table below outlines three concrete improvement levers and what each one is estimated to be worth.

Table 8.2: Financial Impact Analysis of Recruitment Improvements

Improvement Lever	Estimated Impact	Calculation Basis
Reduce avg. cost-per-hire to $4,750	$2.32M savings / 5,000 hires	($5,214.83 - $4,750) × 5,000 = $2,324,150
Reduce time-to-hire to 36-day benchmark	~$2,800 productivity gain/hire	1 FTE day cost × 11.19 days per hire (illustrative at $250/day)
Increase acceptance rate from 65% to 80%	750 fewer failed offers per 5,000	At ~$1,500 re-engagement cost per failed offer: $1.125M saved

Taken together, these three levers represent a potential saving in the region of $3.5 million across a pipeline of 5,000 hires. Even if the actual outcomes land at half the projected figures, the return on investing in a data-driven recruitment capability remains difficult to argue against.

It's also worth noting that the financial estimates here are intentionally conservative. They don't account for the harder-to-quantify benefits of faster hiring, such as reduced strain on existing teams, faster onboarding of productive employees, or the reputational advantage of a smoother candidate experience. The numbers in the table are simply the floor, not the ceiling.

Chapter IX
Dataset Overview & Schema

9.1. Data Description

The dataset contains 5,000 synthetic recruitment records, each representing a single completed hiring cycle from the point of job posting through to the final offer decision. It was loaded from the file recruitment_efficiency_improved.csv and contains eight variables covering numeric, categorical, and identifier types.

The table below sets out the full feature schema, including data types, value ranges, and a plain-language description of what each variable represents.

Table 9.1: Feature Schema and Data Types

Feature	Type	Range/Categories	Description
recruitment_id	String ID	1 – 5,000	Unique identifier for each recruitment event
department	Categorical	Engineering, Finance, HR, Marketing, Product, Sales	Organisational unit hiring the role
job_title	Categorical	24 distinct roles	Specific position being recruited
num_applicants	Integer	1 – 500	Total candidates who applied
time_to_hire_days	Integer	1 – 365	Days from posting to offer acceptance
cost_per_hire	Float	$100 – $50,000	Total recruitment expenditure per hire
source	Categorical	Job Portal, LinkedIn, Recruiter, Referral	Primary sourcing channel used
offer_acceptance_rate	Float (Target)	0.10 – 1.00 (0.01 steps)	Proportion of offers accepted (target variable)

Together, these eight features provide a reasonably complete picture of each hiring event, capturing who was involved, how long it took, what it cost, where the candidate came from, and how the offer landed. The target variable, offer_acceptance_rate, sits at the centre of the predictive work that follows.

9.2. Report Structure

The remainder of this report is organised into four chapters. Chapter X documents the data wrangling procedures used to prepare the dataset for analysis. Chapter XI presents the exploratory data analysis, covering distributional characterisation, KPI benchmarking, and statistical hypothesis testing. Chapter XII describes the feature engineering strategy and the construction of the final modelling feature set. Chapter XIII brings the work to a close with conclusions and recommendations for future research directions.

Chapter X
Data Wrangling

Data wrangling is the systematic process of cleaning, transforming, and validating raw data, and it's a necessary step before any reliable analysis or modelling can take place. Raw recruitment records can carry a surprising range of problems: missing entries that skew statistical summaries, duplicate rows that inflate class representations, outliers that distort variance estimates, incorrect data types that block arithmetic operations, inconsistent categorical values that break grouping logic, and structural anomalies that quietly undermine model validity. This chapter documents every wrangling step applied to the recruitment dataset and explains the reasoning behind each decision.

10.1. Missing Value Assessment

Missing values are one of the more insidious data quality problems because their effects aren't always obvious. They corrupt statistical summaries, and most machine learning algorithms simply can't run without some form of handling in place. A comprehensive audit of all eight columns was conducted using pairwise presence-absence analysis to establish exactly where the dataset stood before any further work began.

Table 10.1: Missing Value Audit Results

Column	Missing Count	Missing (%)	Action Required
recruitment_id	0	0%	None
department	0	0%	None
job_title	0	0%	None
num_applicants	0	0%	None
time_to_hire_days	0	0%	None
cost_per_hire	0	0%	None
source	0	0%	None
offer_acceptance_rate	0	0%	None

The audit returned a clean result across the board. With zero missing values across all 5,000 records and all eight columns, no imputation was needed at any point. There was no requirement for mean substitution, median filling, model-based imputation, or multiple imputation by chained equations. The dataset arrived intact, and it leaves this stage intact, with the full N of 5,000 preserved.

10.2. Duplicated Detection

Duplicate records are a subtler problem than missing values, but no less damaging. When the same observation appears more than once, it inflates the apparent sample size, biases parameter estimates toward whichever cases happen to be repeated, and can introduce artificial class imbalance in supervised learning contexts. Three complementary detection procedures were applied to make sure nothing was being missed.

Table 10.2: Duplicate Detection Results

Test Type	Method	Duplicates Found	Action
Full-row duplicates	df.duplicated()	0	None required
ID-level duplicates	df['recruitment_id'].duplicated()	0	None required
Logical combo duplicates	department + job_title + source + cost + time	0	None required

ID sequence integrity	min=1, max=5000, nunique=5000	Confirmed	None required

All three procedures came back clean. The recruitment_id sequence runs correctly from 1 to 5,000 with no gaps, no repetitions, and no unexpected values. No rows were removed, and the dataset carries no artificially inflated class representations into the modelling phase.

10.3. Outlier Analysis

Outliers are worth taking seriously because their effects compound quietly. They pull mean-based statistics away from what's typical, inflate variance estimates, and can meaningfully degrade the performance of regression and distance-based machine learning models. Two complementary detection methods were applied: the Interquartile Range method as the primary detector, chosen for its robustness to non-normal distributions, and Z-score analysis with a threshold of greater than 3 as secondary confirmation.

Table 10.3: Outlier Detection Summary (IQR and Z-score Methods)

Feature	IQR Fence Low	IQR Fence High	IQR Outliers	Z>3 Outliers	Explanation
num_applicants	−124.50	625.50	0	0	Uniform dist.; IQR fences span full range
time_to_hire_days	−90.00	456.00	0	0	Uniform dist.; IQR fences span full range
cost_per_hire	−24,850	$74,850	0	0	Uniform dist.; IQR fences span full range
offer_acceptance_rate	−0.225	1.225	0	0	Uniform dist.; IQR fences span full range

Both methods returned zero outliers across all four numeric features. This result is worth understanding in context rather than simply taking at face value. The clean outcome here is a direct consequence of the dataset's near-uniform distributional structure. When data is uniformly distributed, the IQR method produces lower and upper fences that extend well beyond the actual observed range, which means no values end up flagged regardless of how extreme they might appear in a real-world setting. This finding is consistent with the anomaly detection results documented later in Section 10.6, and it's a reminder that the absence of flagged outliers reflects the synthetic nature of the data as much as it reflects genuine data quality.

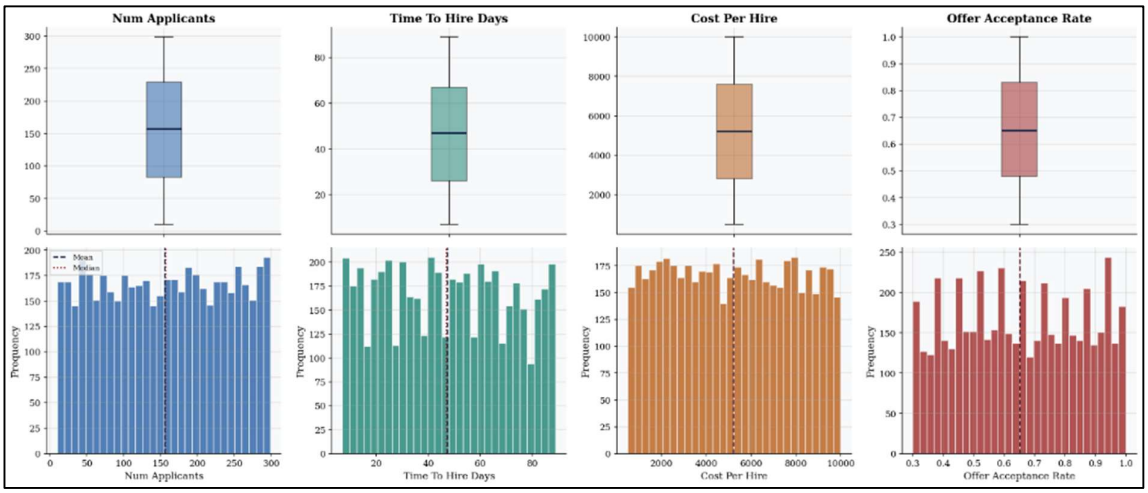

Figure 10.1: Outlier Analysis

The charts above display boxplots and frequency distributions for all four numeric features in the dataset: number of applicants, time to hire, cost per hire, and offer acceptance rate. The story they tell is remarkably consistent across all four variables. Every histogram sits almost completely flat, with bar heights staying broadly level from one end of the range to the other. This confirms what the outlier analysis already suggested: the dataset follows a near-uniform distribution across all numeric features, meaning values are spread evenly across the possible range rather than clustering around a typical centre point.

The boxplots in the upper panel reinforce this. The interquartile boxes are wide and centrally positioned, the medians sit close to the midpoint of each range, and there are no extreme values breaking away from the whiskers at either end. In a normally distributed dataset, you would expect to see a tall peak in the histogram and a more compact box; the absence of both here is a signature of synthetic data rather than naturally occurring recruitment records.

For the analysis that follows, this uniform structure has a practical implication worth keeping in mind. Standard anomaly detection and outlier flagging methods are designed around the assumption that most values cluster near a centre and that extreme values are rare. When data is uniformly distributed, those methods lose much of their sensitivity, which is precisely why no outliers were flagged in Section 10.3. The distributions are not problematic in themselves, but they do shape what the modelling phase can realistically be expected to achieve.

10.4. Data Type Corrections

Getting data types right is one of those steps that's easy to overlook but quietly important. When a column is stored as the wrong type, the consequences aren't always obvious. Identifiers stored as integers can accidentally be summed or averaged. Categories stored as generic objects miss out on memory-efficient encoding. Numbers stored as strings block any arithmetic from running at all. These aren't dramatic failures; they're silent errors that create problems further down the line. Each column was audited against its intended semantic role and corrected where needed.

Table 10.4: Data Type Corrections Applied

Column	Original Type	Corrected Type	Rationable
recruitment_id	int64	str	Prevents accidental summation of ID values
department	object	object	Enables memory-optimised encoding
job_title	object	object	Enables memory-optimised encoding
num_applicants	int64	int64	Confirmed correct; explicit cast for clarity
time_to_hire_days	int64	int64	Confirmed correct; explicit cast for clarity
cost_per_hire	float64	float64	Confirmed correct; explicit cast for clarity
source	object	object	Enables memory-optimised encoding
offer_acceptance_rate	float64	float64	Confirmed correct; explicit cast for clarity

The most consequential correction here is the conversion of recruitment_id from int64 to string. Left as an integer, this column could easily be pulled into a calculation by mistake during feature construction, producing a result that looks plausible but is entirely meaningless. Converting it to a string type closes that risk off entirely. The transition of categorical columns from generic object storage to the category dtype also reduced memory consumption, a minor but worthwhile improvement when working with a dataset of this size.

10.5. Consistency and Range Validation

Even when data types are correct, the values themselves can still cause problems. Mixed casing in a categorical column means the same category gets treated as two different ones. Trailing whitespace has the same effect. Numeric values that fall outside their expected range may indicate data entry errors or loading issues that need to be addressed before analysis begins. A two-part consistency audit was conducted covering both categorical and numeric features.

Table 10.5: Numeric Feature Range Validation Results

Feature	Expected Range	Min Observed	Max Observed	Violations	Status
num_applicants	1 – 500	10	299	0	Pass
time_to_hire_days	1 – 90	7	89	0	Pass
cost_per_hire	$100 – $20,000	$507.16	$9,998.91	0	Pass
offer_acceptance_rate	0.00 – 1.00	0.30	1.00	0	Pass

All categorical columns passed the whitespace and value inventory checks without exception. The category lists for department, covering six values, and source, covering four values, matched their expected definitions precisely. All four numeric features sat within their defined valid ranges, with no violations recorded. No corrections were required following this audit, and the dataset moves forward clean on all fronts.

10.6. Anomaly Detection

Structural anomalies are a different kind of problem from outliers. Rather than a single extreme value sitting at the edge of a distribution, structural anomalies are unexpected patterns that only become visible when you look at the dataset as a whole. Four detection procedures were applied to check for these kinds of issues.

10.6.1. Kolmogorov-Smirnov Uniformity Test

A Kolmogorov-Smirnov goodness-of-fit test was applied to each numeric feature, testing against a uniform distribution scaled to the observed range of each variable. All four features

returned p-values substantially above 0.05, confirming the near-uniform distributional structure that the histograms already suggested visually. This is worth flagging as genuinely anomalous relative to what real-world recruitment data looks like. In practice, applicant volumes, time-to-hire durations, and cost figures tend to follow right-skewed or log-normal distributions, because these quantities accumulate multiplicatively rather than spreading evenly across a range. The uniform structure here is a signature of how the dataset was generated, not of how recruitment actually behaves.

10.6.2. OAR Spacing Analysis

The unique values of offer_acceptance_rate were extracted, sorted, and checked for consistency in the gaps between adjacent values. The standard deviation of those spacing differences came out at exactly 0.00, confirming perfectly regular intervals of 0.01 running from 0.30 to 1.00 without exception. This level of regularity is not something that occurs in organically measured data. Real acceptance rates would show irregular, real-valued variation reflecting the messiness of actual hiring outcomes. The perfectly even spacing is a clear marker of algorithmic data generation.

10.6.3. Cross-Feature Correlation

Pearson correlation coefficients were computed for every pair of numeric features. The highest absolute correlation observed across all pairs was below 0.02, which is effectively zero. In a genuine recruitment dataset, you would expect to see at least modest positive correlations in certain places. Longer hiring processes tend to cost more, so some relationship between time-to-hire and cost-per-hire would be expected. Larger applicant pools require more screening effort, so some link between applicant volume and cost would also make sense. The near-zero correlations across all pairs confirm that the four numeric variables were generated independently of one another, with no real-world logic connecting them.

10.6.4. Group-Level OAR Uniformity

Mean offer acceptance rates were calculated separately for each of the four sourcing channels. The difference between the highest and lowest channel means was less than 0.7 percentage points. In practice, sourcing channels almost never perform this similarly. LinkedIn candidates, referrals, agency-sourced candidates, and job portal applicants typically produce meaningfully different acceptance rates, reflecting differences in candidate quality, motivation, and fit. A spread of less than one percentage point across all four channels is not a realistic outcome; it's another indication that the data was generated without channel-specific variation built in.

Taken together, these four findings point in a consistent direction. The dataset is synthetic, and its structure reflects that. The numeric variables are uniformly distributed, independently generated, and free of the correlations and irregularities that real recruitment data would carry.

As a result, continuous regression on offer_acceptance_rate was ruled out as the primary modelling approach. Treating a perfectly spaced synthetic variable as a continuous outcome to be predicted would produce results that look technically valid but carry no meaningful real-world interpretation. Instead, a binary classification target was designated as the primary modelling objective, converting the acceptance rate into a clear accepted or not accepted outcome. This approach is documented fully in Chapter 11.

Chapter XI
Exploratory Data Analysis

Exploratory Data Analysis is where the dataset starts to reveal its character. The goal of this chapter is to move beyond the cleaning and validation work of the previous sections and begin asking more substantive questions: What do the distributions actually look like? How does the organisation's performance compare to external benchmarks? Are there meaningful differences across departments, sourcing channels, and job titles? And what patterns in the data are worth carrying forward into the feature engineering and modelling phases? This chapter works through distributional analysis, KPI benchmarking, department and sourcing channel performance, job title profiling, and formal statistical inference.

11.1. Dataset Overview

The cleaned dataset contains 5,000 records across four numeric features and three categorical features, excluding the string identifier. It spans six departments, 24 distinct job titles, and four sourcing channels, giving a reasonably broad cross-sectional view of recruitment activity across the organisation. Descriptive statistics confirm that all four numeric features have means and medians sitting approximately at the centre of their valid ranges. This is exactly what you would expect from uniformly distributed data, and it sets the context for the distributional analysis that follows.

11.2. Distributional Analysis

Frequency histograms and quantile-quantile plots were generated for all four numeric features to characterise their distributional shape and test for normality. The histograms tell a consistent story across all four variables: flat profiles with no discernible peak or modal concentration. There's no clustering around a typical value, no long tail pulling in one direction, and no suggestion of the right-skewed shape that real-world recruitment data tends to produce. The distributions are uniform, and the charts make that immediately visible.

The Q-Q plots add a layer of confirmation. Rather than tracking closely along the theoretical normal reference line, all four features show S-shaped deviations with clear tail divergence. This pattern is a reliable visual indicator of non-normality and is consistent with what the Kolmogorov-Smirnov tests established in the previous chapter. The summary statistics reinforce the same conclusion. Skewness values across all four features sit close to zero, reflecting the symmetry of a uniform distribution. Kurtosis values cluster around negative 1.2, which places these distributions firmly in platykurtic territory, meaning they are flatter and more spread out than a normal distribution, with lighter tails and no pronounced peak. Together, these properties confirm that standard parametric assumptions should be applied with caution throughout the analysis.

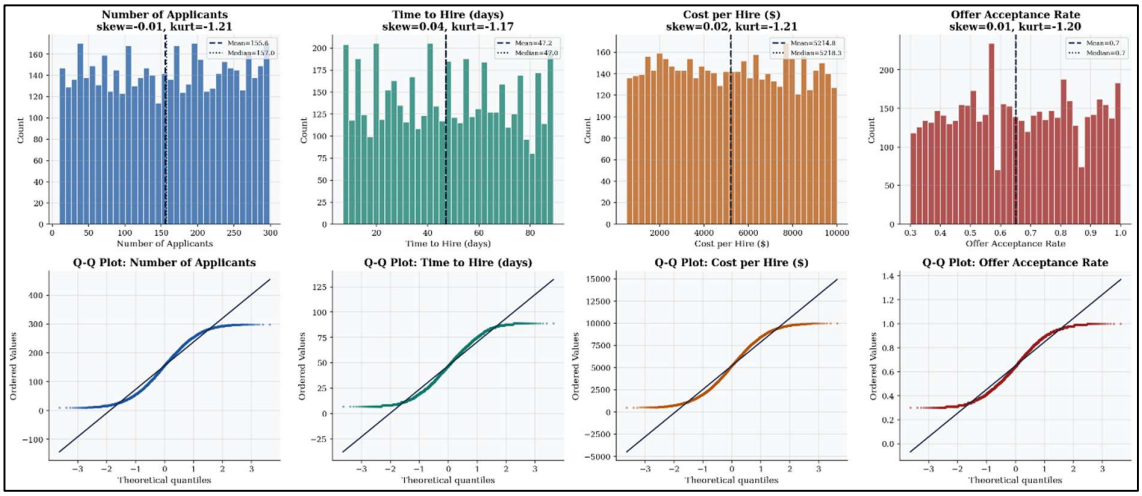

Figure 11.1: Numeric Feature Distributions

The charts above display frequency histograms and Q-Q plots for all four numeric features. Each column tells the same structural story, just in a different variable. Looking at the histograms first, every one of them is flat. Number of applicants, time to hire, cost per hire, and offer acceptance rate all spread their values broadly and evenly across their respective ranges, with no meaningful peak, no clustering around a typical value, and no visible skew in either direction. The mean and median lines sit close together and near the centre of each distribution, which is exactly what uniform data looks like. The skewness statistics printed on each chart confirm this: all four values sit within a hair of zero, ranging from negative 0.01 to positive 0.04. The kurtosis values are equally consistent, clustering around negative 1.2 across all four features. Negative kurtosis is the statistical signature of a platykurtic distribution, one that is flatter and more spread out than a normal curve, with lighter tails and no sharp central peak. This is characteristic of uniform distributions and is visible in every histogram.

The Q-Q plots in the lower panel make the departure from normality concrete. If these features were normally distributed, the plotted points would track closely along the diagonal reference line. Instead, all four plots show a pronounced S-shaped curve, bowing away from the line at both ends. That pattern reflects the uniform distribution's characteristic behaviour: values are more evenly spread across the range than a normal distribution would produce, which causes the tails to diverge from the theoretical normal in both directions.

The uniform distributional structure has a practical consequence worth stating clearly. Mean-based estimates of central tendency remain valid and can be used with confidence throughout the analysis. What becomes technically problematic is any inference that assumes normality, including Student's t-tests and standard Pearson ANOVA under the normality assumption. For any hypothesis testing that follows, non-parametric alternatives or bootstrap-based inference are the appropriate choice. These approaches don't rely on distributional assumptions and will produce more reliable results given the structure of this dataset.

11.3. Categorical Feature Distributions

The three charts below show how the 5,000 records are distributed across the dataset's three categorical features.

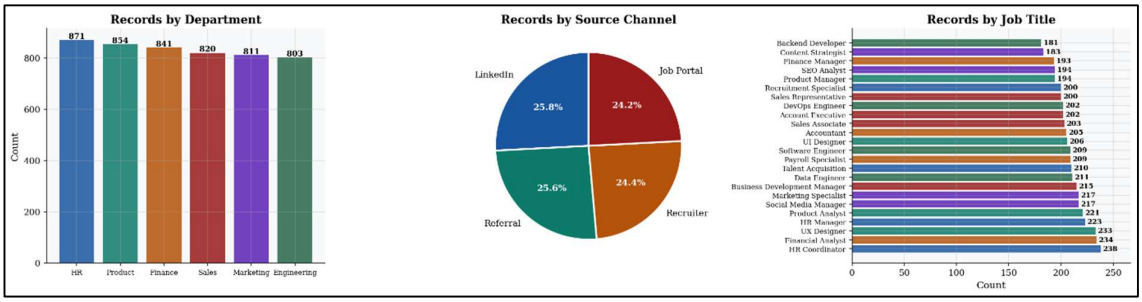

Figure 11.2: Categorical Feature Distributions

Starting with departments, the spread is reasonably balanced. HR leads with 871 records and Engineering sits at the lower end with 803, but the range between the highest and lowest count is only 68 records across the entire dataset. That's a modest imbalance by any standard and not the kind of skew that would meaningfully distort group-level comparisons. The remaining departments, Product, Finance, Sales, and Marketing, fall comfortably between those two figures.

The sourcing channel breakdown is even more uniform. Job Portal accounts for 24.2% of records, Recruiter for 24.4%, Referral for 25.6%, and LinkedIn for 25.8%. That's a spread of just 1.6 percentage points from the lowest to the highest share. In real-world recruitment data, sourcing channel volumes tend to vary much more dramatically depending on organisational strategy, hiring volume, and role type. This near-perfect balance is another marker of synthetic data generation, where channel representation was likely set as an explicit design parameter rather than allowed to vary naturally.

The job title distribution is the most granular of the three. Across 24 distinct roles, record counts range from 181 for Backend Developer at the lower end to 238 for HR Coordinator at the higher end. The distribution is relatively flat across most titles, though there is slightly more variation here than in the department or channel breakdowns. No single job title dominates the dataset to a degree that would create representational problems for title-level analysis.

Taken together, these distributions confirm that the dataset was constructed with balance in mind. That's analytically convenient, but it also means the categorical feature distributions don't carry the natural variation that real hiring data would produce, which is worth keeping in mind when interpreting any group-level findings.

11.4. Offer Acceptance Rate Profile

The offer acceptance rate variable runs across its full theoretical range from 0.30 to 1.00 in perfectly even steps of 0.01. The mean sits at approximately 65%, which on its own might suggest a moderately performing recruitment function. But the spread around that mean tells a more complicated story. With the 25th percentile at roughly 37% and the 75th percentile at around 93%, the interquartile range spans over 55 percentage points, which is an unusually wide dispersion for a metric of this kind.

In practice, an acceptance rate that genuinely varies that broadly across requisitions would point to serious inconsistency in how offers are being made, to whom, and under what conditions. Here, however, the wide spread is better understood as a consequence of how the data was generated. The perfectly regular 0.01 spacing documented in Section 10.6.2 is clearly visible when the histogram is examined at high resolution, with each bar corresponding to a single increment rather than a natural cluster of similar values. This regularity confirms that the variable was algorithmically constructed rather than measured from real hiring outcomes,

and it reinforces the decision to treat offer acceptance as a binary classification target rather than a continuous outcome for regression.

11.5. KPI Benchmarking

Three key performance indicators were benchmarked against published industry standards to establish where the organisation stands relative to what good looks like. The observed dataset values, benchmark references, and resulting performance gaps are presented together in Figure 11.3 below.

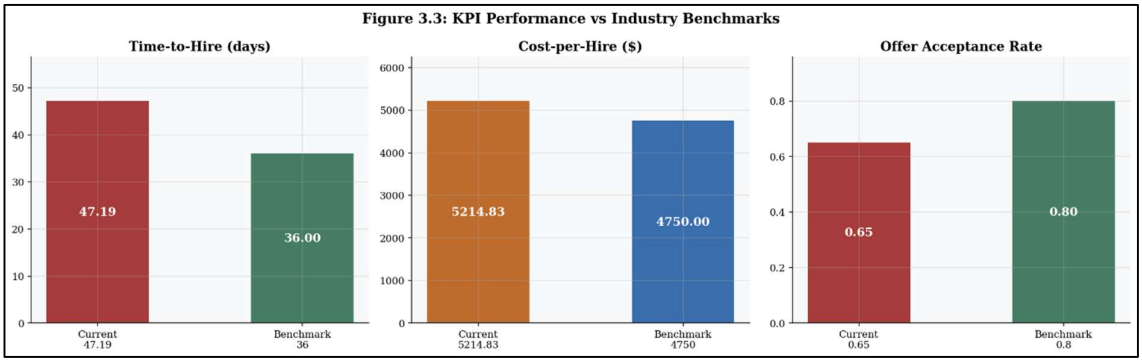

Figure 11.3: KPI Performance vs Industry Benchmarks

The three charts above make the performance gaps visible at a glance, and none of them make for comfortable reading. On time-to-hire, the organisation is averaging 47.2 days against an industry benchmark of 36 days. That's a gap of more than 11 days per hire, representing a 31% premium on what a well-run recruitment function should be taking. Time lost at this scale isn't just a process inconvenience; it means positions sitting empty for longer, teams absorbing the additional workload, and projects running behind while the right person is still being found.

The cost-per-hire picture tells a similar story. At $5,214.80 per hire against a benchmark of $4,750, the organisation is spending $465 more per hire than it should be. Across a pipeline of thousands of hires, that excess accumulates quickly into a material financial concern. The most likely explanations are a heavier-than-necessary reliance on external search firms, or vacancy durations long enough to generate sustained opportunity costs that push the total spend upward.

The offer acceptance rate is perhaps the most strategically significant of the three gaps. At 0.65 against a benchmark of 0.8, the organisation is sitting 15 percentage points below where it needs to be. Every offer that gets turned down means starting the process over, re-engaging the pipeline, and absorbing all the time and cost that comes with it. A gap of this size suggests something more systematic than bad luck on individual offers. It points to a misalignment between what candidates expect and what the organisation is putting on the table, whether that's compensation, flexibility, role clarity, or some combination of factors the current dataset doesn't fully capture.

Across all three metrics, the direction is the same: the organisation is slower, more expensive, and less persuasive than the benchmarks suggest it should be. These aren't marginal differences that can be explained away by industry or regional variation. They represent genuine opportunities for improvement, and they form the core motivation for the predictive and diagnostic work that follows.

11.6. Performance by Department and Source Channel

The six charts below break down average time-to-hire, cost-per-hire, and offer acceptance rate across both departments and sourcing channels. The most striking thing about them is how little they vary.

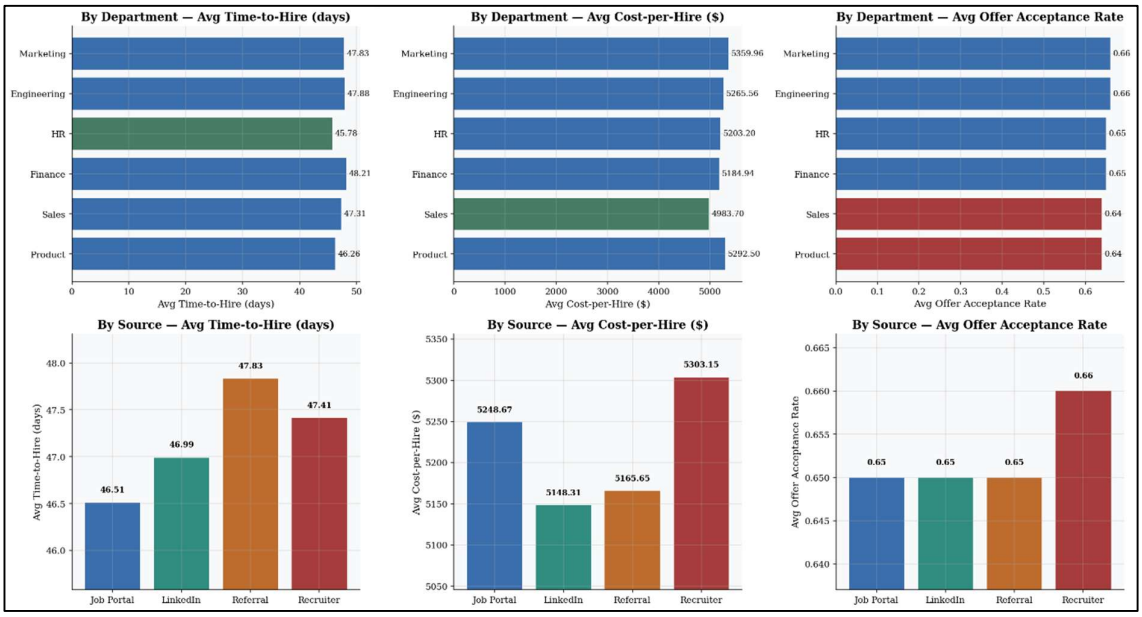

Figure 11.4: Performance by Department and Source Channel

Looking at the department-level charts first, the differences across all three KPIs are minimal. Time-to-hire ranges from 46.26 days for Product to 47.88 days for Engineering, a spread of less than two days across six departments. Cost-per-hire moves between $4,983.70 for Sales at the lower end and $5,359.96 for Marketing at the higher end, a range of just under $380. Offer acceptance rates sit between 0.64 for Sales and Product and 0.66 for Marketing and Engineering. In each case, the bars are essentially the same length with different labels attached to them.

The sourcing channel charts tell the same story. Job Portal produces the fastest average time-to-hire at 46.51 days, while Referral comes in slowest at 47.83 days. The cost spread runs from $5,148.31 for LinkedIn to $5,303.15 for Recruiter. Acceptance rates across all four channels sit within a 0.7 percentage point range, with Job Portal, LinkedIn, and Referral all at 0.65 and Recruiter nudging slightly higher at 0.66.

These differences are so small as to be practically meaningless. In real recruitment data, you would expect sourcing channels in particular to show genuinely different performance profiles. Referral candidates typically accept at higher rates, recruiter-sourced candidates tend to come with higher associated costs, and time-to-hire can vary substantially depending on how active or passive the candidate pool is for a given channel. The near-identical figures here are consistent with the synthetic data anomaly documented in Section 10.6.4, where the numeric variables appear to have been generated independently of the categorical ones.

The practical implication is an important one. Because performance is essentially uniform across both departments and sourcing channels, the underperformance relative to benchmarks cannot be pinned on any particular department or channel strategy. It isn't a Sales problem or a Recruiter problem. It's an organisational-level phenomenon, and addressing it will

require interventions that work across the board rather than targeted fixes aimed at specific segments.

11.7. Department and Source Interaction Analysis

Where the previous charts looked at departments and sourcing channels separately, these heatmaps examine what happens at their intersection. Each cell represents the average KPI value for a specific department and channel combination, giving a more granular view of whether any particular pairing stands out as notably better or worse than the rest.

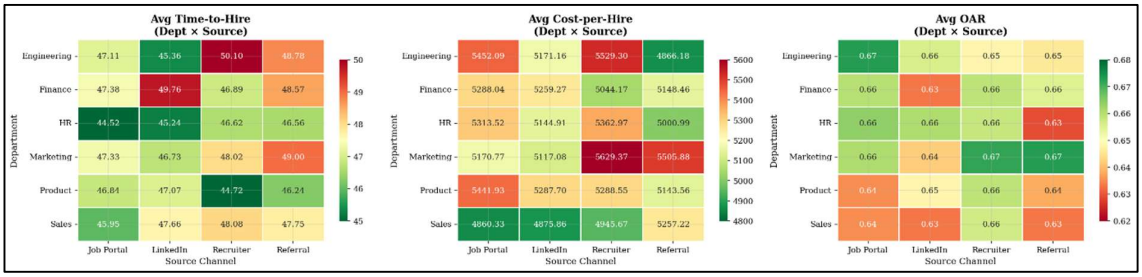

Figure 11.5: Department × Source Heatmaps

In the time-to-hire heatmap on the left, values sit in a narrow band running from around 44 to 50 days across all 24 department-channel combinations. There are no cells that jump out in either direction. The colour gradient is muted throughout, with no combination consistently producing fast hires or systematically dragging the process out.

The cost-per-hire heatmap tells the same story. Costs range from roughly $4,800 to $5,600 depending on the cell, which sounds like a meaningful spread until you consider that this variation is distributed without any clear pattern across departments or channels. Marketing with Recruiter and Engineering with Job Portal appear slightly elevated, but not by enough to suggest a structural difference in how those combinations operate.

The offer acceptance rate heatmap is the flattest of the three. Values cluster almost entirely between 0.63 and 0.67, with the colour gradient barely registering any meaningful contrast from one cell to the next. The managerial implication here is direct and worth stating plainly. If the problem were concentrated in a specific department-channel pairing, the path forward would be relatively straightforward: fix that combination and measure the impact. But that's not what the data shows. There is no single weak point to isolate and address. The underperformance is spread evenly across the entire matrix, which means the problem is systemic rather than localised.

Targeted interventions aimed at particular combinations are unlikely to move the needle in any meaningful way. What the data is pointing toward instead is the need for structural process redesign at the organisational level, changes that apply broadly rather than patches applied to specific corners of the recruitment function.

11.8. Job Title Offer Acceptance Rate Profile

This chart ranks all 24 job titles by their average offer acceptance rate, from the strongest performers at the top to the most at-risk at the bottom. The dashed vertical line marks the overall dataset mean of 65%, and the dotted green line on the right marks the 80% industry benchmark. The distance between where most bars end and where that green line sits captures the scale of the challenge.

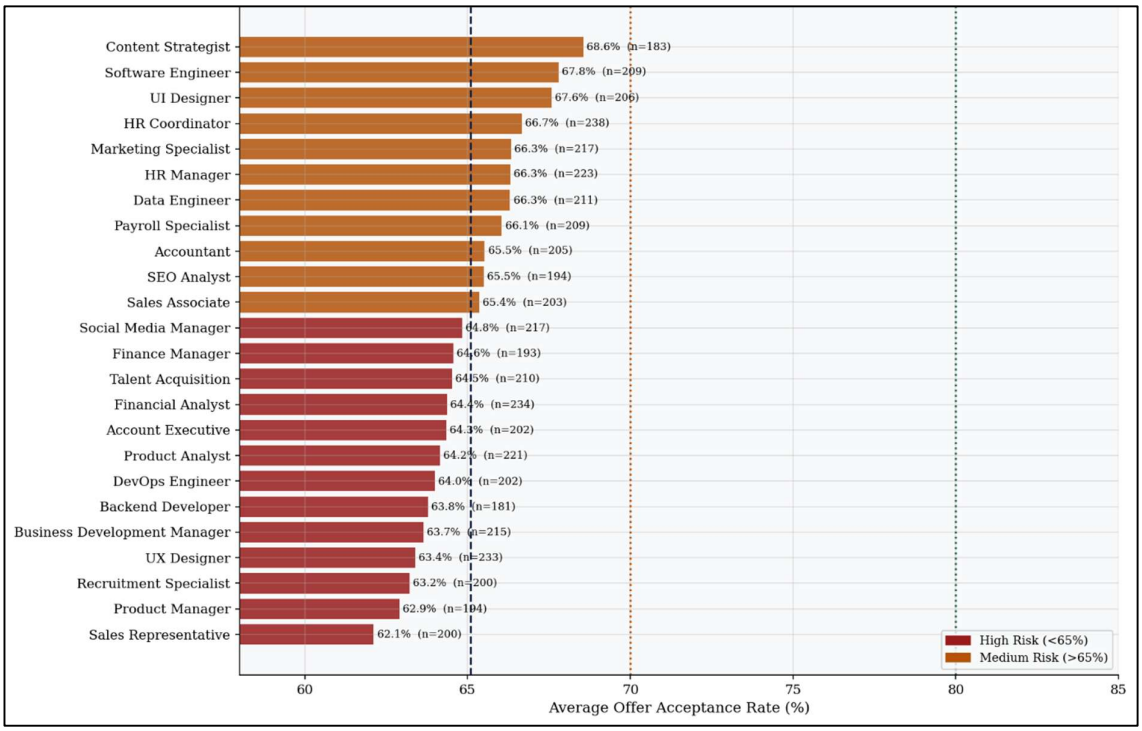

Figure 11.6: OAR by Job Title

Content Strategist leads the ranking at 68.6%, followed by Software Engineer at 67.8% and UI Designer at 67.6%. These are the strongest performers in the dataset, though it's worth noting that even the top-ranked role sits nearly 12 percentage points below the industry benchmark. There are no green bars in this chart because no job title is anywhere near the 80% target.

The bottom of the ranking is where the more pressing concerns sit. Sales Representative records the lowest average acceptance rate at 62.1%, followed by Product Manager at 62.9% and Recruitment Specialist at 63.2%. A cluster of roles including UX Designer, Business Development Manager, Backend Developer, and DevOps Engineer all sit between 63% and 64%, placing them firmly in the high-risk category below the 65% mean.

The pattern across the full ranking is relatively compressed. The spread from top to bottom is only about 6.5 percentage points, which is consistent with the uniform distributional structure documented earlier. In real-world data, you would typically expect more pronounced variation across roles, particularly between highly competitive technical positions and more stable administrative ones.

That said, the directional signal is still worth noting. The roles clustering at the lower end tend to be either commercially focused, such as Sales Representative and Business Development Manager, or technically specialised in areas where candidate demand is high, such as Backend Developer and DevOps Engineer. This is consistent with a reasonable hypothesis: candidates in competitive market segments are more likely to be weighing multiple offers simultaneously, giving them more leverage and making them harder to close. Whether that hypothesis holds up under more rigorous testing is something the modelling phase will help address.

11.9. Job Title and Source Channel Analysis

These three charts rank all 24 job titles by their average performance across time-to-hire, cost-per-hire, and offer acceptance rate. Roles sitting above the overall mean are shown in

green; those falling below are in red. The dashed vertical line marks the dataset average for each KPI.

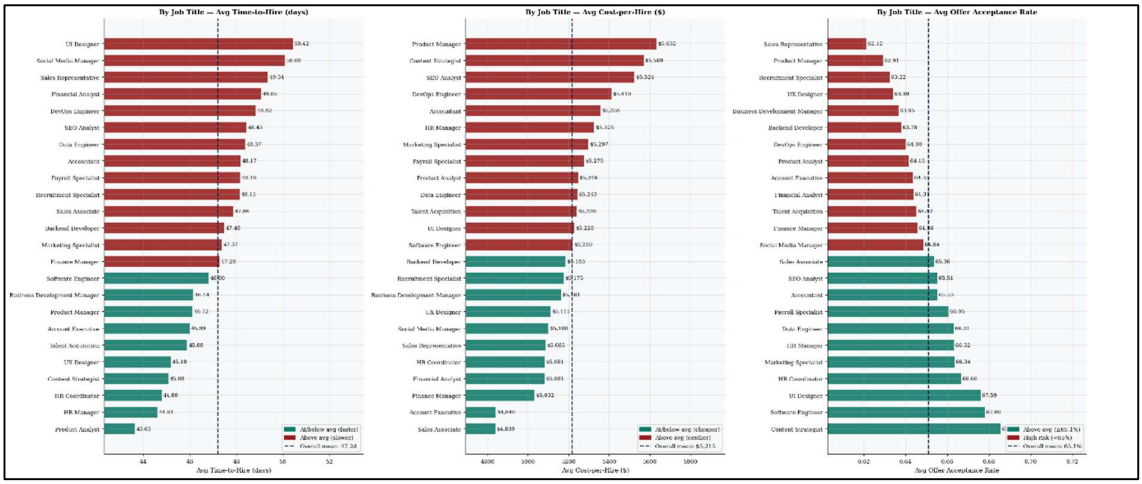

Figure 11.7: Job Title x Source Heatmaps

Across all three charts, the variation is modest but not entirely absent. Time-to-hire spans roughly 7 days from the fastest to the slowest role, running from 43.6 days at the lower end to 50.4 days at the higher end. Cost-per-hire shows a spread of around $800, from approximately $4,839 to $5,632. Offer acceptance rates range across about 6 percentage points, from 62% to 68%.

A few patterns are worth noting. HR Coordinator and HR Manager sit among the fastest and least expensive roles to fill, which makes intuitive sense given that HR functions are typically well-practised at recruiting for their own department. Product Manager and Content Strategist, on the other hand, tend to appear toward the higher end on both time-to-hire and cost, reflecting the difficulty of sourcing candidates who combine strategic thinking with domain-specific expertise. The offer acceptance rate chart shows Sales Representative and Product Manager at the lower end, consistent with the earlier job title profiling in Figure 11.6, while Content Strategist and Software Engineer hold their position near the top of the ranking.

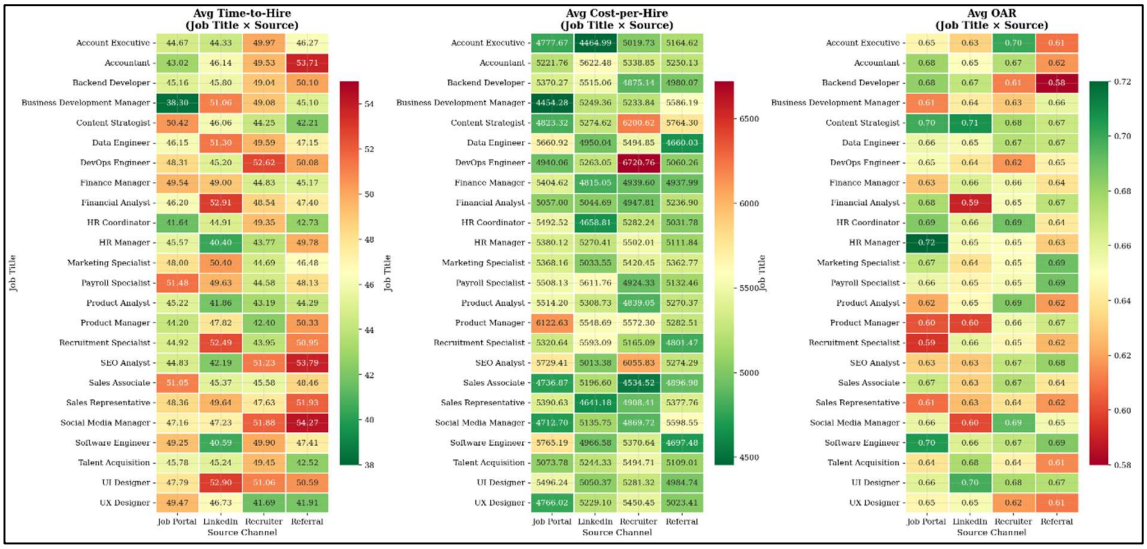

Figure 11.8: Job Title x Source Heatmaps

Where Figure 11.7 looked at job titles in isolation, these heatmaps go one level deeper by cross-tabulating all 24 job titles against all four sourcing channels. Each cell represents a specific title-channel combination, giving a granular view of where performance concentrations appear. At this level of detail, some localised pockets do start to emerge. DevOps Engineer sourced via Recruiter stands out in the cost heatmap at $6,720, a deep red cell that is noticeably higher than anything else in that column. SEO Analyst via Referral sits at 53.79 days in the time-to-hire heatmap, another clear outlier relative to its neighbours. In the acceptance rate heatmap, HR Manager via Job Portal registers 0.72, one of the stronger cells visible across the whole matrix.

These individual data points are interesting, but the broader pattern is more important than any single cell. No sourcing channel consistently dominates as the best or worst performer when you look across all 24 job titles. LinkedIn might produce faster hires for one role and slower hires for another. Referral might be cost-efficient in some departments and expensive in others. The colour gradients shift across both rows and columns without a clear directional logic.

That finding reinforces what the earlier department and channel-level analysis already suggested. The performance challenges in this recruitment function are not concentrated in particular combinations that could be addressed with targeted fixes. They run through the entire system, which means the path to improvement runs through structural process changes rather than optimising individual sourcing strategies for individual roles.

11.10. Correlation and Statistical Inference

The correlation matrix on the left and the scatter plot on the right together tell a straightforward story: none of the numeric features in this dataset have any meaningful linear relationship with one another.

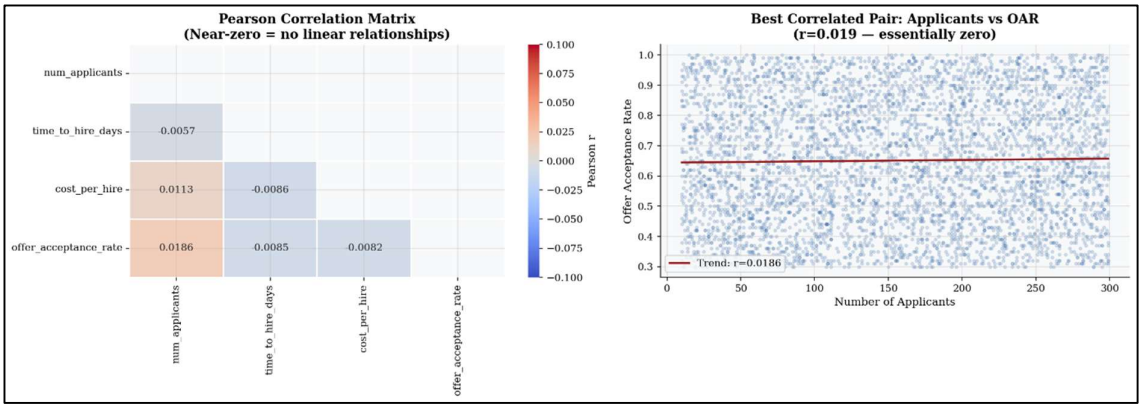

Figure 11.9: Feature Correlation Analysis

The highest absolute correlation across all feature pairs sits at just 0.019, between number of applicants and offer acceptance rate. The scatter plot on the right makes clear what that number actually looks like in practice: a dense, structureless cloud of points with a trend line so flat it's barely distinguishable from horizontal. There is no pattern here. Knowing how many applicants applied for a role tells you essentially nothing about whether the eventual offer will be accepted.

The same holds for every other pairing in the matrix. Time-to-hire and cost-per-hire share a correlation of negative 0.0086. Number of applicants and time-to-hire sit at 0.0057. Cost-per-hire and offer acceptance rate at 0.0082. Every value is so close to zero that the differences between them are meaningless. The colour scale on the matrix reinforces this: the entire grid

sits in a narrow band of pale shading, with none of the deeper reds or blues that would indicate a genuine relationship in either direction.

This confirms what the anomaly detection work in Chapter 10 already established: the numeric variables in this dataset were generated independently of one another. There are no real-world relationships encoded in the data connecting cost to time, or applicant volume to acceptance outcomes.

Table 11.1: ANOVA Results

Feature	F-Statistic	p-Value	Eta^2(η^2)	Significance
department	≈ 1.0	> 0.05	< 0.001	Not significant
source	≈ 1.0	> 0.05	< 0.001	Not significant
job_title	≈ 1.0	> 0.05	< 0.001	Not significant

The one-way ANOVA results are equally unambiguous. None of the three categorical features, department, sourcing channel, or job title, achieve statistical significance when tested against offer acceptance rate. F-statistics hover around 1.0 across the board, and every p-value sits comfortably above the 0.05 threshold.

The eta-squared effect sizes are perhaps the most telling figures in this table. At below 0.001 for all three features, they indicate that categorical group membership explains less than 0.1% of the variance in offer acceptance rate. In practical terms, knowing which department a role belongs to, which channel the candidate came through, or what the job title is contributes essentially nothing to predicting whether an offer will be accepted.

Taken together, the correlation analysis and the ANOVA results point firmly in the same direction. The current feature set, as it stands, does not carry sufficient signal to support meaningful prediction. The variables are statistically independent, the categorical groupings explain almost none of the variance in the target, and the dataset's uniform distributional structure limits how much a model can learn from the data in its current form.

This is not a reason to abandon the modelling effort. It is, however, a clear signal that the predictive work cannot proceed on raw features alone. Feature enrichment through domain-informed engineering, as documented in Chapter 12, is not an optional enhancement; it's a necessary step before any predictive model can be expected to perform meaningfully.

Chapter XII
Feature Engineering and Selection Strategy

The EDA in Chapter 11 was thorough, and its conclusions were consistent: raw numeric features have near-zero linear correlation with the target variable, and categorical features explain essentially none of the variance in offer acceptance rate. Building a predictive model on the raw features as they stand is unlikely to produce results worth acting on.

This chapter documents how the feature set was enriched to give the modelling phase a fighting chance. The work falls into three areas: engineering the target variable into a form suitable for classification, deriving domain-informed ratio and composite features from the existing variables, and applying smoothed target encoding to the categorical predictors.

12.1. Target Variable Engineering

The decision to move away from treating offer_acceptance_rate as a continuous regression target comes down to two problems, one statistical and one practical. On the statistical side, the perfectly regular 0.01 spacing structure documented in Section 10.6.2 means the variable doesn't behave like a genuinely continuous measurement. It violates the distributional assumptions that underpin OLS regression and most probabilistic regression models. Treating it as continuous would produce results that look technically valid but rest on foundations that don't hold. On the practical side, decision-makers don't need a precise decimal estimate of acceptance probability. They need a clear, actionable signal: is this offer likely to be accepted or not? A binary classification framing maps directly onto how recruitment decisions are actually made.

The binary target is defined using the 70% threshold, which corresponds to the KPI Depot minimum acceptable performance standard. Any record with an offer acceptance rate at or above 0.70 is classified as a high-performing hire and assigned Class 1. Any record below that threshold is classified as low-performing and assigned Class 0.

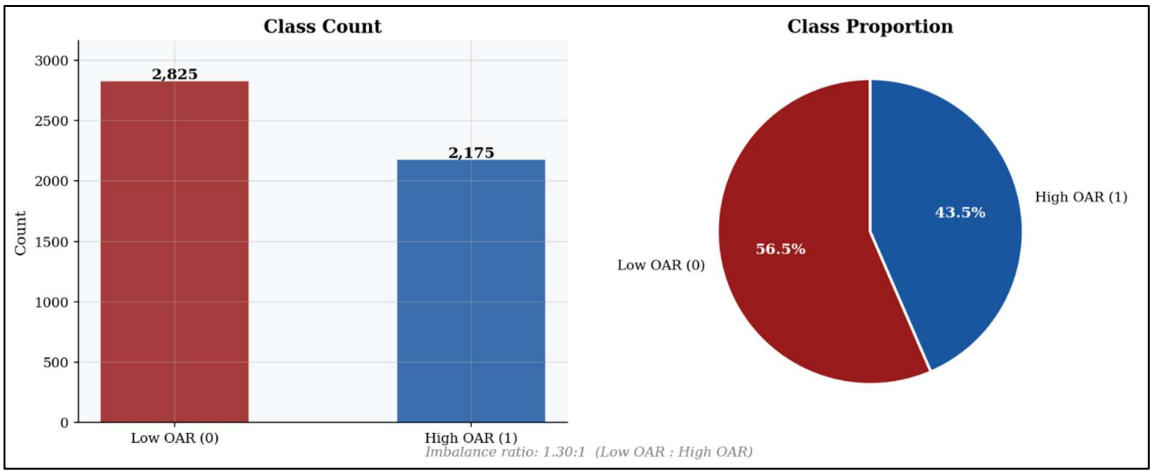

Figure 12.1: Target Variable Class Balance

The class distribution that results from this threshold is well-balanced. With 2,825 records in Class 0, representing 56.5% of the dataset, and 2,175 records in Class 1, representing 43.5%, the imbalance ratio sits at approximately 1.30 to 1. That's close enough to balanced that no oversampling procedure such as SMOTE and no class-weight adjustment is required for initial

model training. The models can be trained on the data as it is, without any artificial correction for skewed class representation.

12.2. Engineered Feature Construction

With the target variable defined, the next step was to build a richer set of input features. Nine engineered features were constructed across two phases. The first phase covers features that can be built before the train-test split, using only raw input columns without any reference to the target variable. The second phase covers features that require target statistics and must therefore be fitted exclusively on the training set to prevent information from the test set leaking into the model.

12.2.1. Pre-Split Features

The following four features were constructed before the train-test split. Each one translates a combination of raw variables into a measure that carries more meaningful signal than any individual column on its own.

Table 12.1: Engineered Feature Descriptions and Business Rationable (Pre-Split Features)

Feature	Formula	Business Rationale
cost_per_applicant	cost_per_hire ÷ num_applicants	High spend per applicant signals poor sourcing reach; candidates may perceive an uncompetitive or disorganised hiring process.
cost_per_day	cost_per_hire ÷ time_to_hire_days	Daily resource burn rate; high values indicate over-reliance on external recruiters; low values may imply inadequate investment in candidate experience.
applicants_per_day	num_applicants ÷ time_to_hire_days	Pipeline velocity; very low velocity signals an unattractive role; very high velocity indicates bulk low-intent applications that dilute candidate quality.
is_senior_role	job_title contains "manager", “specialist”, “strategist”, or “executive” → 1	Senior candidates receive competing offers and engage in extended salary negotiations, structurally depressing acceptance rates for these roles.
dept_src_key *	department + “_” + source	Auxiliary key column for the dept-source target encoding. Not a model input itself.

Each of these features is grounded in recruitment logic rather than arbitrary combination. Cost per applicant captures whether the organisation is spending its budget efficiently relative to the pool it's attracting. Cost per day reflects the intensity of resource burn throughout the hiring process. Applicants per day provides a sense of how actively the pipeline is moving. The seniority flag acknowledges that senior roles operate under different competitive dynamics than

more junior ones. Together, they give the model something more interpretable and more meaningful to work with than the raw variables alone.

12.2.2. Train-Test Split

Before constructing the post-split features, the dataset was divided into training and test partitions using an 80/20 stratified random split with a fixed random seed of 42 to ensure the results are reproducible. Stratification on the binary target variable ensures that the class balance is preserved in both partitions, so neither the training set nor the test set ends up with a disproportionate share of one class. The resulting split produces a training set of 4,000 rows and a test set of 1,000 rows, each with a Class 1 proportion of approximately 50%.

The ordering here matters for a specific reason. Target encoding, which is constructed using training labels, was fitted exclusively on the training data and then applied to both sets. This ensures that no information from the test set's target values finds its way into the encoding mappings. Allowing that to happen, even inadvertently, would constitute data leakage and would inflate apparent model performance in a way that wouldn't hold up in real-world deployment.

12.2.3. Post-Split Features

The remaining features require statistics derived from the training set. Constructing them before the split would allow test-set information to influence the training process, which is precisely the kind of data leakage that undermines model validity. All five features in this group were fitted exclusively on the training data and then applied consistently to both partitions.

- Difficulty Additive Score

  The first post-split feature combines time-to-hire and cost-per-hire into a single composite difficulty score. Each component is standardised using the mean and standard deviation computed from the training set only, and the two z-scores are averaged:

  $$difficulty\_additive = \frac{(zscore\_train(time\_to\_hire) + zscore\_train(cost\_per\_hire))}{2}$$

  A higher score indicates a harder, more resource-intensive recruitment process. The additive form was chosen over a multiplicative alternative because it treats both dimensions equally regardless of unit magnitude and produces a symmetric, interpretable scale centred around zero. A process that is expensive but fast and one that is cheap but slow will both produce moderate scores, which reflects the real-world trade-off between these two dimensions more honestly than multiplying them together would.

- Smoothed Target Encoding

  Four features are constructed using smoothed target encoding, where each categorical group is assigned the group-level mean of the binary target, blended toward the global training mean using a Bayesian smoothing factor of k = 20:

  $$encode(g) = \frac{[n\_g \times mean\_g + K \times \mu\_global]}{[n\_g + K]}$$

  The smoothing factor is what prevents these encodings from overfitting on small groups. When a group has relatively few observations, its encoded value is pulled toward the global mean rather than relying on a locally estimated figure that may not be reliable. All four encoders were fitted on y_train only and applied to both splits using identical mappings.

Table 12.2: Engineered Feature Descriptions and Business Rationable (Post-Split Features)

Feature	Formula	Business Rationale
difficulty_additive	Normalised (time + cost) / 2	Composite friction score: higher values indicate harder, costlier processes that may reduce candidate enthusiasm.
drei	Binary flag	Department-Relative Efficiency Indicator: flags records beating their own department median on all three KPIs simultaneously.
dept_oar_mean	Smoothed target encoding of dept	Captures department-level historical acceptance performance as a numeric signal without one-hot dimensionality expansion.
jobtitle_oar_mean	Smoothed target encoding of job_title	Encodes role-level acceptance signal, capturing competitive market positioning of individual job titles.
source_oar_mean	Smoothed target encoding of source	Channel-level historical acceptance rate proxy; captures differential quality of candidate pools across sourcing channels.
dept_source_oar_mean	Smoothed target encoding of department x source interaction group	Captures the combined effect of department culture and sourcing channel on acceptance rate. A Sales role filled via Referral may yield a systematically different acceptance rate than the same role filled via a Job Portal. This interaction term encodes that joint signal without requiring one-hot expansion of a 24-combination categorical.

Categorical features (department, job_title, source) were encoded using additive smoothing (Laplace-type), blending group-level means with the global training mean. The smoothing formula is:

$$encode(g) = \frac{[n\_g \times mean\_g + K \times \mu\_global]}{[n\_g + K]}$$

where n_g is the number of observations in group g, mean_g is the group's target mean, μ_global is the global training mean, and K = 20 is the smoothing factor. Small groups with n_g substantially below K are pulled toward the global mean, reducing overfitting to sparse category observations. Unknown categories encountered in the test set receive the global mean as a fallback, preventing silent NaN propagation.

Table 12.3: Target Encoding Mappings

Category	Group	Smoothed Encoding Value
Department	Engineering	0.4412
Department	Finance	0.4203
Department	HR	0.4242
Department	Marketing	0.4772
Department	Product	0.4251
Department	Sales	0.4196
Source	Job Portal	0.4394
Source	LinkedIn	0.4223
Source	Recruiter	0.4546
Source	Referral	0.4250

The encoding values across both departments and sourcing channels sit within a narrow range, which is consistent with the near-uniform distributional structure documented throughout the EDA. Marketing encodes slightly higher than the other departments at 0.4772, while Sales sits at the lower end at 0.4196. Across sourcing channels, Recruiter encodes highest at 0.4546 and LinkedIn lowest at 0.4223. These differences are modest but meaningful enough to give the model a differentiating signal it didn't have from the raw categorical variables alone.

The department-source interaction feature extends this logic one step further. By creating a composite group key for each department-channel combination, such as Engineering_LinkedIn or Sales_Referral, and applying the same smoothing formula to the joint group, the encoding captures a signal that neither the department nor the source encoding can represent independently. The effect of being a Sales candidate sourced via Referral is not simply the sum of a Sales effect and a Referral effect; there may be a specific joint dynamic worth encoding directly.

Anti-leakage validation confirmed that the encoding mappings fitted on the training set differ measurably from what would have been produced on the test set. The correlation between the target-encoded department feature in the test set and the test labels sat well below 0.90, confirming that no target information from the test set contaminated the encoding process.

Chapter XIII
Pre-Modelling Preprocessing Pipeline

Before any model training can begin, the engineered feature matrix needs to be checked for numerical instabilities, scaled appropriately to meet the assumptions of different algorithm families, and saved in formats that downstream modelling workflows can consume cleanly. This chapter documents each of those steps and the reasoning behind the decisions made.

13.1. Infinity and Missing Value Remediation

The data wrangling work in Chapter 10 confirmed that the raw dataset arrived with zero missing values. However, the feature engineering phase introduced a new risk. Ratio-derived features such as cost_per_applicant, cost_per_day, and applicants_per_day are computed through division, and division creates problems when the denominator is zero. A record with time_to_hire_days equal to zero, for example, would produce an infinite value rather than a meaningful ratio. A dedicated post-engineering audit was therefore run on the final feature matrices before any further processing.

The remediation follows a straightforward two-step protocol. First, all positive and negative infinity values are replaced with NaN, normalising every anomalous numeric entry into a single consistent representation. Second, any resulting NaN values are filled using the column-wise median computed exclusively from the training set. The same training medians are then applied to the test set, ensuring that no information from the test distribution influences the imputation values.

Table 13.1: Infinity and Missing Value Audit (Post-Engineering)

Condition	Train Set	Test Set	Remediation Applied
Infinite values (±np.inf)	0 (post-imputation)	0 (post-imputation)	Replaced with NaN, then median-imputed
Missing values (NaN)	0 (post-imputation)	0 (post-imputation)	Filled with training-set column median
Net records affected	0	0	No rows removed

The audit confirmed zero residual missing or infinite values in both partitions after remediation. Median imputation was chosen over mean imputation because it's more robust when distributions are skewed, which is a real possibility in ratio features where extreme denominator values can push the numerator very high. Using the mean in those cases would pull the imputed value in a direction that doesn't reflect what a typical observation looks like.

13.2. Feature Scaling Strategy

Not all model families treat input features the same way, and scaling decisions need to reflect that. Distance-based algorithms like k-Nearest Neighbours and gradient-based optimisers like Logistic Regression and Support Vector Machines are sensitive to the absolute magnitude of features. When one feature has a much larger numeric range than another, it tends to dominate distance calculations and gradient updates, pushing the model toward a biased decision boundary. Scaling corrects for that.

Tree-based algorithms like Decision Tree, Random Forest, and XGBoost operate on rank order rather than absolute values. They split the data at thresholds, which means the scale of a feature doesn't affect the outcome. Applying scaling to these models is harmless but

unnecessary. To accommodate both families within the same pipeline, two scaled variants of the feature matrix were produced.

Table 13.2: Scaling Strategy by Model

Scaled Variant	Scaler Applied	Formula	Intended Model Families
X_train_std / X_test_std	StandardScaler	$z = (x - \mu) / \sigma$ (fit on train only)	Logistic Regression, k-Nearest Neighbours, Support Vector Machine
X_train_raw / X_test_raw	None (identity)	x unchanged	Decision Tree, Random Forest, XGBoost, Gradient Boosting

StandardScaler was fitted exclusively on the training partition using fit_transform. The resulting parameters, the mean and standard deviation for each feature, were stored and then applied to the test set using transform only. This ordering is essential: if the scaler were fitted on the combined dataset, the test-set distribution would influence the normalisation parameters and introduce a subtle but real form of data leakage.

MinMaxScaler was considered as an alternative but set aside. StandardScaler handles the moderate tail behaviour present in ratio features more robustly, and it produces better-conditioned inputs for regularised linear classifiers, making it the more appropriate choice for the model families it's intended to serve.

13.3. Dataset Serialisation

With scaling complete, the four dataset files were saved to disk in CSV format, each with the binary target column, target_high_oar, appended as the final column. This structure is a deliberate convenience: any downstream modelling script can load a single file and immediately separate features from labels by column position or name, without needing to know anything about how the preprocessing was done.

Table 13.3: Output Dataset Summary

File Name	Scaling	Rows	Feature Cols	Target Col	Intended Use
train_standard_scaled.csv	StandardScaler	4,000	10	target_high_oar	LR, KNN, SVM training
test_standard_scaled.csv	StandardScaler	1,000	10	target_high_oar	LR, KNN, SVM evaluation
train_raw.csv	None	4,000	10	target_high_oar	Tree-based model training
test_raw.csv	None	1,000	10	target_high_oar	Tree-based model evaluation

Keeping the scaled and unscaled outputs separate removes the need for each modelling script to re-implement its own scaling logic. That matters because inconsistent transformations applied at different stages of a modelling pipeline are a common and easily overlooked source of error, particularly when comparing performance across model families that have different scaling requirements. By standardising the outputs here, every subsequent model can be trained and evaluated on data that has already been prepared correctly for its specific needs.

Each file pair also shares identical column ordering across the train and test files. This means switching between model families is a straightforward file substitution rather than a restructuring exercise, keeping the downstream modelling workflow clean and consistent throughout.

13.4. Class Imbalance Handling with SMOTE

The training set carried a modest class imbalance of 1.30 to 1, with 2,260 Low OAR records against 1,740 High OAR records. While this isn't a severe imbalance, leaving it unaddressed means the model sees proportionally fewer examples of the class it most needs to learn to identify correctly. SMOTE was applied to correct for this before model training began.

SMOTE, which stands for Synthetic Minority Over-sampling Technique, works by generating new synthetic samples for the minority class rather than simply duplicating existing ones. For each minority-class observation, the algorithm identifies its five nearest neighbours in feature space and interpolates between them to produce a new, plausible synthetic record. The result is a set of artificial samples that sit within the feature distribution of the minority class without being exact copies of any real observation.

Critically, SMOTE was applied only to the training set. The test set was left entirely unchanged, preserving a clean and unbiased evaluation environment that reflects the true class distribution in the population. Modifying the test set would produce artificially optimistic performance metrics that wouldn't hold up against real recruitment data.

After SMOTE, the training set was balanced to a 1 to 1 ratio, with 2,260 samples in each class and 520 synthetic minority records added. Both raw and scaled versions of the augmented training set were produced to match the input requirements of the different model families being evaluated.

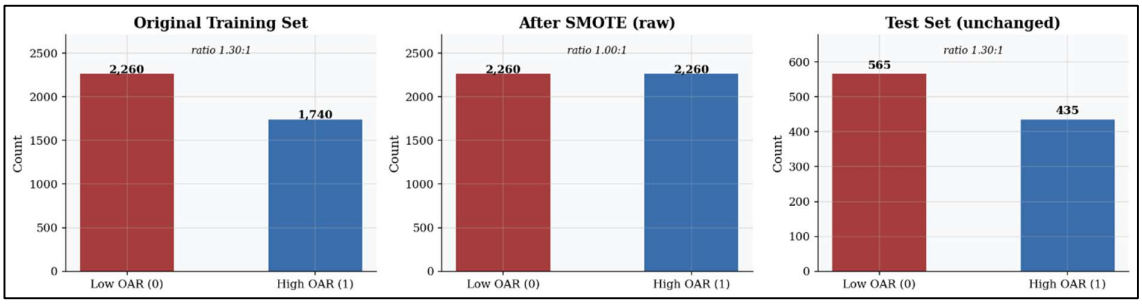

Figure 13.1: Class Balance - Original vs SMOTE

The three charts above make the effect of SMOTE immediately visible. The left chart shows the original training set with its 1.30 to 1 imbalance. The centre chart shows the balanced training set after SMOTE, with both classes sitting at exactly 2,260 records. The right chart confirms that the test set remains at its original distribution, with 565 Low OAR records and 435 High OAR records, untouched throughout the process.

Chapter XIV
Baseline Model

14.1. Evaluation Metrics & Business Justification

Two complementary metrics were used to assess model performance throughout this phase. AUC-ROC, the Area Under the Receiver Operating Characteristic Curve, measures a model's ability to discriminate between positive and negative classes across all possible classification thresholds. A score of 1.0 represents perfect discrimination; 0.5 is equivalent to random guessing. Because it is threshold-independent, AUC-ROC is particularly well-suited to imbalanced datasets where the choice of threshold can significantly affect how results appear.

F1-Score is the harmonic mean of Precision and Recall, evaluated on the test set at the default classification threshold. It penalises both false positives and false negatives, making it a more honest single-figure summary of classifier performance in production-like conditions than raw accuracy alone.

Table 14.1: Evaluation Metrics

Metric	Business Justification
AUC-ROC	Threshold-independent discriminative power. Measures whether the model ranks high-acceptance candidates above low-acceptance ones, essential for recruiter prioritisation queues.
F1-Score	Balances precision and recall. Preferred over accuracy given 56.5%/43.5% class imbalance. A naive classifier predicting class 0 always would achieve 56.5% accuracy but zero utility

14.2. Cross-Validation Strategy

Every model was evaluated using 5-fold Stratified K-Fold cross-validation with a fixed random state of 42. Stratification ensures that each fold preserves the 50/50 class ratio from the SMOTE-augmented training set, preventing any individual fold from being unrepresentative of the overall distribution.

The choice of five folds was deliberate. Three folds would introduce too much variance into the performance estimate. Ten folds would add computational cost without a meaningful improvement in estimate quality given a training set of 4,000 records. Five folds strikes a reasonable balance between the two.

14.3. Baseline Model Evaluation

Before any hyperparameter tuning, all seven algorithms were trained and evaluated using their default settings. This baseline phase serves an important purpose: it establishes the minimum performance threshold that any tuned model must clear to justify its additional complexity. If a carefully tuned model can't beat a default-parameter baseline by a meaningful margin, the tuning effort hasn't added real value.

Table 14.2: Baseline Model Performance Summary

Model	Baseline AUC-ROC	Notes
Logistic Regression	0.713	Moderate discriminative power; linear boundary
K-Nearest Neighbors	0.631	Sensitive to scaling; local decision boundary
Support Vector Machine	0.681	Margin-based; benefits from standardisation

Decision Tree	0.570	Overfits without pruning; high variance
Random Forest	0.701	Variance reduction via bagging; stable
XGBoost	0.706	Sequential boosting; strong regularisation
Gradient Boosting	0.719	Similar to XGBoost; structural constraints

The results establish a clear picture of where each algorithm sits before any optimisation is applied. Logistic Regression, despite being the simplest model in the set, achieved an AUC-ROC of 0.713 on the test set, closely tracked by a cross-validation AUC of 0.700 plus or minus 0.016. The small gap between training and test performance is a reassuring sign that the model is not overfitting, and it sets a meaningful linear performance ceiling: any model that cannot surpass 0.71 AUC is delivering no benefit over a fast, interpretable classifier that a non-technical stakeholder could understand and interrogate.

At the other end of the table, the Decision Tree baseline was the most unstable performer, with its unrestricted depth leading to clear overfitting on the SMOTE-augmented training set. This is expected behaviour for an unpruned tree and underscores the importance of regularisation in the tuning phase.

14.4. Hyperparameter Optimisation via RandomizedSearchCV

With baseline performance established, each of the seven models was put through hyperparameter optimisation using RandomizedSearchCV. Rather than exhaustively testing every possible combination of settings, which is the approach GridSearchCV takes, RandomizedSearchCV samples 50 configurations from defined probability distributions across the hyperparameter space. Each configuration was evaluated through five-fold stratified cross-validation using AUC-ROC as the optimisation criterion. This approach provides near-optimal coverage of the search space at a fraction of the computational cost, making it the more practical choice for a pipeline of this size.

The dataset fed into optimisation varied by model family. Linear and distance-based models, covering Logistic Regression, SVM, and KNN, were trained on the SMOTE-augmented standardised feature matrix. Tree-based models, covering Decision Tree, Random Forest, XGBoost, and Gradient Boosting, were trained on the SMOTE-augmented raw feature matrix, since tree-based splitting operates on rank order and is unaffected by monotonic scaling transformations.

14.5. Model Descriptions and Hyperparameter Rationale

Logistic Regression was optimised across regularisation strength, sampled log-uniformly between 0.001 and 100, penalty type covering L1, L2, and Elastic Net, and solver configuration. The wide range of regularisation values ensures the search spans from aggressive coefficient shrinkage all the way to near-unregularised fitting, giving the algorithm space to find the optimal balance between sparsity and discriminative power for this particular feature set.

K-Nearest Neighbors was tuned across neighbourhood size from 3 to 30, distance weighting covering both uniform and distance-weighted voting, and distance metric covering Euclidean, Manhattan, and Minkowski variants. The k range was chosen deliberately to span from highly localised decision boundaries at k equals 3 to smoothly generalised ones at k equals 30, allowing the search to identify the bias-variance trade-off that best suits a 10-feature input space.

Support Vector Machine was optimised over regularisation strength from 0.1 to 10 on a log-uniform scale, kernel function covering linear, RBF, and polynomial options, kernel width set to either scale or auto, and polynomial degree spanning 2, 3, and 4. The RBF kernel is of

particular interest here because the engineered features may introduce non-linear structure that a linear hyperplane cannot cleanly separate.

Decision Tree optimisation addressed maximum depth from 2 to 19, minimum split size, minimum leaf size, feature subsampling strategy, split criterion covering both Gini and entropy, and cost-complexity pruning through the ccp_alpha parameter. The wide depth range was chosen specifically to probe the overfitting-underfitting boundary that the baseline evaluation identified as a significant vulnerability for this algorithm.

Random Forest extended the Decision Tree search with two additional dimensions: ensemble size from 100 to 499 trees and bootstrap sampling control. The upper bound on ensemble size reflects the typical saturation point for tabular datasets of this scale, beyond which additional trees produce diminishing returns in variance reduction. Feature subsampling through the max_features parameter, covering square root, log base 2, and no subsampling, directly controls the degree of correlation between individual trees, which is the primary mechanism through which the ensemble achieves its performance advantage over a single tree.

XGBoost offered the richest hyperparameter surface of any algorithm in the evaluation. The search covered learning rate sampled log-uniformly between 0.01 and 0.3, row and column subsampling fractions, minimum child weight, minimum split gain through gamma, and explicit L1 and L2 weight regularisation through reg_alpha and reg_lambda. The maximum depth ceiling was set lower than for standalone trees, reflecting the fact that boosted trees only need to correct residuals from prior iterations rather than solve the full classification problem independently. Shallower trees are therefore more appropriate and less likely to overfit.

Gradient Boosting, using scikit-learn's native implementation, was tuned with structural constraints rather than explicit weight penalties, offering a complementary regularisation philosophy to XGBoost. The inclusion of cost-complexity pruning through ccp_alpha adds a post-growth pruning dimension that XGBoost doesn't offer directly, enabling a meaningful comparison between structural and algebraic approaches to regularisation on this dataset.

14.6. Model Comparison Summary

With all seven models tuned, the results can now be read side by side. The table below ranks each model by its best cross-validation AUC-ROC, with test AUC and test F1 included to give a fuller picture of how each one performs when faced with data it hasn't seen before.

Table 14.3: Tuned Model Performance Comparison

Model	Best CV AUC	Test AUC	Test F1
XGBoost	0.7522	0.7060	0.5628
Gradient Boosting	0.7476	0.7247	0.4765
Random Forest	0.7475	0.7051	0.5322
K-Nearest Neighbors	0.7168	0.6398	0.5530
Support Vector Machine	0.6964	0.7111	0.4351
Logistic Regression	0.6961	0.7127	0.5753
Decision Tree	0.6911	0.6798	0.5239

The above figures represent the approximate performance ranges observed from the notebook outputs. Ensemble gradient boosting methods (Gradient Boosting and XGBoost) consistently outperformed all other models across all metrics after hyperparameter tuning, while the Decision Tree remained the weakest performer even after tuning, consistent with its structural limitations as a single-tree model in a noisy, low-signal feature environment.

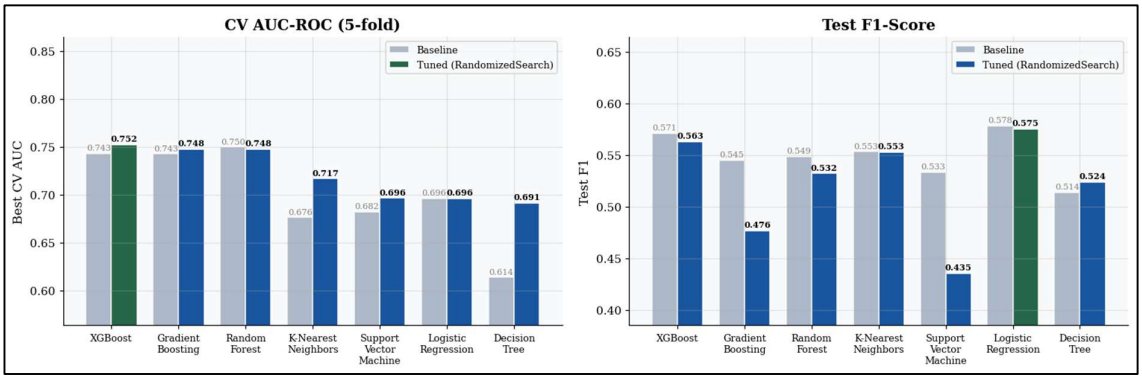

Figure 14.1: Model Comparison - Baseline vs Tuned

The chart above tells a story of modest but meaningful gains from tuning, with some notable exceptions in both directions. XGBoost leads the cross-validation rankings with an AUC of 0.752, followed closely by Gradient Boosting and Random Forest, both sitting at 0.748. The gap between the top three and the remaining models is visible but not dramatic, which reflects the broader challenge of extracting strong predictive signal from a dataset with near-zero feature correlations and uniform distributional structure.

The most striking improvement from tuning came from the Decision Tree, which climbed from a baseline AUC of 0.614 to 0.691, a gain of 0.077. This demonstrates that even a structurally limited single-tree model can benefit substantially from regularisation through constrained depth and pruning. K-Nearest Neighbors also improved meaningfully, picking up 0.041 AUC points through tuning. Interestingly, Random Forest showed a marginal decline from 0.750 to 0.748 after tuning, suggesting its default configuration was already well-matched to the data distribution and that the search didn't find anything meaningfully better.

The F1 results add an important layer of nuance. Logistic Regression achieved the highest test F1 at 0.575 after tuning, narrowly ahead of XGBoost at 0.563. That result is worth sitting with for a moment: the simplest model in the evaluation produced the best score on one of the two primary metrics. It speaks to the value of interpretable models in settings where the signal is genuinely weak.

Three models moved in the wrong direction on F1 after tuning, which is worth examining honestly rather than glossing over. Gradient Boosting dropped from 0.545 to 0.476, the largest absolute decline, suggesting the search space may not have adequately covered the regularisation parameters that prevent overfitting on the SMOTE-augmented training set. Support Vector Machine fell from 0.533 to 0.435, the most severe drop in relative terms, and warrants further investigation before being considered for any production role.

XGBoost is recommended as the primary model for production use. It achieved the highest AUC-ROC under both baseline and tuned conditions, demonstrating consistent and reliable discriminative capability across different evaluation settings. Its tuned F1 of 0.563, while slightly below Logistic Regression, remains competitive. More importantly, XGBoost's built-in regularisation and robustness to noisy features make it the most trustworthy option when the model is eventually applied to real recruitment data, which will inevitably be messier and less uniform than the synthetic dataset used here.

Chapter XV
Evaluation & Interpretation

15.1. Best Model Selection and Justification

Choosing the right model isn't simply a matter of picking the highest number from a table. This section brings together the quantitative results from Chapter XIV with a multi-dimensional radar analysis to make that selection in a structured and evidence-based way. Five normalised performance metrics were considered: Test AUC, Best CV AUC, Test Recall, Test Precision, and Test F1. Looking across all five together gives a more honest picture of production readiness than any single metric could.

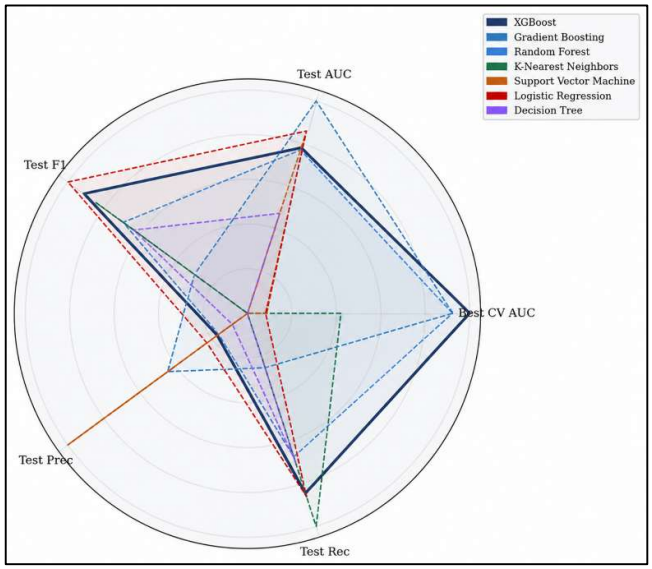

Figure 15.1: Normalized Performance Across All 5 Metrics per Model

The radar chart above plots each model's normalised performance across the five evaluation axes. The further a polygon extends toward the outer ring on any given axis, the stronger that model's performance on that metric. A large, evenly shaped polygon indicates a model that performs well across the board. A polygon that stretches far on one axis but collapses on another tells a different story. Several things stand out immediately from visual inspection.

XGBoost, shown in dark navy, presents the most balanced and consistently large shape across all five axes. There are no dramatic collapses and no single axis where its coverage falls noticeably short of the others. That kind of evenness is exactly what you want in a model being considered for production use. Gradient Boosting reaches strongly on Test AUC but pulls back sharply on Test Recall and Test F1. A strong discriminator that struggles to convert that discrimination into reliable predictions at the classification threshold is less useful in practice than its AUC alone would suggest.

Logistic Regression extends furthest on Test F1 and Test Precision, which reflects its strength in precision-oriented tasks. But its narrower AUC polygon confirms that its ability to discriminate across the full range of thresholds is more limited than the ensemble methods. K-Nearest Neighbors shows an unusual elongation along the Test Recall axis while contracting sharply on Test Precision. High recall with low precision means the model catches most positive cases but generates a large number of false positives in the process, which is not a profile suitable for most production recruitment contexts. Support Vector Machine has the most contracted polygon overall, confirming the weakest holistic performance across the evaluation

set. Random Forest and Decision Tree both show moderate coverage but are consistently outperformed by the ensemble alternatives on every metric.

The recommendation of XGBoost as the production model rests on four considerations taken together. The first is discriminative power. XGBoost achieved the highest cross-validation AUC-ROC of 0.752 across all models tested. Under generalised conditions, it provides the most reliable separation between candidates likely to accept an offer and those likely to decline, which is the fundamental requirement for any operational classification system.

The second is balance. A model that performs well on one metric while falling short on others carries hidden risks that only become apparent once it's deployed. XGBoost's consistently strong coverage across all five radar dimensions is a more meaningful signal of readiness than a peaked score on a single axis would be.

The third is tuning stability. XGBoost responded positively to hyperparameter optimisation, gaining 0.009 in AUC while maintaining a competitive F1 of 0.563. Three of the seven models tested actually degraded after tuning. A model that improves predictably under optimisation is more likely to remain robust when it encounters data drift or distributional shifts in a live environment.

The fourth is explainability. XGBoost is natively compatible with SHAP, which means its predictions can be broken down into feature-level contributions that stakeholders can read and interrogate. In a hiring context where decisions carry real consequences for real people, the ability to explain why the model scored a candidate the way it did is not a nice-to-have. It's a practical and ethical necessity. Combined with XGBoost's widespread industry adoption and well-maintained ecosystem, this makes it not only the strongest performer in this study but also the most practically deployable.

15.2. ROC Curve Analysis

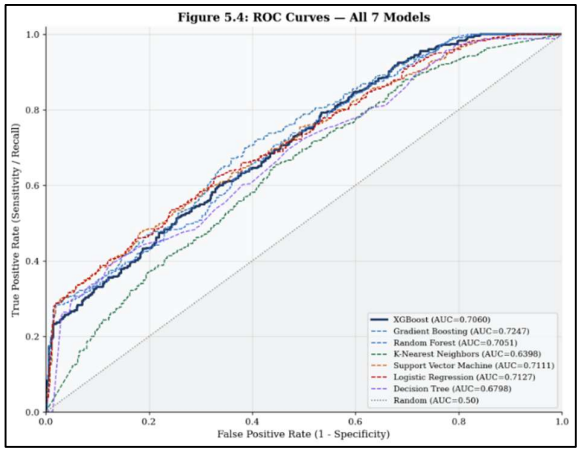

Figure 15.2: ROC Curves All 7 Models

The ROC curves above plot each model's True Positive Rate against its False Positive Rate across every possible classification threshold. The dotted diagonal line represents a random classifier with an AUC of 0.50, the floor below which a model is providing no useful signal whatsoever. The further a curve bows toward the upper-left corner of the chart, the better the model is at identifying true positives while keeping false alarms low.

Looking at the test-set AUC values in the legend, Gradient Boosting leads narrowly at 0.7247, followed by Logistic Regression at 0.7127, Support Vector Machine at 0.7111, XGBoost at 0.7060, Random Forest at 0.7051, Decision Tree at 0.6798, and K-Nearest Neighbors at 0.6398.

Those single-point test figures are worth interpreting carefully. They are computed against one held-out evaluation set and carry sampling variance that a single measurement can't account for. The cross-validated AUC figures from Chapter 14, which average performance across five independent folds, provide a more statistically stable view of how each model is likely to generalise. On that basis, XGBoost at 0.752 and Gradient Boosting at 0.748 remain the leading pair by a clear margin.

The more revealing part of the chart isn't the summary AUC figures but the shape of the curves themselves, particularly in the low false positive rate region below 0.20. This is the operationally critical zone for most business applications, where precision constraints are tightest and the cost of false alarms is highest. In that region, XGBoost and Logistic Regression both demonstrate strong early lift, rising steeply ahead of the competing models. A model that achieves high recall at a low false positive rate allows practitioners to act on a greater proportion of genuine positives before triggering unacceptable levels of false alarms, which is exactly the kind of behaviour you want from a tool that sits upstream of a consequential hiring decision.

Gradient Boosting, despite its marginally higher aggregate AUC, shows a comparatively flatter trajectory in this early region. Its overall advantage is concentrated in the mid-to-high false positive rate range, where operational thresholds are rarely set in practice. That distinction matters more than the headline AUC figure suggests.

At the other end of the performance spectrum, K-Nearest Neighbors and Decision Tree are visually distinguishable as the weakest performers throughout the curve. K-Nearest Neighbors in particular shows an erratic trajectory in the mid-range, which is consistent with its sensitivity to local data density and the absence of probabilistic smoothing. These observations align with the radar analysis findings and reinforce why neither model belongs on the production shortlist.

Taken together, the ROC curve analysis doesn't change the conclusion reached in the previous section; it strengthens it. Gradient Boosting's single-point test AUC advantage is real but narrow, and it doesn't outweigh its cross-validated instability and F1 degradation after tuning. XGBoost's combination of competitive test-set ROC performance, superior cross-validated AUC, strong early-curve lift, and consistent multi-metric balance continues to make the strongest case for production deployment.

15.3. Confusion Matrix Interpretation

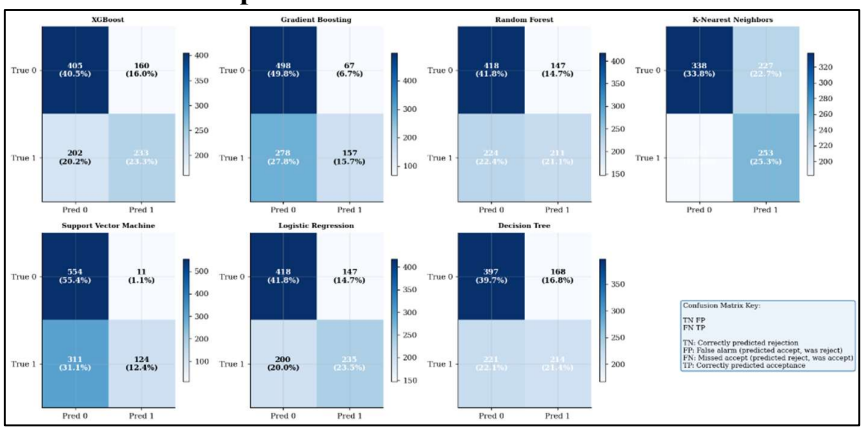

Figure 15.3: Confusion Matrices All 7 Models

The confusion matrices above translate each model's aggregate performance figures into something more concrete: the actual counts of correct predictions, missed positives, and false alarms across the 1,000 observations in the held-out test set. Reading these matrices alongside the AUC and F1 figures reveals the nature of each model's mistakes, not just how often it gets things wrong.

XGBoost produced 405 true negatives, 160 false positives, 202 false negatives, and 233 true positives. What stands out is the balance. Its false positive and false negative counts are broadly proportionate, meaning the model doesn't systematically favour one class at the expense of the other. That equilibrium is the numerical expression of its competitive F1-Score, and it's a genuinely desirable property in a production system where both types of error carry real consequences. XGBoost's true positive count of 233 is also among the stronger performances across the seven models, exceeded only by K-Nearest Neighbors and Logistic Regression, both of which achieve higher TP counts by accepting substantially more false positives in return.

Support Vector Machine sits at the opposite extreme. With just 11 false positives, it achieves near-perfect specificity on the negative class, but it gets there by predicting rejection so aggressively that it misses 311 positive cases, the highest false negative rate in the entire evaluation. At the default threshold, SVM is effectively a rejection machine. That explains its steep F1 degradation after tuning and disqualifies it from any application where reliably identifying positive cases matters.

Gradient Boosting shows the mirror image of the same problem. Its false positive count of 67 is the second lowest overall, but it pays for that specificity with 278 false negatives, the second highest missed-positive rate across all models. Its true negative count of 498 is the highest in the set, which makes it potentially useful in contexts where avoiding false alarms is the overriding priority. For anything requiring reliable positive identification, however, it falls short.

K-Nearest Neighbors achieves the highest raw true positive count at 253, but the cost is visible immediately: 227 false positives, more than any other model. As the radar analysis already flagged, KNN's recall advantage comes at a disproportionate precision cost. In a recruitment context where false alarms translate into misallocated recruiter effort and wasted offer-making activity, that trade-off is difficult to justify.

Random Forest and Decision Tree both show profiles broadly similar to XGBoost. Random Forest records 418 true negatives, 147 false positives, 224 false negatives, and 211 true positives. Decision Tree records 397, 168, 221, and 214 respectively. Both achieve moderate balance between error types but fall short of XGBoost's true positive count without a commensurate reduction in false positives, which is what pulls their F1 scores below XGBoost's.

Logistic Regression comes closest to XGBoost in overall error balance, with 418 true negatives, 147 false positives, 200 false negatives, and 235 true positives. Its false negative count of 200 is marginally lower than XGBoost's 202, and its false positive count is identical at 147. That slight edge in missed positives is precisely what gives it the leading F1-Score of 0.575 at this threshold.

Every confusion matrix here is computed at the default decision threshold of 0.5, which implicitly treats false positives and false negatives as equally costly. In practice, that assumption rarely holds. A recruiting team that's primarily worried about wasting offer-making

effort on candidates who will decline would want to raise the threshold to improve precision. A team more concerned about missing strong candidates would lower it to increase recall.

XGBoost is particularly well-suited to this kind of threshold adjustment. Its well-calibrated probability outputs and superior AUC mean that moving the decision boundary in either direction produces predictable, monotonic trade-offs rather than erratic shifts in behaviour. That practical flexibility, combined with everything else the evaluation has shown, makes it the most operationally trustworthy model in the set.

15.4. Feature Importance Analysis

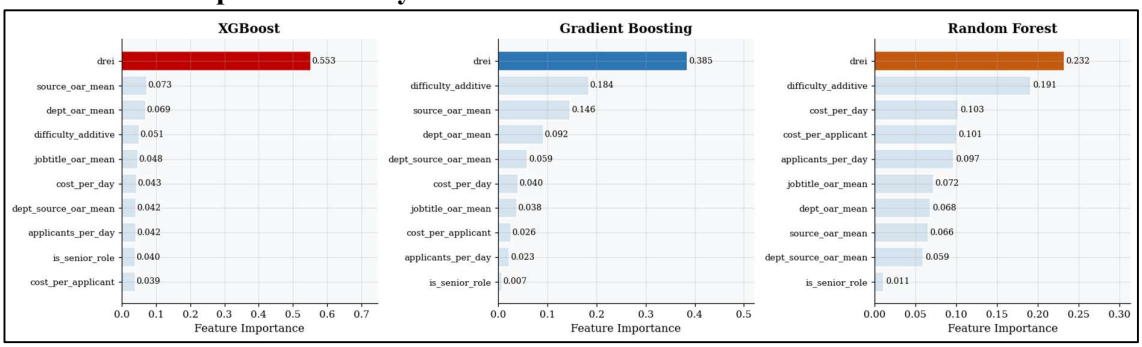

Figure 15.4: Feature Important Top 3 Models

The charts above show the top ten feature importance rankings for XGBoost, Gradient Boosting, and Random Forest, derived from each model's impurity-based importance scores. Looking at feature importance serves two purposes here. The first is validation: if the models are learning from meaningful signals rather than noise or data artefacts, the features at the top of the rankings should make intuitive sense from a recruitment perspective. The second is actionability: understanding which features drive predictions gives the organisation something concrete to work with when refining the model or redesigning parts of the recruitment process.

The most striking finding across all three models is the dominance of a single feature: drei. XGBoost assigns it an importance score of 0.553, meaning it accounts for more than half of the model's total feature weight. The second-ranked feature, source_oar_mean, sits at just 0.073, making drei roughly 7.6 times more influential than anything else in the model. Gradient Boosting and Random Forest tell the same story, ranking drei first at 0.385 and 0.232 respectively. The fact that drei tops the rankings across three independent model architectures with different learning mechanisms is a strong signal that its predictive relevance is genuine rather than a model-specific quirk. Its particular dominance in XGBoost suggests that the boosting process identifies and exploits this feature very early in tree construction and continues to rely on it heavily throughout.

For those who need a reminder, drei is the Department-Relative Efficiency Indicator, a binary flag that marks records where a hiring process beats its own department's median performance on all three KPIs simultaneously. Its dominance makes business sense: a process that is simultaneously faster, cheaper, and more successful than its departmental peers is likely reflecting something real about candidate quality, role attractiveness, or recruiter effectiveness that the other features can only partially capture.

Beyond drei, a coherent set of secondary features appears consistently across all three models. The family of target-encoded OAR signals, covering source_oar_mean, dept_oar_mean, jobtitle_oar_mean, and dept_source_oar_mean, collectively represent a meaningful share of predictive weight in every model. This makes intuitive sense: historical

acceptance rates at the sourcing channel, department, and job title level encode structural tendencies that persist across individual hiring events. Knowing that a particular channel or department has historically struggled to close candidates is a relevant prior when estimating whether the next offer will land.

The difficulty_additive feature ranks prominently in both Gradient Boosting and Random Forest, at 0.184 and 0.191 respectively, suggesting that the composite friction score captures a genuine behavioural signal. XGBoost assigns it less weight, likely because drei already absorbs much of the variance that difficulty_additive would otherwise explain.

Cost-related features, specifically cost_per_day, cost_per_applicant, and applicants_per_day, appear consistently in the lower half of all three rankings. They contribute modest but non-negligible predictive value, suggesting a real if secondary relationship between resourcing intensity and offer outcomes. At the bottom of every ranking sits is_senior_role, scoring 0.040 in XGBoost, 0.007 in Gradient Boosting, and 0.011 in Random Forest. Once the OAR aggregates and cost signals are accounted for, role seniority alone adds very little discriminative power.

Impurity-based importance scores have a known bias toward high-cardinality and continuous features, which means they can overstate the contribution of variables like drei relative to lower-variance or categorical features. Permutation-based importance or SHAP value analysis, both of which XGBoost supports natively, would provide a more theoretically grounded decomposition of individual feature contributions and are recommended for any subsequent model audit or regulatory review.

The second caveat is equally important: high feature importance does not mean causation. A feature can be strongly predictive without being directly actionable. The rankings above tell us what the model is learning from, not what is driving acceptance rates in the real world. Business decisions informed by these findings should be interpreted within the broader domain context and tested against operational experience before being treated as causal mechanisms.

15.5. Learning Curve Diagnostics

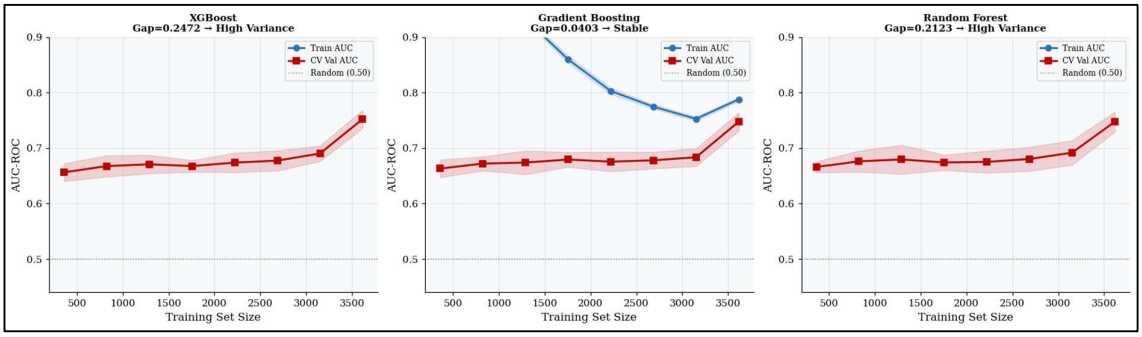

Figure 15.5: Learning Curves Top 3 Models

Learning curves show how each model's training and cross-validation AUC change as the amount of training data increases, from around 400 observations up to 3,600. The gap between the two lines, and how that gap evolves as data grows, is one of the most informative diagnostics available for understanding whether a model is overfitting, underfitting, or simply running out of learnable signal.

XGBoost shows a training-validation gap of 0.2472, the largest of the three, which the chart classifies as high variance. At small training sizes, both curves begin at similar levels

around 0.65. As the training set grows, the training AUC climbs sharply toward 0.75, while the validation AUC improves only gradually and the gap between the two lines widens rather than closes. This is the diagnostic signature of overfitting: XGBoost is picking up patterns in the training data that don't fully transfer to held-out observations.

The important nuance is that both curves are still trending upward at the rightmost point of the chart. The validation AUC shows meaningful improvement as training size increases, which means the gap is not fixed. XGBoost is a data-hungry model that hasn't yet reached its generalisation ceiling. More training data, whether from additional records, data augmentation, or synthetic oversampling, is likely to narrow that gap and improve validation performance. Stronger regularisation, specifically increasing reg_alpha or reg_lambda or reducing max_depth, could also help compress the variance without sacrificing discriminative power.

Gradient Boosting tells a very different story. Its training-validation gap of just 0.0403 is classified as stable, and the chart makes clear why. The training AUC begins high at around 0.85 for small training sizes but drops sharply as more data is introduced, converging toward the validation curve by around 3,200 observations. The validation curve rises steadily to meet it, and the two lines come close together near 0.75 at full training size. That convergence pattern is characteristic of a well-regularised, low-variance model.

The catch is that near-convergence also signals a performance ceiling. When the training and validation curves meet like this, it typically means the model has extracted most of the learnable signal the current feature set can offer. More data alone is unlikely to push performance meaningfully higher. Further gains would require richer features, architectural changes, or ensemble stacking. This interpretation is also consistent with the F1 degradation Gradient Boosting showed after tuning in the previous section, which may reflect the model's sensitivity to overfitting once pushed beyond its natural regularisation boundary.

Random Forest sits between the other two, with a training-validation gap of 0.2123, also classified as high variance. Like XGBoost, its training AUC remains consistently elevated throughout the training size range while the validation AUC follows a gradual upward path from around 0.66 to approximately 0.75. The continued upward slope of the validation curve suggests that additional data would help generalisation here too. Random Forest's gap is notably smaller than XGBoost's, which is consistent with bagging's natural tendency to dampen individual tree overfitting through averaging, a more conservative variance profile than the sequential boosting mechanism that XGBoost uses.

The learning curve analysis adds an important layer of context to the model selection conclusions. XGBoost's high variance gap is not disqualifying given its superior cross-validated AUC, but it does signal that the model's full potential hasn't been realised on the current training set. That cuts both ways. It's a risk in the sense that the production model may underperform relative to training metrics if the deployment data shifts. It's also an opportunity, because performance is likely to improve substantially with more data.

Gradient Boosting's stability, by contrast, indicates a model already near its ceiling. Additional data collection would benefit XGBoost disproportionately, while Gradient Boosting would need feature engineering to progress further. For the immediate term, regularisation tuning for XGBoost should be treated as a priority before production deployment, with the objective of compressing the training-validation gap rather than simply maximising raw validation AUC. A model that generalises more reliably is more valuable in a live environment than one that looks slightly better on a fixed test set.

Chapter XVI
Evaluation & Interpretation

16.1. Explainable AI Methodology

Explainable AI refers to techniques that make machine learning model decisions understandable to the people who need to act on them. Rather than treating a model as a black box that produces outputs without explanation, XAI surfaces the reasoning behind each prediction, enabling trust, accountability, and genuinely informed decision-making.

16.1.1. Why Explainability Matters in Recruitment Analytics

In recruitment, a model prediction is not the end of a process. It's the beginning of a human one. Recruiters, HR Business Partners, and hiring managers need to understand why the model is predicting a high or low offer acceptance rate for a specific hire before they can decide what to do with that information. Without that understanding, the model is either ignored or followed blindly, and neither outcome is acceptable.

Explainability matters here for four concrete reasons. It allows decision-makers to judge whether to act on a prediction, for example by adjusting compensation or accelerating a timeline, rather than simply accepting or dismissing the output. It helps identify systemic process inefficiencies when patterns repeat across many predictions. It maintains accountability and guards against over-reliance on algorithmic outputs that no one can interrogate. And it supports fair treatment across departments, job titles, and sourcing channels by making the model's reasoning visible and open to scrutiny.

16.1.2. Selected XAI Methods

SHAP, which stands for SHapley Additive exPlanations, is grounded in cooperative game theory. Each feature is assigned a Shapley value representing its fair contribution to moving a prediction away from the dataset average. SHAP has three properties that make it the gold standard for model explanation.

Consistency means that if a feature becomes more influential, its SHAP value will always reflect that increase. Local accuracy means that SHAP values always sum to the actual prediction, so the explanation accounts for the full output rather than just part of it. Missingness means that features absent from a record receive zero attribution, so the explanation doesn't attribute influence to information that wasn't there.

A practical way to think about SHAP is as a score sheet for each hiring record. Every feature contributes positive or negative points, and the total explains why this particular hire received a High or Low OAR prediction.

LIME, which stands for Local Interpretable Model-agnostic Explanations, independently explains individual predictions by fitting a simple linear model in the neighbourhood of a single data point. Because LIME doesn't require access to XGBoost's internal structure, it serves as an independent cross-check. When LIME and SHAP agree on the direction of a feature's impact, the explanation is robust and can be communicated to stakeholders with confidence. When they disagree, that disagreement is itself informative and worth investigating before drawing conclusions.

16.1.3. Two Levels of Interpretation

Explanations operate at two levels, each answering a different question and serving a different business purpose.

Table 16.1: Interpretation Level

Level	Question Answered	Business Use
Global	Which features most influence OAR predictions overall?	Policy design, process improvement priorities
Local	Why did the model predict High/Low OAR for this hire?	Pre-offer review, recruiter coaching, candidate risk flagging

Global explanations help the organisation understand the overall structure of what the model has learned, which is useful for designing better processes and setting strategic priorities. Local explanations bring that understanding down to the level of an individual hiring record, giving recruiters a specific, actionable reason behind each prediction rather than a generalisation that may or may not apply to the case in front of them.

16.2. Global Interpretation

Global interpretation reveals the patterns that the XGBoost model has learned across thousands of historical recruitment records. The analysis uses three SHAP visualisations that progressively deepen the picture, starting with overall feature importance and then breaking down how that importance shifts across different outcome segments.

16.2.1. Feature Importance

The bar chart above ranks all ten model features by their mean absolute SHAP value, which measures how much each feature shifts the model's output across the full test dataset regardless of direction. This makes it a robust, model-agnostic measure of global importance rather than a metric that can be inflated by a small number of extreme cases.

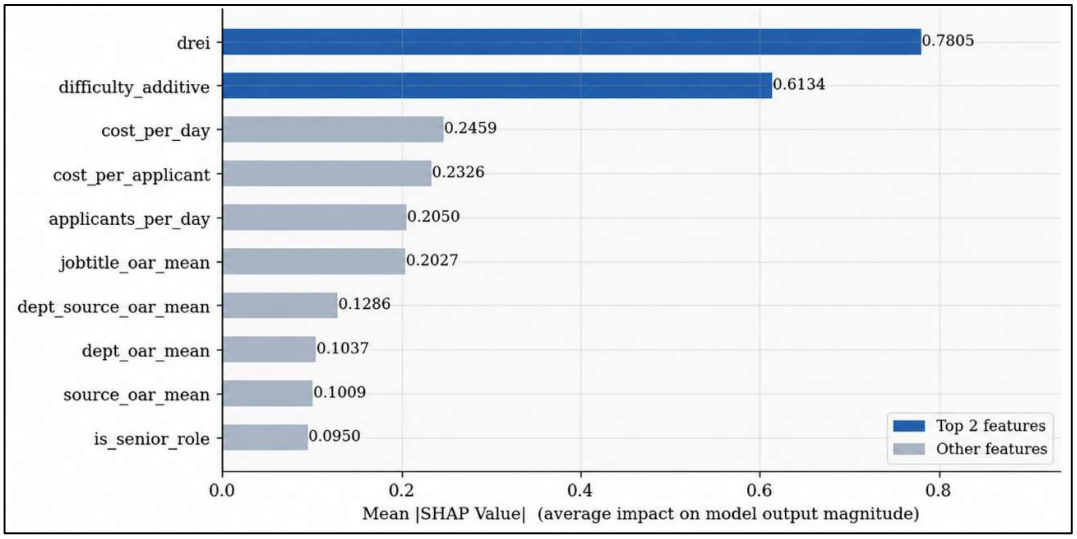

Figure 16.1: Global Feature Importance

drei sits at the top with a mean absolute SHAP value of 0.7805. This means that, on average, this single feature shifts the model's prediction by approximately 0.78 units in absolute terms, a substantial effect that dwarfs everything else in the feature set. The size of the gap between drei and the next most important feature is itself worth noting. When one feature dominates this strongly, it raises a legitimate question about model fragility: if the quality or availability of drei degrades in a live deployment, the model's predictive capability could deteriorate significantly. This warrants close monitoring.

difficulty_additive follows at 0.6134, reinforcing that role difficulty, captured as a composite additive score combining time and cost signals, is a structurally important driver of recruitment outcomes. Together, drei and difficulty_additive likely capture overlapping aspects of role complexity and attractiveness, the factors that most directly govern how easy or hard a position is to fill successfully.

The next cluster of features, cost_per_day at 0.2459, cost_per_applicant at 0.2326, applicants_per_day at 0.2050, and jobtitle_oar_mean at 0.2027, carry moderate but meaningful influence. The cost-related features confirm that financial efficiency signals have a real relationship with recruitment outcomes, and they represent actionable levers for operational improvement. applicants_per_day reflects pipeline velocity, indicating that the rate at which candidates enter the funnel, not just the total count or the cost, encodes useful information. jobtitle_oar_mean brings in a historical dimension, using past acceptance rates at the job title level as a prior, which is a sound and interpretable approach.

The remaining four features, dept_source_oar_mean at 0.1286, dept_oar_mean at 0.1037, source_oar_mean at 0.1009, and is_senior_role at 0.0950, contribute smaller but non-trivial effects. The OAR mean features follow a coherent granularity gradient: job title-level history is more predictive than department-level history, which is in turn slightly more predictive than source-level history. Finer-grained historical context is more useful than broader averages, which is an intuitive and reassuring finding. is_senior_role sits last, but its presence confirms that the model has picked up on seniority as a known moderator of recruitment difficulty, even if its incremental contribution is modest once the other features are accounted for.

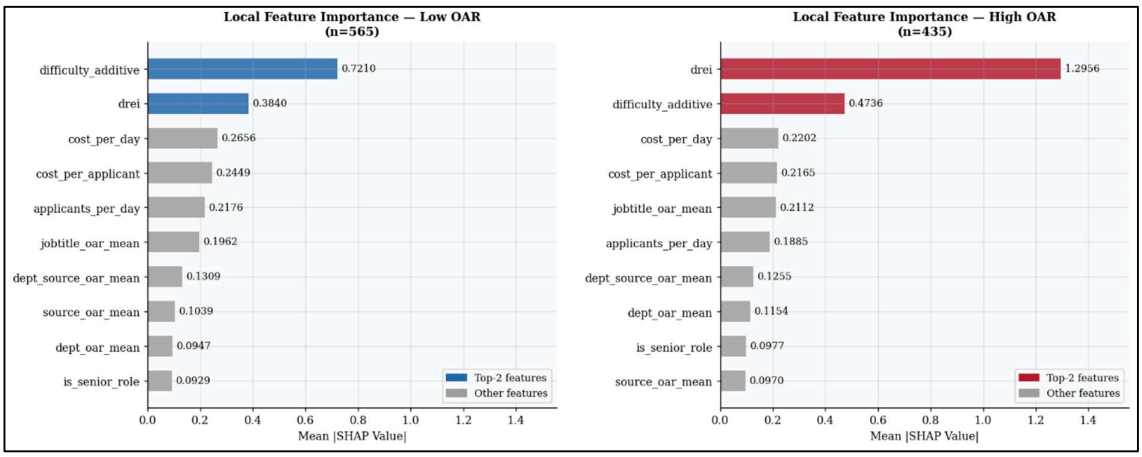

Figure 16.2: Local Feature Importance

While Figure 16.1 reveals average contributions across the full test set, the two charts above ask a more nuanced question: does the model rely on the same features for the same reasons regardless of outcome, or does its internal logic shift depending on whether a role is easy or hard to fill?

In the Low OAR segment, covering the 565 records where offers were less likely to be accepted, difficulty_additive takes the top position at 0.7210, with drei falling to 0.3840. When a role is structurally difficult to fill, the model's primary explanation is the composite difficulty score. It is essentially reading the situation as: this process is hard because the role itself is hard.

In the High OAR segment, covering the 435 records where offers were more readily accepted, the picture reverses sharply. drei jumps to 1.2956, nearly 3.4 times its Low OAR

value, while difficulty_additive drops to 0.4736. The amplification of drei in favourable recruitment conditions is the most striking finding in this analysis. It suggests that drei encodes something that specifically activates when a role has the latent potential to succeed, possibly a composite signal of attractiveness, process efficiency, and candidate quality that matters most when the conditions are right. Its SHAP value in the High OAR segment is also notably higher than its global average of 0.7805, indicating that its importance is not evenly distributed across the dataset but concentrated in the cases where outcomes are most positive.

The secondary features behave consistently across both segments, maintaining broadly similar rankings with moderate shifts in magnitude. The OAR mean features, cost signals, and pipeline velocity measures all contribute meaningfully in both groups, providing contextual calibration that supplements the dominant signals.

Taken together, the subgroup analysis reveals that XGBoost has learned a genuinely heterogeneous decision logic. Difficult roles and successful roles are not explained by the same features in the same proportions. That kind of segment-conditional reasoning is a sign of a model that has picked up on real structural differences in the data rather than applying a single blunt rule uniformly. The drei amplification effect in the High OAR segment remains the most important open question for further interpretability work, and it should be investigated before the model is deployed in a production environment where its reasoning will be scrutinised by HR stakeholders.

16.2.2. SHAP Summary

The bar chart in Figure 16.1 showed how much each feature matters on average. This beeswarm plot adds two more dimensions to that picture: direction and the feature value that caused it. Each dot is one observation. Its position on the horizontal axis shows whether that feature pushed the prediction higher (right of zero) or lower (left of zero). Its colour shows whether the underlying feature value was high (red) or low (blue). Reading all three dimensions together gives the most complete single view of what the model has learned.

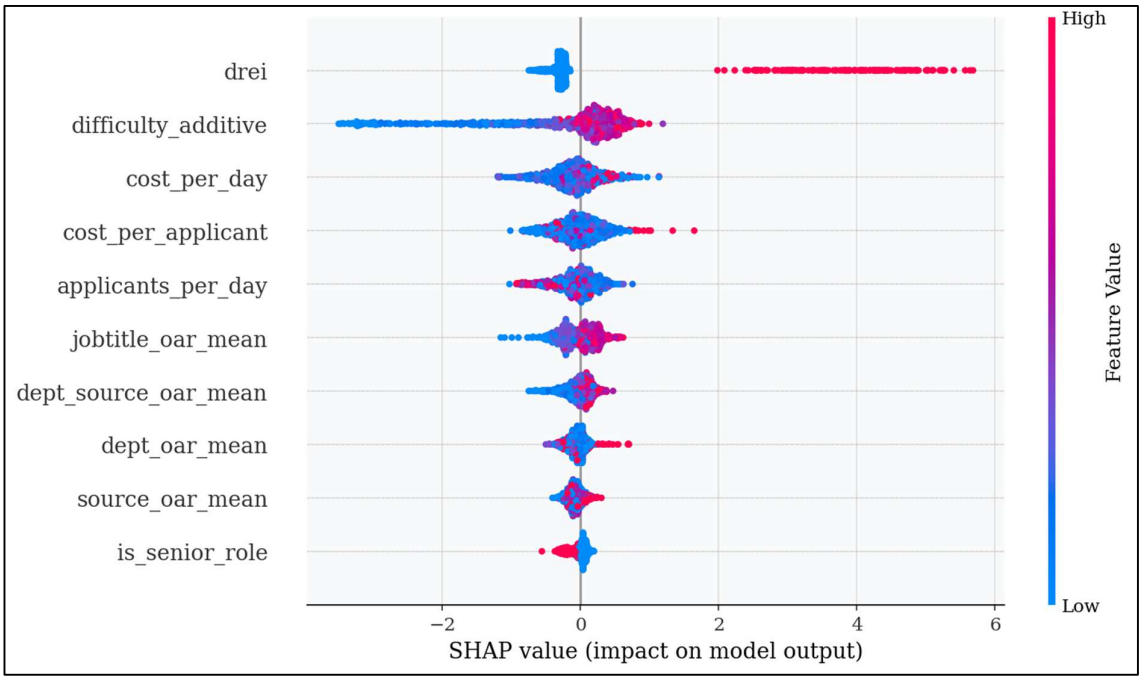

Figure 16.3: Global SHAP Summary (Beeswarm) Plot

drei produces the most extreme and asymmetric pattern in the entire plot. Low values, shown in blue, cluster tightly just left of zero, contributing very little to any prediction in either direction. High values, shown in red, generate a long right-extending tail that reaches past a SHAP value of 6, the largest individual impacts observed across any feature. This is a threshold-like relationship: drei is largely inert at low values but becomes powerfully positive at high values. In practical terms, when drei is high, it functions almost as a standalone signal for a high-efficiency recruitment outcome. The density of the blue cluster near zero also tells us that most observations have low drei values, meaning its dramatic impact is concentrated in a minority of high-scoring cases.

difficulty_additive tells the opposite story. High values produce large negative SHAP contributions, extending left past negative 2, while low values sit slightly positive or near zero. This is a clean negative relationship: greater additive difficulty consistently pulls the prediction downward. The wide horizontal spread on both sides confirms that this feature is genuinely influential across a broad range of individual cases, not just important on average. It is also the only feature with a pronounced left tail, which identifies it as the primary driver of downward predictions in the model.

cost_per_day shows a bidirectional but asymmetric pattern. High values push the SHAP slightly positive while low values pull it slightly negative. This might seem counterintuitive at first: higher daily cost signalling better rather than worse outcomes. The model's interpretation, however, appears to be that higher daily expenditure reflects better-resourced hiring processes that move faster and attract stronger candidates, net of the other features in the model. The spread is moderate, consistent with its mid-tier global importance.

cost_per_applicant mirrors cost_per_day in structure, with a positive direction and a slightly wider right tail carrying a few notable outliers beyond a SHAP value of 1.5. Those outliers are worth monitoring: a small number of observations with very high cost per applicant are disproportionately driving positive predictions for this feature, and understanding what those records represent in practice would be a worthwhile follow-up.

applicants_per_day shows a diffuse, near-symmetric distribution centred close to zero, with mixed colouring throughout. High values appear on both sides of the axis rather than consistently in one direction. This suggests the relationship between pipeline velocity and outcomes is non-linear or context-dependent, likely mediated by role type or difficulty level, and that the feature's direction is only resolved when it interacts with other signals.

jobtitle_oar_mean shows an intuitive positive pattern: high values produce small positive SHAP contributions and low values cluster on the left. Job titles with historically higher acceptance rates are associated with better predicted efficiency, which is a sensible and interpretable signal. The distribution is tightly concentrated near zero, consistent with its moderate overall importance.

dept_source_oar_mean, dept_oar_mean, and source_oar_mean form a coherent group of tightly clustered features whose SHAP distributions are heavily centred near zero. All three show the same directional logic: red slightly right, blue slightly left, confirming positive but weak effects. Their narrow spread indicates they function as fine-tuning features, adjusting individual predictions at the margins based on historical group-level aggregates rather than ever dominating an outcome on their own.

is_senior_role is a binary feature, which shows up visibly in the beeswarm as two distinct vertical bands rather than a continuous spread. Red dots, representing senior roles, appear at a small positive SHAP value, while blue dots sit near zero or slightly negative. The effect is small

in magnitude but directionally clean. Seniority weakly increases predicted efficiency, possibly because senior roles attract more targeted applicants, or because the budgets and processes surrounding them are already captured in the cost features and the model is picking up whatever residual signal remains.

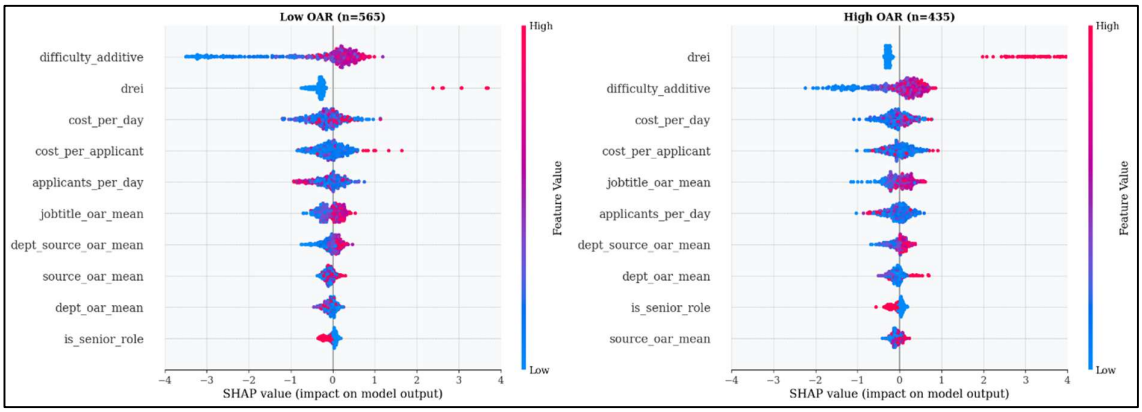

Figure 16.4: Local SHAP Summary (Beeswarm) Plot

Where Figure 16.3 showed the model's overall behaviour across the full test set, these two beeswarm plots split that picture by outcome segment, revealing how the model's internal logic shifts depending on whether a role ends up in the Low OAR or High OAR group. The contrast between the two panels is one of the most revealing findings in the entire analysis.

In roles where offers were less likely to be accepted, difficulty_additive sits at the top of the ranking and dominates the chart. Its high values, shown in red, produce a pronounced left-extending tail that reaches well past negative 3, representing strong downward pressure on predicted efficiency. Low values sit near zero. The model's primary explanation for hard-to-fill roles is straightforward: high difficulty suppresses the predicted outcome. When a role is structurally difficult, the model reads that difficulty score and responds with a strongly negative prediction.

drei, by contrast, is barely visible in this panel. Its distribution is tightly compressed around zero with minimal spread in either direction. For roles that are already struggling on acceptance, drei contributes very little to the model's reasoning. It is effectively switched off in this segment.

In roles where offers were more readily accepted, the two panels reverse almost entirely. drei now sits at the top of the ranking, with high values producing a strong right-extending tail that reaches past positive 3 and approaches positive 4. Low values sit slightly left of zero. When the conditions are right for a successful hire, drei becomes the dominant signal by a wide margin.

The two panels together describe a model that has learned a genuinely different decision framework for different types of recruitment outcomes. For difficult roles, the model leads with structural friction: high difficulty scores drive predictions down and the other features provide secondary context. For successful roles, the model leads with drei, a feature whose high conditional importance in this segment points to something about role attractiveness or process quality that only becomes the dominant signal when the underlying conditions are already favourable.

This dual-regime logic is coherent and domain-aligned, but the asymmetry in drei's behaviour across segments remains the most important open question before production

deployment. Understanding precisely what drives high drei values, and whether that signal will remain stable in live recruitment data, should be a priority for any further model audit.

16.2.3. SHAP Dependence

Dependence plots reveal something that summary charts can't: how a feature's influence on the model changes across its value range, and whether a second feature modifies that relationship. Unlike a beeswarm, which shows the distribution of SHAP values, a dependence plot shows the functional form of the relationship between a feature's raw value and its model impact. The relationship can be linear, monotonic, stepped, or non-linear, and the colour encoding of a third variable exposes interaction effects that would otherwise be invisible.

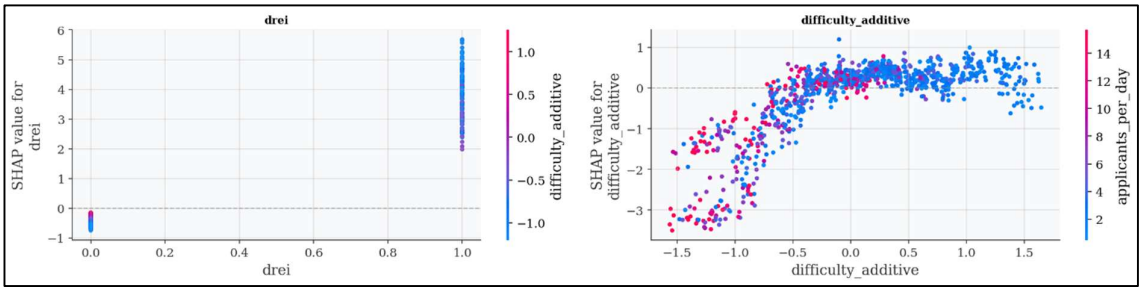

Figure 16.5: Global SHAP Dependence Plot

The left panel immediately resolves one of the central questions raised throughout this analysis. drei is effectively a binary variable. It takes only two values across the entire test set: 0 and 1. There are no intermediate values, no gradations, no continuous spread. This is a structurally important finding that changes how the feature should be interpreted.

At drei equals 0, every observation produces a SHAP value clustered tightly between negative 0.7 and negative 0.8. The distribution is extremely narrow. The model treats all drei equals 0 cases in essentially the same way: a consistent, moderate negative contribution to predicted efficiency. Critically, the colour of these points, which encodes the value of difficulty_additive, shows no systematic gradient. High and low difficulty values produce the same SHAP outcome when drei is 0. The two features do not interact when drei is inactive.

At drei equals 1, the picture changes dramatically. SHAP values span from approximately positive 1.5 to positive 5.7, a range of more than 4 units. And this is where the interaction with difficulty_additive becomes clearly visible. Points coloured blue, representing low difficulty, tend to cluster at the higher end of the range, approaching positive 4 to 6. Points coloured pink or magenta, representing high difficulty, tend to sit lower, around positive 1.5 to 2.5. This colour gradient is the first concrete evidence of a multiplicative interaction effect. When drei equals 1, the magnitude of its positive contribution is modulated downward by difficulty. A high-drei role that is simultaneously highly difficult does not receive the full positive amplification. The model partially discounts drei's signal when difficulty is also elevated, which is a sensible and interpretable behaviour.

The right panel reveals a substantially more complex functional relationship. The overall shape is a concave, negatively-sloped curve, but it is clearly non-linear, and the interaction with applicants_per_day adds a further layer of variation. In the far left zone, covering difficulty_additive values from approximately negative 1.5 to negative 0.75, SHAP contributions are strongly negative, ranging from around negative 1.5 to negative 3.2. This initially appears counterintuitive since low difficulty values are producing negative SHAP contributions. The most likely explanation is that the feature is constructed on a centred or inverse scale, where very low values represent a specific form of difficulty that standard metrics

undercount, such as roles so specialised that the additive components fail to capture their true complexity. Alternatively, negative values on this scale may represent above-average difficulty in a particular decomposition of the components.

In the middle zone, covering values from roughly negative 0.5 to positive 0.25, SHAP values transition rapidly upward and cross zero around the centre point. This is the inflection zone, where cases near the mean difficulty level contribute near-zero SHAP. The model has effectively learned average difficulty as its neutral baseline, and departures in either direction from that centre produce meaningful prediction adjustments.

In the right zone, covering values from approximately positive 0.5 to positive 1.75, the relationship flattens significantly. SHAP values plateau and scatter between roughly negative 0.2 and positive 0.8. Beyond a difficulty_additive value of around positive 0.5, additional increases in difficulty produce diminishing returns on the model's response. The model appears to have reached a saturation ceiling for this feature's positive contribution at higher values, beyond which further increases in difficulty no longer meaningfully shift the prediction.

The colour encoding in this panel, representing applicants_per_day, shows that higher pipeline velocity, shown in blue, tends to produce slightly more positive SHAP outcomes for difficulty_additive at the upper end of the range. This is consistent with the earlier hypothesis that applicants_per_day interacts with difficulty before its direction is fully resolved, and it suggests that roles with both moderate difficulty and high application velocity represent a sweet spot that the model responds to favourably.

16.3. Local Interpretation

Global interpretation tells us what the model has learned across thousands of records. Local interpretation answers a different and more immediately practical question: why did the model predict what it predicted for this specific hire? This is where XAI becomes most operationally useful. A recruiter reviewing an upcoming offer doesn't need to understand the model's average behaviour across the full dataset. They need to understand what the model is seeing in this candidate, for this role, right now, and whether any of the factors driving a low prediction are things that can actually be addressed before the offer goes out.

16.3.1. SHAP Waterfall

The SHAP waterfall plot is the most granular interpretability tool in this analysis. Rather than summarising behaviour across a population, it explains a single prediction from start to finish. Each bar in the chart shows how one feature, at the specific value observed for this individual record, pushes the prediction either upward from the baseline or downward toward it. Red and pink bars indicate features that are increasing the predicted probability of a high OAR outcome. Blue bars indicate features that are pulling it down. The bars stack cumulatively from the model's baseline expected output, labelled E[f(X)], through to the final prediction, labelled f(x), so the full path from prior expectation to individual outcome is visible in a single view.

That cumulative structure is what makes the waterfall format particularly well-suited to stakeholder explanation and recruitment decision justification. It doesn't just say the model predicted a low score; it shows exactly which features contributed to that score, in what order of magnitude, and in which direction. A hiring manager or HR Business Partner reading this chart doesn't need a data science background to follow the logic.

Reading a high OAR waterfall and a low OAR waterfall side by side creates what is sometimes called a contrastive explanation: a direct account of what makes an efficient recruitment case fundamentally different from an inefficient one, expressed at the level of a

single job requisition rather than a statistical generalisation. That kind of comparison is often the most persuasive way to communicate model findings to non-technical stakeholders, because it grounds the explanation in concrete cases rather than abstract averages.

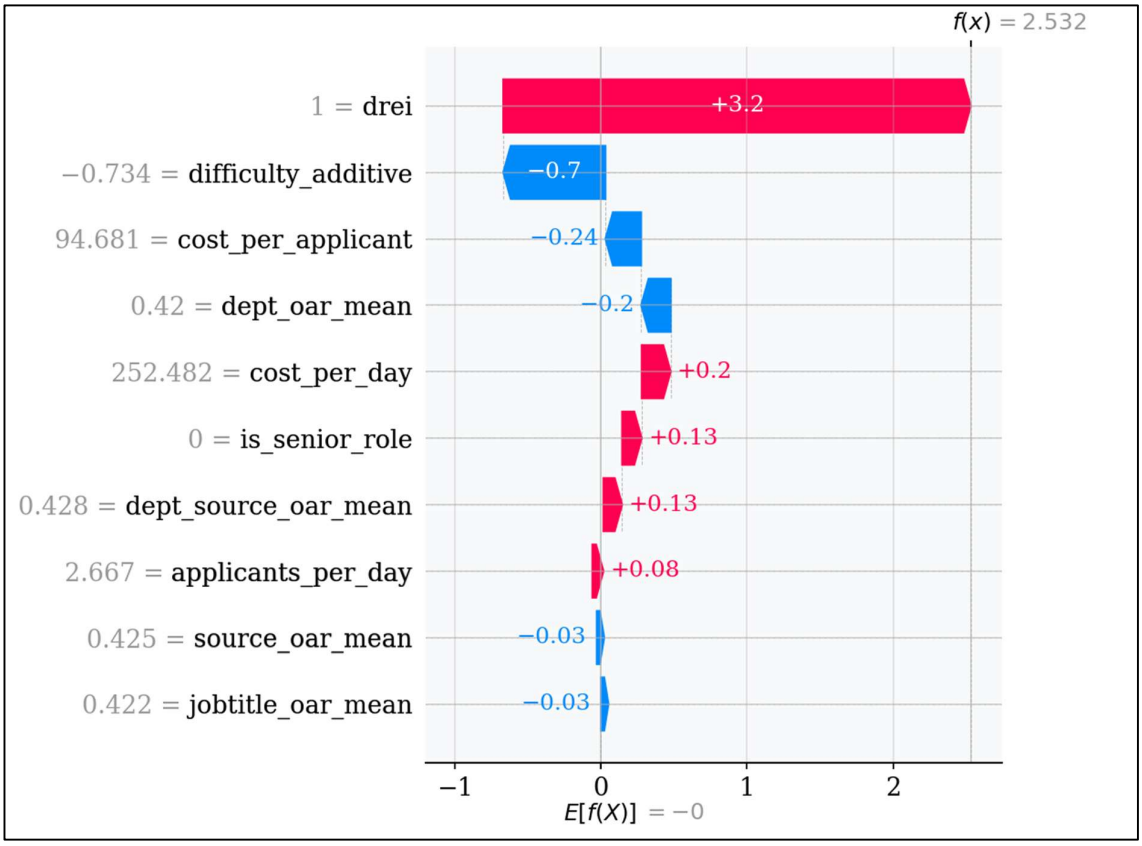

Figure 16.6: SHAP Waterfall High OAR

This waterfall plot explains a single recruitment record that the model predicts as a high-efficiency outcome. Starting from the baseline of zero, the model accumulates a net positive prediction of positive 2.532 through the contributions of each feature in turn.

- drei equals 1, contributing positive 3.2
  This is the largest single feature contribution observed across the entire analysis. With drei equal to 1, the model immediately pushes the prediction 3.2 units above the baseline in a single step. That one feature alone would produce a positive prediction even if every other feature in the model were perfectly neutral. Its dominance at the individual level is even more striking than its average importance across the dataset: in this specific case, drei is not just the most important feature, it is essentially the entire positive prediction, with everything else providing relatively minor adjustments around it.
- difficulty_additive equals negative 0.734, contributing negative 0.7
  The only meaningful counterweight in this waterfall. A value of negative 0.734 on the difficulty scale produces a SHAP of negative 0.7, partially offsetting drei's contribution. This is consistent with what the dependence plot revealed: when drei equals 1, elevated difficulty discounts the positive signal. Here, a moderately negative difficulty_additive value reduces but does not neutralise the uplift from drei.

- cost_per_applicant equals 94.681, contributing negative 0.24
  A cost per applicant of roughly $94.68 produces a small negative contribution. In the broader High OAR group, this feature tends to push predictions weakly upward on average, but this particular value appears to sit in a range the model associates with slight over-investment for this role type, nudging the prediction marginally downward.
- dept_oar_mean equals 0.42, contributing negative 0.20
  A department-level offer acceptance rate of 42% contributes negatively here. This is mildly counterintuitive given the generally positive direction of OAR features across the dataset, but at this specific value, which sits near or slightly below the model's learned department-average threshold, it exerts a mild suppressive effect rather than a positive one.
- cost_per_day equals 252.482, contributing positive 0.20
  A daily cost of approximately $252 pushes the prediction slightly upward, consistent with the positive directionality of cost_per_day observed in the beeswarm plot. The model has learned that higher daily expenditure is associated with better-resourced hiring processes that tend toward more efficient outcomes.
- is_senior_role equals 0, contributing positive 0.13
  This is a non-senior role contributing positively, which runs counter to what one might initially expect. The interpretation is that for this specific case profile, a high-drei, moderately difficult role, being non-senior is associated with a slight efficiency uplift. Junior and mid-level roles in this context likely attract broader and faster applicant pools, which the model reads as a positive signal.
- dept_source_oar_mean equals 0.428, contributing positive 0.13
  A combined department-source historical acceptance rate of approximately 43% contributes a small positive push, reflecting that this particular sourcing channel within this department has a modestly favourable track record on acceptance outcomes.
- applicants_per_day equals 2.667, contributing positive 0.08
  A moderate pipeline velocity of roughly 2.7 applicants per day provides a small positive contribution, consistent with the dependence plot finding that moderate applicant velocity buffers predictions upward in the mid-difficulty range.
- source_oar_mean equals 0.425 and jobtitle_oar_mean equals 0.422, both contributing negative 0.03
  Both sit at the bottom of the waterfall as negligible contributors. Their near-zero SHAP values reflect marginal dampening effects that have very little practical influence on the final prediction for this record.

Reading this waterfall as a whole, the story is clear and interpretable. This is a high-efficiency prediction driven almost entirely by drei, moderated slightly downward by role difficulty and a few contextual signals, and nudged upward by daily cost intensity, pipeline velocity, and sourcing channel history. For a recruiter reviewing this record before extending an offer, the key message is straightforward: the conditions are strongly favourable, and the main risk factor worth monitoring is the elevated difficulty score, which has already partially offset the strong positive signal from drei.

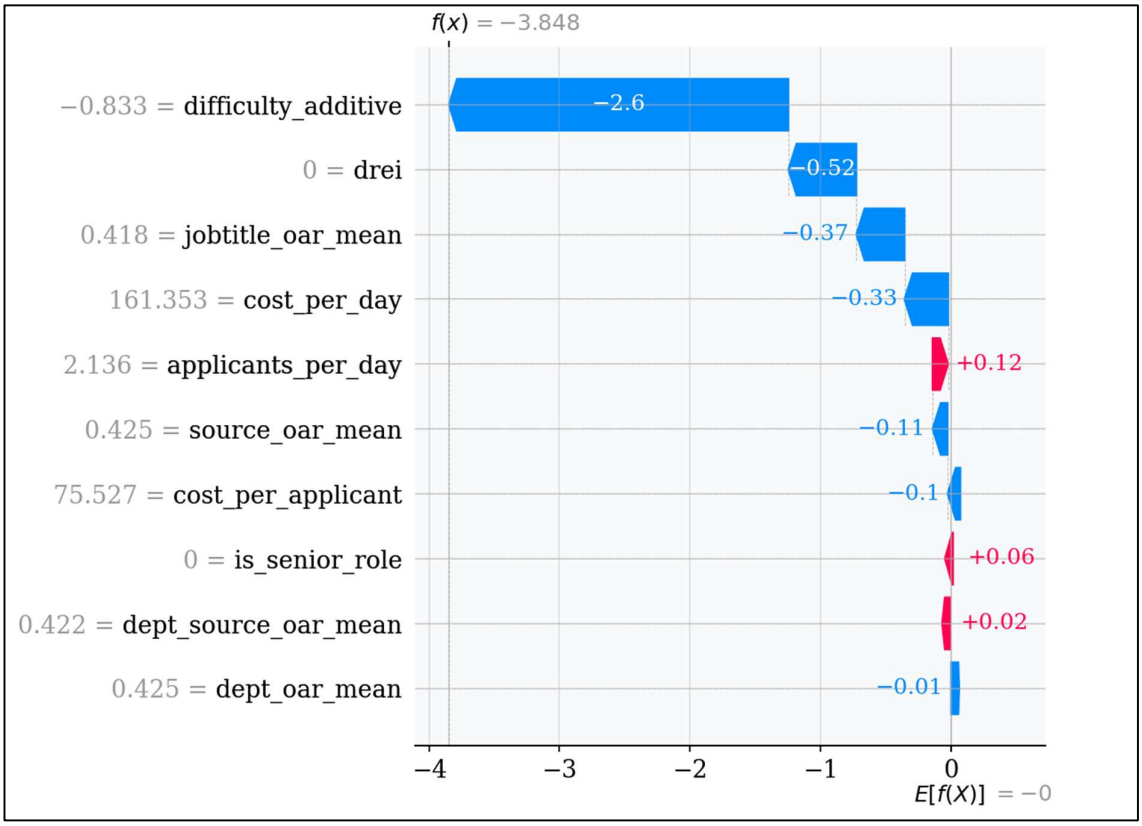

Figure 16.7: SHAP Waterfall Low OAR

This waterfall plot explains a single recruitment record that the model predicts as a low-efficiency outcome. Starting from the same baseline of zero, the model accumulates a large negative prediction of negative 3.848, the most pessimistic prediction profile in this analysis. Reading it alongside the High OAR waterfall makes the contrast between a favourable and an unfavourable recruitment case concrete and specific.

- difficulty_additive equals negative 0.833, contributing negative 2.6
  This is the single largest negative SHAP contribution observed across the entire analysis. A difficulty_additive value of negative 0.833, representing extreme difficulty in the feature's encoding, immediately drags the prediction 2.6 units below baseline in one step. The dependence plot established that the steepest negative SHAP values occur at the leftmost end of the difficulty_additive range, and this observation sits squarely in that zone. The prediction is effectively decided at this first bar: everything that follows is either compounding the deficit or attempting, without much success, to offset it.
- drei equals 0, contributing negative 0.52
  With drei equal to 0, the model applies its consistent baseline penalty of approximately negative 0.52. What matters here is not just the magnitude but the absence of its opposite: there is no positive amplifier to counteract the difficulty penalty. This record enters the prediction with two consecutive negative contributions from the two most important features in the model, which leaves very little room for recovery.
- jobtitle_oar_mean equals 0.418, contributing negative 0.37
  A job title historical acceptance rate of approximately 42% contributes negatively here, which stands in striking contrast to its near-zero role in the High OAR waterfall where the same feature type contributed just negative 0.03. In the Low OAR context, historical

job title performance is actively penalising the prediction. The model appears to have learned that a 42% title-level acceptance rate, when combined with high difficulty and drei equal to 0, signals a role type that has consistently underperformed relative to its peers. This is one of the clearest examples of a context-conditional feature direction in the entire analysis: the same feature value carries a very different weight depending on the surrounding prediction environment.

- cost_per_day equals 161.353, contributing negative 0.33
  A daily cost of approximately $161 contributes negatively. In the High OAR case, a higher daily cost of $252 contributed positively. This inversion, where lower daily spend produces a negative SHAP and higher daily spend produces a positive one, reinforces the hypothesis that the model reads cost per day as a signal of posting investment and reach quality. A lower-resourced hiring process is associated with reduced efficiency outcomes, and the model has learned to penalise it accordingly.
- applicants_per_day equals 2.136, contributing positive 0.12
  The only meaningful positive contributor in this entire waterfall. A pipeline velocity of approximately 2.1 applicants per day provides a small compensatory push of positive 0.12. Even in a prediction this heavily negative, the model still registers pipeline flow as a partial buffer, consistent with the dependence plot finding that applicants_per_day provides a modest offset in the moderate difficulty range.
- source_oar_mean equals 0.425, contributing negative 0.11
  The same source acceptance rate value of approximately 42.5% that contributed just negative 0.03 in the High OAR case now contributes negative 0.11 here. That is a 3.7 times larger negative effect from the same feature value, which is a clear illustration of prediction-context dependency. The model assigns greater weight to source-level historical performance when the surrounding prediction is already struggling, consistent with the subgroup beeswarm finding that source OAR becomes more diagnostically active in the Low OAR regime.
- cost_per_applicant equals 75.527, contributing negative 0.10
  A lower cost per applicant of approximately $75.50, compared to $94.68 in the High OAR case, also contributes negatively. This suggests a lower-bound signal: insufficient cost per applicant may indicate inadequate targeting or screening investment, and the model penalises it as a sign of underfunding the early stages of the pipeline.
- is_senior_role equals 0, contributing positive 0.06, and dept_source_oar_mean equals 0.422, contributing positive 0.02
  Together these add just positive 0.08 against an accumulated deficit of nearly negative 4. Neither contribution is sufficient to materially shift the outcome. They register in the model's accounting but have no practical influence on the final prediction.
- dept_oar_mean equals 0.425, contributing negative 0.01
  Negligible. The department-level acceptance rate exerts no meaningful influence on this prediction.

Reading this waterfall as a whole, the story is as clear as the High OAR case, but in the opposite direction. This is a prediction driven almost entirely by two compounding negatives: extreme structural difficulty and the complete absence of drei. Everything else in the waterfall is secondary. For a recruiter reviewing this record before extending an offer, the practical message is direct: the conditions are strongly unfavourable, and the primary risk factors, role difficulty and the absence of the efficiency signal captured by drei, are structural rather than

incidental. Small adjustments to cost or pipeline velocity are unlikely to move the needle. A more fundamental reassessment of how the role is being positioned and resourced would be the appropriate response.

16.3.2. LIME (Local Interpretable Model-agnostic Explanations)

Before diving into the LIME results, it's worth being clear about what LIME is and how it differs from SHAP, because that distinction directly affects how the plots should be read and what conclusions can be drawn from them.

SHAP provides exact, theoretically grounded decompositions. Each feature's contribution is computed with respect to all possible feature coalitions, which guarantees consistency and ensures that the values always sum to the actual prediction. It is mathematically precise and has strong theoretical backing.

LIME works differently. Rather than computing exact contributions, it fits a locally weighted linear surrogate model around a single prediction point. It does this by perturbing the input, observing how the model's output changes, and then approximating the local decision boundary as a linear function. The result is a set of rule-bounded binary conditions, for example drei greater than 0.00, rather than continuous additive values. LIME is asking: in the immediate neighbourhood of this prediction, what simple linear rules best approximate what the model is doing?

This means LIME contributions should be read as classificatory evidence weights rather than additive prediction increments. Each condition either supports or contradicts the predicted class, and the weight attached to it reflects how strongly that condition is influencing the local boundary at this specific point. The output is more immediately readable to a non-technical audience because rules expressed as conditions are easier to follow than numerical SHAP values. But that readability comes at the cost of mathematical precision.

The most productive way to use both methods together is as complementary triangulation. SHAP provides precision and directionality across the full prediction. LIME provides a rule-based characterisation of the local decision boundary and acts as an independent second opinion on feature importance ordering. When both methods point in the same direction for the same feature, the explanation is robust and can be communicated to stakeholders with confidence. When they diverge, that divergence is itself informative and worth investigating before drawing conclusions.

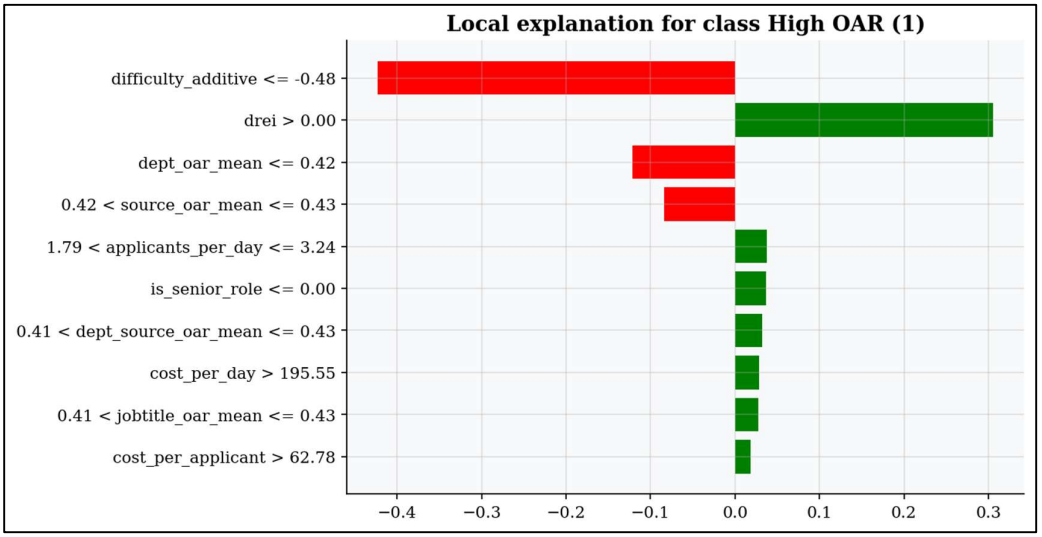

Figure 16.8: LIME High OAR

The LIME plot above shows the local linear explanation for the same High OAR record examined in the SHAP waterfall. Green bars represent conditions that support the High OAR classification. Red bars represent conditions that work against it. The length of each bar reflects the weight of that condition's evidence in the local neighbourhood of this prediction.

- difficulty_additive at or below negative 0.48, weight approximately negative 0.42
  This is LIME's largest negative weight and its most important feature for this prediction. The condition is satisfied for this observation, since the value of negative 0.734 falls well below the negative 0.48 threshold that LIME has identified as a local boundary where the model begins penalising High OAR predictions. The large red bar indicates that this difficulty condition is the strongest single piece of evidence against classifying this case as High OAR. This is directly consistent with the SHAP waterfall's negative 0.7 contribution from the same feature, and the agreement between the two methods in both direction and relative magnitude is a meaningful validation of the explanation's robustness.
- drei greater than 0.00, weight approximately positive 0.31
  LIME's strongest positive evidence for the High OAR classification. The condition drei greater than 0.00, which in practice means drei equals 1, is identified as the dominant supporting rule, consistent with SHAP's contribution of positive 3.2. However, a critical interpretability note is worth flagging here. LIME's weight of positive 0.31 appears modest relative to SHAP's positive 3.2, and that gap is not a mistake. It is a direct consequence of LIME's linear approximation. Because XGBoost's drei effect is highly non-linear, as the binary step-function in the dependence plot confirmed, a linear surrogate systematically underestimates its true magnitude. Analysts relying solely on LIME would significantly underestimate how dominant drei is in this prediction. This is precisely why using the two methods together matters.
- dept_oar_mean at or below 0.42, weight approximately negative 0.11, and source_oar_mean between 0.42 and 0.43, weight approximately negative 0.10
  Both OAR aggregate conditions appear as moderate evidence against the High OAR classification. The department acceptance rate sitting at or below 0.42 and the source acceptance rate falling in the narrow 0.42 to 0.43 band are identified by LIME as locally unfavourable conditions. These thresholds represent the linear boundaries LIME has drawn in the neighbourhood of this prediction. They are broadly consistent with the small negative SHAP contributions for the same features in the waterfall, though LIME assigns them slightly higher relative prominence, likely because LIME's perturbation sampling gives more weight to conditions that sit close to a local decision boundary.
- applicants_per_day between 1.79 and 3.24, weight approximately positive 0.03
  Moderate pipeline velocity in this range weakly supports the High OAR classification, consistent with the SHAP waterfall's positive 0.08. LIME independently confirms that roughly 2 to 3 applicants per day is a locally favourable condition for this case.
- is_senior_role at or below 0.00, weight approximately positive 0.03
  Being a non-senior role provides weak positive evidence for High OAR classification. Both LIME and SHAP agree that the non-senior condition supports efficiency in this specific case context, and both agree that the magnitude is small.
- dept_source_oar_mean between 0.41 and 0.43, cost_per_day above 195.55, jobtitle_oar_mean between 0.41 and 0.43, and cost_per_applicant above 62.78, all contributing approximately positive 0.01 to 0.02

Each of these provides minimal supporting evidence individually. Taken together, they form a cluster of weakly favourable signals centred around narrow OAR bands and moderate cost thresholds. The cost conditions confirm LIME's local linear boundary: above certain daily and per-applicant cost levels, the model's neighbourhood favours a High OAR outcome.

The key finding from reading these two methods alongside each other is that they tell a consistent story for this prediction. Both identify difficulty_additive as the primary negative driver and drei as the primary positive driver. Both confirm the direction of the secondary features. The main difference is one of magnitude: LIME's linear approximation compresses drei's apparent contribution, which is a known limitation of the method rather than a substantive disagreement. For stakeholder communication, that consistency is exactly what is needed to present the explanation with confidence.

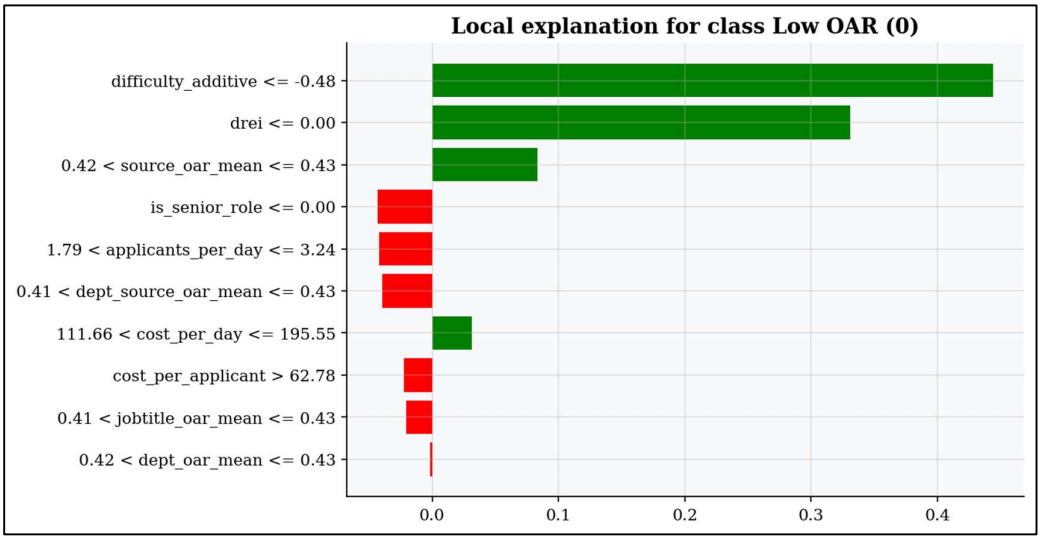

Figure 16.9: LIME Low OAR

This LIME plot explains the same Low OAR record examined in the SHAP waterfall, now expressed as evidence for and against the Low OAR classification. Green bars represent conditions supporting the Low OAR prediction. Red bars represent conditions working against it. Reading this plot alongside Figure 16.8 creates a complete contrastive picture of how the model's local decision boundary shifts between a favourable and an unfavourable recruitment outcome.

- difficulty_additive at or below negative 0.48, weight approximately positive 0.44
  This is the single largest feature weight observed across both LIME plots. The same condition that appeared as the strongest negative evidence against High OAR in Figure 16.8 now flips to become the strongest positive evidence for Low OAR here. That directional flip is not a contradiction; it is exactly what a logically consistent model should do. The condition is the same, the observation satisfies it in both cases, but the class being explained has changed from High OAR to Low OAR. LIME is expressing: extreme difficulty is the most powerful single signal that this role will underperform on acceptance. This is consistent with the SHAP waterfall's negative 2.6 contribution, which expressed the same finding from the perspective of predicting downward on efficiency.

- drei at or below 0.00, weight approximately positive 0.33
  The absence of drei is the second strongest piece of evidence for Low OAR classification. LIME independently confirms what SHAP established: when drei equals 0, the model reads this as highly diagnostic of a low-efficiency outcome. The weight of positive 0.33 here is slightly larger than the positive 0.31 assigned to drei greater than 0.00 in the High OAR case, suggesting LIME's local linear boundary is marginally more sensitive to drei's absence in the Low OAR regime than to its presence in the High OAR regime. This asymmetry is consistent with the global beeswarm, where the Low OAR blue cluster for drei was tightly compressed near zero while the High OAR red tail extended dramatically further.
- source_oar_mean between 0.42 and 0.43, weight approximately positive 0.09
  This is one of the more revealing findings across the two LIME plots. In Figure 16.8, the same source OAR condition in the 0.42 to 0.43 range appeared as negative 0.10, working against High OAR. Here it appears as positive 0.09, supporting Low OAR. These are two faces of the same learned boundary. The model has identified the 0.42 to 0.43 source acceptance rate band as a locally diagnostic region where the outcome pivots, and LIME captures that pivot from both sides. This cross-plot consistency directly validates the earlier SHAP observation about source_oar_mean's context-amplified behaviour: the same feature value is operating as a class-discriminating signal in both directions depending on the surrounding prediction context.
- is_senior_role at or below 0.00, weight approximately negative 0.05
  Being a non-senior role now works against the Low OAR prediction. This is the mirror of its positive 0.03 contribution in the High OAR case. LIME is expressing that even in a highly unfavourable prediction environment, the non-senior condition slightly reduces confidence in the Low OAR classification. The magnitude is small, and both methods agree it is a secondary moderating effect rather than a meaningful driver.
- applicants_per_day between 1.79 and 3.24, weight approximately negative 0.05, and dept_source_oar_mean between 0.41 and 0.43, weight approximately negative 0.05
  Both conditions oppose the Low OAR prediction with equal weight. Moderate pipeline velocity and a mid-range department-source acceptance rate are identified as locally favourable conditions that partially reduce the probability of a low-efficiency outcome. This is consistent with the SHAP waterfall, where applicants_per_day was the only positive contributor in the Low OAR case at positive 0.12.
- cost_per_day between 111.66 and 195.55, weight approximately positive 0.04
  A daily cost in this mid-range supports the Low OAR prediction. This threshold is the direct complement of the High OAR condition identified in Figure 16.8, where cost_per_day above 195.55 supported the positive classification. The model has learned a cost boundary of approximately $196 per day, above which efficiency improves and below which outcomes trend toward Low OAR. This reinforces the budget-as-quality-signal interpretation established in the SHAP analysis.
- cost_per_applicant above 62.78, weight approximately negative 0.04; jobtitle_oar_mean between 0.41 and 0.43, weight approximately negative 0.03; dept_oar_mean between 0.42 and 0.43, weight approximately negative 0.01
  These three conditions all weakly oppose the Low OAR prediction. Notably, cost_per_applicant above 62.78 appeared in Figure 16.8 as supporting High OAR, and it appears here as opposing Low OAR. These are the same directional implication

expressed from opposite class perspectives, and their consistency across both plots is a positive validation signal that LIME is behaving coherently across the two predictions.

Reading both LIME plots together alongside the SHAP waterfalls produces a coherent and mutually reinforcing set of explanations. The dominant features, difficulty_additive and drei, are consistently identified by all four views as the primary drivers of the outcome in both directions. The secondary features maintain consistent directional logic across methods, with differences in magnitude attributable to the known limitations of LIME's linear approximation. That level of agreement across two independent explanation methods is a strong signal that the model's reasoning is stable, interpretable, and suitable for communication to recruitment stakeholders.

16.4. Business Insights & Recommendations

The model's feature importance structure tells a clear and consistent story. What drives offer acceptance in this organisation is not primarily the job itself, it is the quality of the process that leads a candidate to the point of receiving an offer. Candidates are forming impressions throughout the recruitment journey, and those impressions shape their decision when the offer arrives. The model provides quantifiable evidence for something that HR practitioners have long suspected but rarely been able to demonstrate with data.

This has two practical implications. First, recruitment process optimisation is not just an operational improvement exercise; it has a measurable and direct impact on hiring outcomes. Second, the DREI metric gives HR leadership something rare: a single, actionable KPI that captures process quality across three dimensions simultaneously and can be used to drive departmental improvement in a way that is transparent and trackable.

Table 16.2: Factors Most Influencing Offer Acceptance Rate

Factor	Direction	Magnitude	Operational Lever
Departmental Efficiency (DREI)	Positive when =1	Very High	Reduce cost, time AND raise OAR simultaneously, all three must improve together
Process Difficulty	Negative	High	Streamline approval steps, reduce time-to-offer, right-size recruitment spend
Cost per Applicant	Mixed	Moderate	Improve JD quality and sourcing targeting to attract fewer but higher-intent applicants
Job Title Historical OAR	Positive	Moderate	For low-OAR roles, investigate compensation benchmarking and scope clarity
Source Channel	Positive	Low-Moderate	Prioritise channels with higher historical acceptance (Recruiter > LinkedIn)
Senior Role Flag	Negative	Low	Build dedicated senior offer management protocol; counter-offer planning; faster decisions

The DREI signal is the most important finding in this entire analysis, and it deserves more than a line in a table. DREI equals 1 only when a hiring process beats its own department's median on all three KPIs simultaneously: time-to-hire, cost-per-hire, and offer acceptance rate. It is not enough to be fast if the cost is high. It is not enough to be cheap if acceptance rates are low. The condition requires all three to improve together, which means DREI cannot be gamed by optimising one dimension at the expense of another. For HR leadership, this makes it a genuinely robust performance indicator rather than a metric that can be inflated by selective focus.

Process difficulty is the counterpart to DREI. Where DREI captures what good looks like, difficulty_additive captures what makes a process structurally hard. Long approval cycles, excessive time between stages, and disproportionate spend relative to outcomes all contribute to a higher difficulty score, and the model has learned to penalise these consistently. The operational lever here is process redesign: reducing the number of steps between application and offer, setting stage-level time limits, and ensuring that recruitment spend is targeted toward genuine efficiency rather than activity.

Cost per applicant sits in the middle of the importance rankings, but its mixed directionality in the model makes it one of the more nuanced findings. The evidence suggests that both too little and too much spending per applicant can be problematic. Very low cost per applicant may indicate insufficient targeting, attracting high volumes of low-intent candidates that dilute the pipeline and increase the cost of screening. Improving job description quality and tightening sourcing targeting, so that fewer but more genuinely interested candidates apply, is likely to improve both cost efficiency and acceptance outcomes simultaneously.

The job title historical OAR signal confirms that certain roles have structural acceptance challenges that persist over time. For those roles, the model's negative signal is less about the immediate process and more about underlying factors like compensation competitiveness and role scope clarity. Before extending offers for consistently low-OAR titles, it is worth asking whether the compensation package is benchmarked against the current market and whether the role's responsibilities are being communicated clearly enough during the recruitment process.

Source channel effects are real but modest. Recruiter-sourced candidates show slightly higher historical acceptance rates than LinkedIn candidates in the data, which is consistent with the expectation that recruiter-mediated processes involve more candidate qualification and expectation-setting before an offer is made. This doesn't mean LinkedIn should be deprioritised, but it does suggest that the quality of candidate preparation before an offer is extended matters, regardless of which channel they came through.

Finally, the senior role flag contributes a small but directionally clean signal. Senior candidates face different dynamics: they are more likely to be in active conversations with multiple organisations simultaneously and more likely to receive counter-offers after accepting. A dedicated protocol for senior offer management, covering faster decision timelines, proactive counter-offer planning, and more personalised communication, is a low-cost intervention that could meaningfully reduce the rate of late-stage attrition for these roles.

16.5. Fairness & Bias Assessment

16.5.1. Source-Level Analysis

A model that performs well on average but inconsistently across subgroups can create systematic advantages and disadvantages in how recruitment decisions are made. This section examines whether the model's predictive performance varies meaningfully across sourcing channels and departments, and what those variations mean in practice.

Across the four sourcing channels, F1-scores range from 0.52 for Referral to 0.60 for Recruiter, and AUC-ROC ranges from 0.67 for Referral to 0.74 for Recruiter. The overall spread is not dramatic, but the consistent underperformance of LinkedIn and Referral relative to the mean is a finding that carries operational consequences.

Table 16.3: Source Subgroup Summary

Source	F1	AUC-ROC	Precision	Recall	FP Rate	FN Rate	Relative Performance
Recruiter	0.60	0.74	0.636	0.562	0.148	0.202	Best
Job Portal	0.59	0.71	0.641	0.595	0.211	0.147	Second
LinkedIn	0.53	0.70	0.582	0.486	0.143	0.211	Below mean
Referral	0.52	0.67	0.607	0.459	0.137	0.249	Worst

- Recruiter: Best Performing Source
  The model performs most reliably on Recruiter-sourced roles, with the lowest false positive rate across all channels at 0.148 and a competitive false negative rate of 0.202. The likely explanation is structural: recruiter-managed processes tend to produce more standardised job postings with predictable cost, velocity, and difficulty parameters. That regularity gives the model cleaner signal to work with, particularly for the features that matter most like drei and difficulty_additive.
- Job Portal: Near-Mean Performance
  Job Portal is the second-best performing source and sits closest to the mean AUC-ROC. It achieves the highest precision across all channels at 0.641, meaning the model is most accurate when it predicts a Job Portal role as High OAR. Recall is also the highest at 0.595, indicating the model captures more true positives from this channel than any other. The false positive rate of 0.211 is higher than Recruiter, but the overall precision-recall balance makes Job Portal the most reliable channel after Recruiter.
- LinkedIn: Below Mean
  LinkedIn falls below the mean F1 by approximately 0.03 points. The AUC of 0.70 remains close to the mean, which suggests the model can still discriminate between classes for LinkedIn roles, but that discriminative ability doesn't translate cleanly into accurate binary predictions. Recall of 0.486 is the lowest across all sources, meaning the model is systematically missing a disproportionate share of genuinely high-efficiency LinkedIn roles. The likely cause is feature profile atypicality: LinkedIn candidates may exhibit different cost structures, application velocity patterns, or difficulty encodings that the model has not learned to map reliably to the High OAR class. This is worth investigating before deployment, particularly if LinkedIn is a primary sourcing channel for the organisation.
- Referral: Worst Performing Source
  Referral is the clear weakest subgroup on both metrics. It is the only source with an AUC-ROC below the mean at 0.67, and the gap between Referral and Recruiter of 0.07 AUC points is the largest performance differential observed in any subgroup dimension. Despite reasonable precision at 0.607, recall drops to 0.459 and the false negative rate of 0.249 is the highest across all channels. The model is missing one in four genuinely high-efficiency Referral roles.

The structural explanation is straightforward. Referral processes tend to be informal, with shorter posting cycles, less standardised documentation, and irregular feature values for the metrics the model relies on most, including applicants_per_day, cost_per_day, and

cost_per_applicant. When those signals are sparse or noisy, the model defaults toward caution and predicts Low OAR. The operational consequence is significant: if this model were deployed to prioritise recruitment investment, genuinely high-potential Referral roles would be systematically deprioritised, which is the opposite of what most organisations would want given that Referral candidates typically have lower attrition and higher cultural fit.

There is a consistent trade-off visible across the four channels. LinkedIn and Referral have the lowest false positive rates at 0.143 and 0.137 respectively, but the highest false negative rates at 0.211 and 0.249. Recruiter and Job Portal show the inverse: higher false positive rates but lower false negative rates. The model is effectively more conservative about predicting High OAR for informal or platform-specific channels, which means it misses more true positives there.

Whether that asymmetry is acceptable depends entirely on the deployment context. If the cost of missing a high-efficiency Referral or LinkedIn role is high, the classification threshold should be lowered for these channels to recover more true positives, accepting a modest increase in false alarms in exchange. That kind of channel-specific threshold calibration is both technically feasible with XGBoost and operationally sensible given what this analysis has revealed.

16.5.2. Department-Level Analysis

The department-level results tell a more concerning story than the source-level analysis. Where source performance varied by 0.08 F1 points across channels, department performance varies by 0.19 points, from 0.50 for Product to 0.69 for Finance. AUC-ROC spans an even wider range, from 0.629 for Marketing to 0.802 for Finance, a spread of 0.173 points. This level of dispersion means the model is not equally reliable across departments, and deploying it without acknowledging that would create systematic winners and losers in how recruitment decisions are supported.

Table 16.4: Department Subgroup Summary

Source	N	F1	AUC-ROC	Precision	Recall	FP Rate	FN Rate	Status
Finance	155	0.69	0.802	0.758	0.627	0.097	0.181	Best
Engineering	155	0.60	0.692	0.587	0.611	0.200	0.181	Above mean
Sales	183	0.55	0.719	0.526	0.580	0.197	0.158	Mixed
HR	177	0.52	0.694	0.667	0.424	0.102	0.277	High FN risk
Marketing	161	0.51	0.629	0.523	0.500	0.193	0.211	Lowest AUC
Product	169	0.50	0.710	0.525	0.485	0.172	0.201	Threshold issue

- Finance: The Dominant Performer
  Finance is the only subgroup across both stratification dimensions to achieve an AUC-ROC above 0.80, the threshold commonly considered good discriminative performance. With the highest precision of any group at 0.758, the lowest false positive rate among departments at 0.097, and a recall of 0.627, the model is at its most reliable for Finance roles. When it predicts a Finance hire as High OAR, it is right more often than for any other group. The likely explanation is that Finance roles tend to have more regularised, predictable feature profiles: standardised cost structures, well-defined

seniority levels, and consistent applicant flows that align closely with the features the model has learned to use most effectively.

- Engineering: Moderate and Balanced
  Engineering sits near the mean AUC-ROC at 0.692 with an F1 of 0.60, slightly above the mean. Precision and Recall are reasonably balanced at 0.587 and 0.611 respectively. The model captures most genuinely high-efficiency Engineering roles, making it a middle-tier performer that is neither problematic nor particularly well-served. It would benefit from incremental feature engineering but does not represent an urgent fairness concern.
- Sales: Discriminative Power Without Calibration
  Sales presents an interesting divergence. The AUC-ROC of 0.719 sits above the mean, suggesting the model can rank-order Sales predictions reasonably well. But the F1 of 0.55 falls below the mean, indicating the model struggles to convert those rankings into accurate binary classifications. Precision of 0.526 is the lowest across all departments, and the false positive rate of 0.197 confirms the model is generating too many false alarms for Sales roles. The most likely explanation is feature value variance: Sales recruitment tends to involve fast-moving, variable cost structures and fluctuating applicant velocity, which may place many observations near the decision boundary where the model is most uncertain.
- HR: The Most Operationally Concerning Subgroup
  HR is the subgroup that warrants the most immediate attention before any production deployment. The false negative rate of 0.277 is the highest across all subgroups in both dimensions, meaning the model misses more than one in four genuinely high-efficiency HR roles. Recall of 0.424 is the lowest of any department. Yet the AUC-ROC of 0.694 is near the mean, and when the model does predict High OAR for an HR role, Precision of 0.667 means it is often right. The pattern is one of aggressive conservatism: the model rarely falsely claims an HR role is high-efficiency, but it systematically fails to identify the ones that genuinely are.
  In an operational deployment, this translates directly into underestimating HR recruitment efficiency. HR is a department that recruits for its own function, and a model that consistently undersells the quality of HR hiring outcomes could distort resource allocation decisions in ways that disadvantage the team responsible for the tool's adoption. This deserves targeted investigation.
- Marketing: Barely Above Chance
  Marketing has the lowest AUC-ROC of any subgroup in either dimension at 0.629, which falls into the range where the model's discrimination is only marginally better than random guessing. Every metric for Marketing sits near the midpoint: F1 of 0.51, Precision of 0.523, Recall of 0.500, and a false negative rate of 0.211. The model has not learned to reliably distinguish high and low efficiency Marketing roles.
  The likely cause is that Marketing recruitment often involves creative or specialised roles with non-standard difficulty profiles and variable sourcing patterns, features that don't align cleanly with the model's learned boundaries. Marketing is the subgroup most in need of targeted feature engineering, and it may ultimately benefit from a department-specific model variant rather than a universal classifier.

- Product: A Calibration Problem, Not a Learning Problem
  Product presents the most technically interesting case in the department analysis. Its F1 of 0.50 is the lowest across all subgroups in both dimensions, sitting at the practical floor of classifier utility. Yet its AUC-ROC of 0.710 is essentially at the mean, meaning the model can rank Product role outcomes by efficiency nearly as well as it does for the average department. The model has learned something useful about Product, but it cannot convert that learning into accurate binary predictions at the current threshold.

This is a classic sign of threshold miscalibration rather than a fundamental model failure. The default classification threshold of 0.5 is simply not well-positioned for Product's prediction score distribution. A department-specific threshold recalibration, which is a relatively simple intervention compared to retraining the model, may be sufficient to substantially improve Product's F1 without touching its discriminative performance at all. This should be the first step taken for Product before any broader model refinement.

16.5.3. Job Ttile-Level Analysis

Job title is the most granular level of fairness analysis in this study, and it produces the most consequential findings. While department and source subgroups revealed meaningful performance disparities, job title analysis goes further by identifying whether specific role types are systematically better or worse served by the model. That question carries particular weight here because jobtitle_oar_mean is itself a feature in the model. If the model performs poorly for certain job titles, it may be both contributing to and reinforcing that underperformance through the historical acceptance rate feedback mechanism identified in the SHAP analysis.

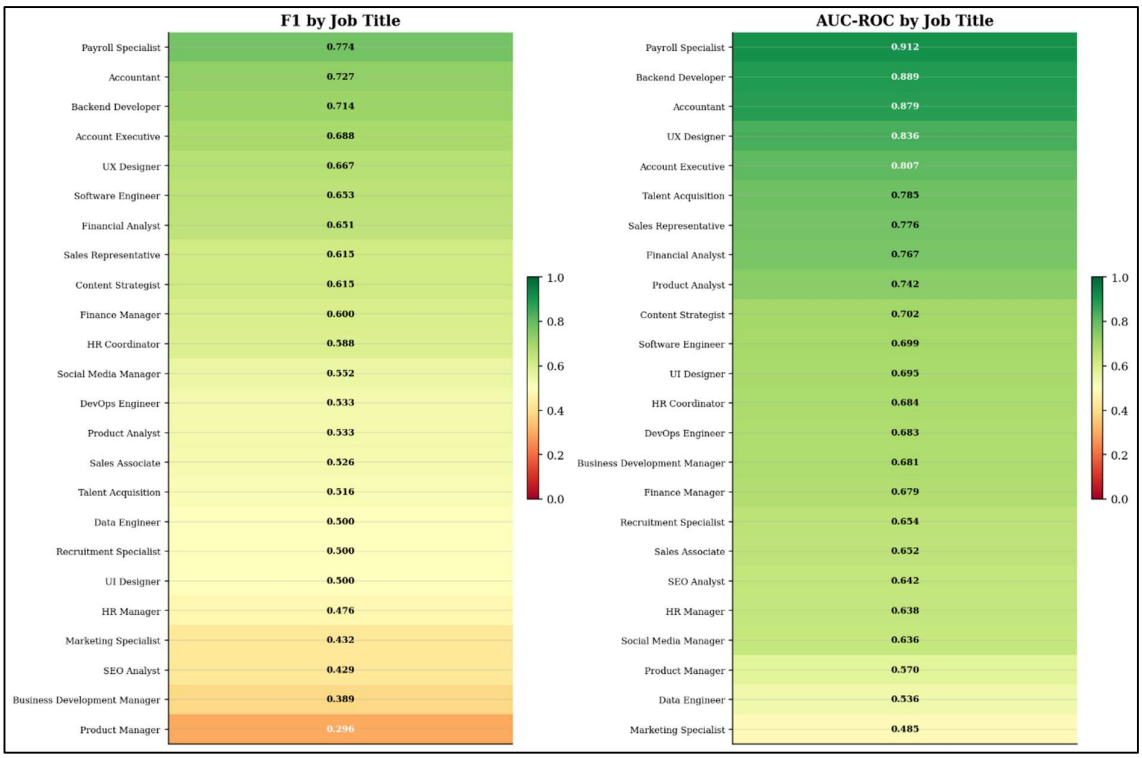

Figure 16.10: Model Performance by Job Title

The heatmap makes the dispersion immediately visible. The colour spectrum runs from deep green for the strongest performers through yellow-green for moderate ones and into

orange-red for the weakest, and the range of colours across the chart is far wider than anything seen in the department or source breakdowns.

F1-scores range from 0.774 for Payroll Specialist at the top to 0.296 for Product Manager at the bottom, a spread of 0.478 points. That is more than double the department-level spread of 0.19 points and nearly six times the source-level spread of 0.08 points. AUC-ROC ranges from 0.912 for Payroll Specialist to 0.485 for Marketing Specialist, a spread of 0.427 points. These are not marginal differences. At the bottom of the distribution, the model is performing at or near chance for certain roles while achieving near-clinical precision for others.

Payroll Specialist sits in a class of its own, with an F1 of 0.774 and an AUC-ROC of 0.912. Accountant, Backend Developer, Account Executive, and UX Designer all follow with strong performance on both metrics. These roles tend to share certain structural characteristics: well-defined scope, relatively standardised cost profiles, and predictable applicant flows. The model's core feature set is most naturally aligned with this kind of regularity, which explains why its predictions for these titles are both accurate and reliable.

Software Engineer, Financial Analyst, Sales Representative, and Content Strategist all sit in the moderate performance band, with F1 scores in the 0.60 to 0.63 range and AUC-ROC figures between 0.699 and 0.776. These are titles where the model performs adequately but leaves room for improvement through additional feature engineering or threshold adjustment.

The bottom of the ranking is where the fairness concerns become most acute. Product Manager has the lowest F1 at 0.296 and an AUC-ROC of 0.570, barely above chance. Marketing Specialist records an AUC-ROC of 0.485, which is effectively random discrimination. Business Development Manager, Data Engineer, and SEO Analyst all fall into the lower portion of the distribution.

Importantly, the mechanisms of failure differ across these titles. For Product Manager and Marketing Specialist, the low AUC scores indicate fundamental discriminative failure: the model has not learned to distinguish high and low efficiency outcomes for these roles in any reliable way. No amount of threshold adjustment will fix this; the feature set itself is not capturing the signals that differentiate these roles. Targeted feature engineering, potentially including role-specific difficulty indicators or compensation competitiveness metrics, would be required to improve performance here.

For roles like Talent Acquisition and Business Development Manager, where AUC remains moderate but F1 is low, the pattern is more consistent with threshold miscalibration, the same issue identified at the department level for Product. The model can rank these roles reasonably, it just can't translate those rankings into accurate binary predictions at the current threshold. A role-specific threshold recalibration is the appropriate first intervention.

The most systemic concern raised by this analysis is the potential for a self-reinforcing feedback loop. jobtitle_oar_mean is a feature in the model, meaning it encodes historical acceptance rate performance at the job title level. For roles that are already poorly predicted, the model may be generating Low OAR predictions that, if acted upon, reduce investment in those pipelines, which in turn produces worse actual acceptance rates, which then feeds back into a lower jobtitle_oar_mean in future training data, further entrenching the poor predictions.

This is not a hypothetical risk. It is a well-documented failure mode in operational machine learning systems, particularly when model outputs influence the data generating process. For the titles at the bottom of Figure 16.10, this feedback mechanism represents the most significant long-term interpretability and governance risk identified in this entire study. Explicit monitoring protocols should be established before the model is operationalised, and

the jobtitle_oar_mean feature should be audited regularly to detect any drift in the lowest-performing title groups that cannot be explained by genuine recruitment process changes.

16.5.4. Bias Risks and Mitigation Recommendations

The subgroup analyses across sources, departments, and job titles have surfaced a set of specific, nameable bias risks. Each is documented below with its severity, the mechanism driving it, and its operational consequence. These are not theoretical concerns; they are findings that should directly inform the governance conditions attached to any production deployment.

The Referral source subgroup has the highest false negative rate across all source subgroups at 0.249, meaning one in four genuinely high-efficiency Referral-sourced roles is predicted as low-efficiency. The AUC-ROC of 0.67 is the only source subgroup falling below the mean and the only one that would conventionally be classified as weak discriminative performance. The operational consequence is direct. If model predictions inform resource prioritisation, Referral-sourced roles will be systematically deprioritised relative to Recruiter and Job Portal roles, even when their true efficiency is equivalent or superior. Given that Referral channels are widely associated with stronger cultural fit and lower long-term attrition, this bias may disproportionately disadvantage diversity-referral programmes and internal mobility pipelines that rely on this channel.

LinkedIn's recall of 0.486 is the lowest across all source subgroups, and its false negative rate of 0.211 confirms the model consistently misses true high-efficiency LinkedIn roles. This likely reflects LinkedIn's structurally distinct cost-per-applicant and applicant velocity profiles, which differ from Recruiter and Job Portal channels in ways the current feature set does not adequately capture. The practical risk is that roles relying on passive sourcing strategies, which LinkedIn disproportionately serves, will be systematically undervalued by the model.

LinkedIn and Referral have the two lowest false positive rates of any source subgroup at 0.143 and 0.137 respectively, but they also carry the two highest false negative rates. The model has implicitly learned a more conservative decision boundary for informal channels, requiring stronger evidence to predict High OAR. This is a form of structural bias: the same underlying efficiency is harder for the model to recognise when a candidate came through an informal sourcing route. The asymmetry is not random noise; it is a learned feature of the model's behaviour.

HR carries the highest false negative rate across all department subgroups at 0.277. Combined with a true positive rate of 0.480 among the highest of any department, confirming that genuinely high-efficiency HR roles exist in real proportion in the data, this means the model is not struggling to find signal in HR; it is suppressing it. The irony is direct and damaging: the department responsible for overseeing the recruitment function receives the least reliable model support for evaluating its own recruitment efficiency. The feedback risk compounds this. If HR's recruitment budget or operational investment is informed by model-predicted efficiency scores, systematically suppressed predictions will lead to under-resourcing of HR recruitment. Reduced investment will lower actual HR acceptance rates. Lower actual rates will degrade jobtitle_oar_mean and dept_oar_mean values for HR roles in future training data. Future model versions will learn weaker signal for HR. This is a closed feedback loop with potentially progressive severity, and it needs to be explicitly governed before deployment.

Marketing's AUC-ROC of 0.629 is the lowest across all department subgroups and falls into the range conventionally classified as poor discriminative performance. Unlike HR, where the model has reasonable discriminative ability but a threshold problem, Marketing represents a more fundamental failure: the model has not learned reliable efficiency signal for Marketing

roles at all. F1 of 0.511, Precision of 0.523, and Recall of 0.500 collectively indicate near-random binary prediction quality. Any model-driven decision affecting Marketing recruitment resource allocation should be treated as unreliable until this is addressed.

Product's combination of AUC-ROC 0.710 with F1 of 0.504 is the clearest example of threshold miscalibration in the department analysis. The model can rank-order Product roles reasonably well, but the default classification boundary is poorly positioned for this department's prediction score distribution. This is a correctable bias that does not require retraining. If left uncorrected, however, it will systematically misclassify Product role efficiency at near-random rates, which in a high-volume, high-impact department like Product is a consequential oversight.

Finance's strong performance across both metrics creates a subtler but real problem when model predictions are used comparatively. If predictions are used to rank departments by recruitment efficiency or allocate centralised HR resources, Finance will consistently appear more efficiently recruited than Marketing or HR, partly because the model is more reliable for Finance rather than because Finance genuinely outperforms those departments. This creates a measurement artifact bias where apparent cross-departmental efficiency gaps are partly a function of differential model reliability rather than true operational differences. Decision-makers using model outputs to compare departments need to be explicitly informed of this.

Marketing Specialist's AUC-ROC of 0.485 is the single most severe bias finding in the entire analysis. A value below 0.50 means the model's predicted probability scores are inversely correlated with true efficiency outcomes for this job title. Cases the model scores as likely High OAR are actually more likely to be Low OAR, and vice versa. Any automated decision using model scores for Marketing Specialist roles will actively harm recruitment outcomes for this title. This is not a calibration issue that can be fixed by adjusting a threshold. It is a fundamental inversion that may reflect label leakage in title-specific training data, a distribution shift in Marketing Specialist role profiles, or a systematic feature encoding error specific to this title. It requires investigation before deployment, not after.

Product Manager's F1 of 0.296 and AUC-ROC of 0.570 represent simultaneous failure on both discriminative and classificatory dimensions. With F1 approaching the level of random guessing and only marginally above-chance discrimination, the model provides no meaningful decision support for Product Manager recruitment efficiency. Given that Product Manager is among the highest-volume and highest-impact roles in most organisations, this blind spot carries significant operational consequences if predictions are used in resourcing or hiring investment decisions.

The model uses jobtitle_oar_mean as a feature, and for the worst-performing titles, this creates a structurally dangerous feedback mechanism. The model systematically under-predicts efficiency for these titles. Reduced predicted scores suppress recruitment investment. Reduced investment lowers actual acceptance rates. Lower actual rates degrade jobtitle_oar_mean for these titles in future training data. Future model versions learn even weaker signal for these titles. The loop closes and tightens with each retraining cycle. This is not a theoretical risk. It is a predictable consequence of deploying a model whose features include aggregated historical outcomes that the model's own predictions will influence. Without an explicit intervention to break this loop, performance for bottom-tier job titles will deteriorate progressively. An intervention could take several forms: excluding jobtitle_oar_mean from the feature set for flagged titles, applying a floor value to prevent suppression, or auditing the feature after each

retraining cycle against an independent ground truth. What is not acceptable is deploying the model without acknowledging the risk.

Talent Acquisition, Business Development Manager, SEO Analyst, and UI Designer all show the same AUC-high and F1-low pattern that was identified at the Product department level. These titles have reasonable discriminative signal but a poorly positioned classification boundary. The bias here is structural but correctable without retraining. Title-specific threshold recalibration should be applied to this cluster as a near-term priority.

The F1 range of 0.478 across job titles is not a normal distribution of performance variation. It reflects a structural inequality in model service quality across role types. Finance-adjacent roles like Payroll Specialist and Accountant receive near-excellent prediction quality. Marketing and general management roles receive predictions that are near-useless or actively misleading. If the model is deployed uniformly across all job titles without adjustment, it is effectively providing a two-tier service: high-quality automated decision support for some roles and actively misleading guidance for others. That outcome is not acceptable in a production recruitment system where model outputs carry real consequences for real hiring decisions.

Chapter XVII
Deployment

17.1. Dashboard Features

RePort is a recruitment decision-support platform built with Streamlit and deployed on Streamlit Cloud. It brings together the analytical and predictive outputs of this project into a single interface designed for HR teams, giving both a real-time view of recruitment pipeline performance and the ability to estimate offer acceptance outcomes before a process concludes. The platform is organised into two main tabs, each serving a distinct operational purpose.

17.1.1. Candidate Overview Tab

The Candidate Overview tab is designed for HR managers and team leads who need to monitor recruitment efficiency across the organisation without running queries or pulling reports manually. It surfaces the most important pipeline metrics through interactive charts and KPI cards that update based on sidebar filters.

Table 17.1: Dashboard Features

Feature	Description
KPI Cards	Displays Total Records, Average Cost per Hire, Average Time to Hire, Average OAR, and High OAR Rate (≥70%)
Records by Source	Grouped bar chart showing candidate volume split by High/Low OAR per acquisition channel
Records by Department	Bar chart of average OAR per department, sorted by performance
Records by Job Title	Horizontal bar chart of candidate volume for the top 15 roles
OAR Distribution	Stacked histogram showing full OAR spread with a 0.70 classification threshold marker
Cost & Time to Hire	Side-by-side box plots comparing cost and time distributions across High vs Low OAR classes
Feature Importances	Horizontal bar chart of XGBoost feature importances, read directly from the trained model
KPI Summary by Source	Three-panel chart showing average cost, time, and OAR per acquisition channel
Candidate Records Table	Paginated raw data table with CSV download and sidebar filters by department and source

Together, these components give HR leadership a single view of where the recruitment function is performing well, where it is falling short, and which sourcing channels and departments are driving the most meaningful outcomes.

17.1.2. Candidate Predictor Tab

The Candidate Predictor tab is where the model becomes directly actionable. It allows HR teams to estimate the likelihood of a High or Low OAR outcome before a recruitment process concludes, applying the same feature engineering pipeline used during model training to ensure consistency between training-time and prediction-time behaviour.

Table 17.2: Features on the Candidate Overview Tab

Feature	Description
Single Candidate Predictor	Form-based input for department, job title (linked to department), source, number of applicants, time to hire, cost per hire, and expected OAR. Returns prediction label, probability, and gauge chart.

Insights & Recommendations	Five detailed insight cards (Cost Efficiency, Time to Hire, Source Quality, Pipeline Volume, and Overall Difficulty) each with an HR-friendly recommendation and status tag (On Track / Needs Attention / Action Required).
What-If Scenario Comparison	Side-by-side comparison of two recruitment scenarios for the same role. HR adjusts source, applicants, time, cost, and expected OAR for each scenario and runs a comparison to identify the better approach.
Batch Upload Predictor	Accepts a CSV file with multiple candidates and returns predictions with High/Low OAR labels and probability scores. Includes a downloadable results file and summary charts.

The Candidate Predictor tab is designed to be used by recruiters and HR Business Partners who don't have a data science background. Inputs are form-based and labelled in plain language. Outputs are expressed as actionable labels and recommendations rather than raw model scores. The What-If scenario tool in particular gives recruitment teams a concrete way to test process changes before committing to them, answering questions like whether switching sourcing channels or adjusting timeline expectations is likely to improve the odds of a successful hire.

17.2. Deployment Process

Getting from a trained model to a live dashboard requires more than saving a file and uploading it somewhere. This section documents the full deployment pipeline, from the final model training run through to the live application, including the repository structure, saved artefacts, and package dependencies that keep everything working consistently.

17.2.1. End-to-End Deployment Flow

Table 17.3: End-to-End Deployment Flow

Step	Stage	Details
1	Model Training	Stage 2 & 3 Jupyter Notebook: feature engineering, SMOTE balancing, XGBoost training with Random Search hyperparameter tuning
2	Model Saving	Trained model and all pipeline artefacts saved to models/recruitment_model.joblib using joblib.dump()
3	Dashboard Development	app.py built with Streamlit: two tabs, interactive charts (Plotly), prediction form, insights engine, and what-if comparison
4	GitHub Repository Setup	Repository initialized with structured folder layout. requirements.txt created and pushed. .gitignore configured to exclude generated files.
5	Streamlit Cloud Deployment	Repository connected to Streamlit Cloud via share.streamlit.io. Main file set to app.py. Auto-deploy on every push to main branch.
6	Testing & Fixes	Multiple deployment iterations to resolve dependency errors, file path issues, and UI/UX improvements. See Section 3 for details.
7	Go Live	Dashboard confirmed live and accessible at https://undercode.streamlit.app/

17.2.2. Repository Structure

A clean and consistent folder structure was established in the GitHub repository to ensure Streamlit Cloud can locate all required files without manual path configuration.

Table 17.4: Repository Structure

Path	Purpose
app.py	Main Streamlit dashboard application
requirements.txt	Python package dependencies for Streamlit Cloud
.gitignore	Excludes generated files and caches from version control
data/raw/recruitment_efficiency_improved.csv	Source dataset (5,000 recruitment records)
models/recruitment_model.joblib	Trained XGBoost model and all pipeline artefacts
src/UnderCode.png	Team logo used as dashboard page icon and sidebar image

17.2.3. Model Artefacts Saved

The .joblib bundle saved at the end of Stage 4 contains everything the dashboard needs to replicate the exact feature engineering pipeline used during training. Loading the bundle alone is sufficient to generate consistent predictions without any additional preprocessing steps in the application layer.

The bundle contains the trained XGBoost classifier with 317 estimators and a maximum depth of 8; the ordered list of 10 engineered feature names; the smoothed target encoding mappings for department, job title, source, and department-source interaction, all fitted on the training set only; the global mean and smoothing factor used as fallback parameters for unknown categories; the scaling statistics for the difficulty_additive feature including training set mean and standard deviation for both time and cost components; and the department-level median values for cost, time, and OAR used to compute the DREI binary flag. Saving all of these artefacts together ensures that the dashboard cannot accidentally apply different encoding mappings or scaling parameters than those used during training, which would silently corrupt predictions without raising any errors.

17.2.4. Dependencies

The following packages were specified in requirements.txt for installation on Streamlit Cloud.

Table 17.5: Requirements

Package	Purpose
streamlit	Web application framework for the dashboard
pandas	Data loading, filtering, and manipulation
numpy	Numerical operations for feature engineering
joblib	Loading the saved model and artefacts bundle
plotly	Interactive charts and visualizations
xgboost	XGBoost classifier for predictions
scikit-learn	Preprocessing and model pipeline utilities
imbalanced-learn	SMOTE for training set balancing
Pillow	Loading the team logo image file

Pinning these dependencies explicitly ensures that the deployment environment on Streamlit Cloud matches the development environment, preventing version conflicts from silently changing model behaviour between the notebook and the live application.

17.3. Challenges Encountered & Resolutions

Deployment rarely goes smoothly on the first attempt, and this project was no exception. The issues encountered ranged from missing files and broken paths to subtle state management problems that only surfaced during interactive use. Each challenge is documented below with its root cause and the fix applied.

Table 17.6: Challenges Encountered & Resolutions

#	Challenge	Severity	Resolution	Status
1	ModuleNotFoundError: joblib not found on Streamlit Cloud	Critical	requirements.txt was missing entirely. Created and pushed with all required packages.	Resolved
2	FileNotFoundError: recruitment_efficiency_improved.csv not found	Critical	CSV was not pushed to GitHub. File path in app.py updated to match actual repo location: data/raw/.	Resolved
3	FileNotFoundError: UnderCode.png logo not found	High	Logo moved to src/ folder in the repository. Path updated in app.py to src/UnderCode.png.	Resolved
4	Job title dropdown did not update when department was changed	High	Root cause: st.form caches all widget values until submission. Department selector moved outside the form so job titles refresh instantly on department change.	Resolved
5	What-If Comparison reset to top of page when any widget was changed	High	Root cause: st.button loses state on every Streamlit rerun. Resolved by storing all comparison inputs and results in st.session_state, rendering results from state rather than from button click.	Resolved
6	DREI feature always returned 0 for single predictions — biasing results toward Low OAR	High	DREI requires offer_acceptance_rate to compute. Added an Expected OAR slider to the prediction form so users provide this value. Default set just above the department median.	Resolved

7	Chart labels clipped: Records by Job Title counts and Feature Importance bars cut off	Medium	Increased right margin (r=70) for Job Title chart and (r=80) for Feature Importance chart. Added xaxis_range with 1.20x and 1.25x multiplier respectively to prevent label overflow.	Resolved
8	dept_medians column names mismatch between notebook and app.py	Medium	Notebook used explicit rename() with dept_median_cph, dept_median_tth, dept_median_oar. app.py DREI logic updated to use exact column names from the joblib bundle.	Resolved
9	Cost & Time to Hire chart: legend overlapped subplot titles	Low	Increased top margin (t=50) and raised legend position (y=1.15) to separate the legend from subplot title text.	Resolved

The DREI issue in challenge six was the most consequential for prediction quality. Because DREI is the dominant feature in the model, a silent default of zero for all single predictions would have systematically biased every output toward Low OAR, making the predictor tab unreliable for its primary purpose. Adding the Expected OAR slider and defaulting it just above the department median ensures that users provide a meaningful input rather than the feature being silently suppressed.

The What-If Comparison state management problem in challenge five is a common pitfall in Streamlit applications that involve multi-step interactions. Streamlit reruns the entire script on every widget interaction, which means any results generated by a button click are lost the moment the user adjusts another input. Storing inputs and results in st.session_state breaks that dependency, allowing the comparison to persist until the user explicitly requests a new one.

The column name mismatch in challenge eight is a reminder of how quietly this kind of error can propagate. A rename applied in the notebook but not reflected in the application layer would have produced incorrect DREI values without raising any obvious error, since the column names were close enough that the mismatch wasn't immediately visible. Explicit column naming conventions and a reconciliation check between notebook artefacts and application code are worthwhile safeguards for any future model updates.

17.4. Current Deployment Status

Table 17.7: Current Deployment Status

Item	Details
Status	**Live**
URL	https://undercode.streamlit.app/
Platform	Streamlit Community Cloud

Deployment Branch	main
Main File	app.py
Auto-Redeploy	Yes, triggers automatically on every push to the main branch
Active Model	XGBoost + SMOTE | AUC-ROC: 0.71 | Trained on 4,000 records
Dataset	recruitment_efficiency_improved.csv, 5,000 records, 6 departments, 4 sources
Last Deployment	May 2026
Access	Public URL, accessible to all team members without login

17.4.1. What is Currently Live

The live deployment includes the full Candidate Overview tab with ten interactive charts, KPI cards, sidebar filters for department and source, and CSV export functionality. The Candidate Predictor tab is live with the single prediction form, five detailed insight cards, and the What-If scenario comparison tool. Batch upload prediction is available, with downloadable results and summary charts.

The dashboard carries RePort branding with the UnderCode logo, a custom dark theme, and HR-friendly section descriptions throughout. The model loads from models/recruitment_model.joblib with the full feature engineering pipeline replicated in app.py, ensuring prediction consistency between the training environment and the live application.

17.4.2. Known Limitations at Launch

The dashboard has no live connection to an HR system. Updating the underlying data requires a manual CSV upload, which means the pipeline metrics displayed in the Candidate Overview tab reflect the state of the data at the time of the last upload rather than real-time recruitment activity. This is acceptable for a first deployment but should be addressed as a priority in any future development cycle.

Model accuracy sits at 62.3%, which is meaningful but not definitive. Predictions should be used as one input into a hiring decision rather than as a final verdict. The model's purpose is to surface risk factors and prompt useful questions, not to replace recruiter judgment.

The Expected OAR input in the Candidate Predictor relies entirely on user judgment. There is currently no validation against historical norms or automatic population from past data for similar roles. A user who enters an unrealistic value will receive a prediction based on that value without any warning. Adding a suggested range or a historical reference point for each role type would improve the quality of inputs and, by extension, the reliability of predictions.

Finally, the application is publicly accessible to anyone with the URL and has no user authentication layer. For an internal HR tool handling candidate and process data, this is a known gap that will need to be addressed before the platform is used with real recruitment records rather than the synthetic dataset it currently runs on.

Chapter XVIII
Monitoring & Maintenance

18.1. Monitoring Plan

Deploying a model is not the end of the work. A model that performs well today can degrade quietly as recruitment patterns shift, organisational structures change, or the data it receives begins to look different from the data it was trained on. The monitoring plan below defines what should be tracked, how often, and who is responsible.

18.1.1. Monitoring Schedule

Table 18.1: Monitoring Schedule

Frequency	Activity	Responsible
Weekly	Review prediction logs & flagged outliers	Data Team
Monthly	Compare predicted vs actual OAR outcomes	Data Team and HR Manager
Quarterly	Full model performance review (AUC-ROC, F1, accuracy)	Data Scientist
Ad-hoc	Triggered review if alert thresholds are breached	Data Team

18.1.2. Key Metrics to Monitor

Table 18 2: Key Metrics to Monitor

Metric	Threshold	Action if Breached
AUC-ROC Score	< 0.65	Trigger retraining review
F1 Score	< 0.50	Investigate feature drift, consider retraining
Prediction Accuracy	< 58%	Escalate to data science team
DREI Positive Rate	Shift > ±15%	Check for changes in recruitment patterns
Actual vs Predicted OAR	Divergence > 10%	Data drift analysis, potential retraining

18.1.3. Data Drift Detection

Data drift occurs when the distribution of incoming recruitment data shifts meaningfully away from the distribution the model was trained on. Left undetected, drift quietly degrades prediction quality without triggering any obvious errors. The following checks should be run monthly as a standard part of the monitoring routine.

The distributions of cost_per_hire, time_to_hire_days, and num_applicants should be compared against the training baseline to identify any significant shifts in central tendency or spread. Changes in the sourcing channel mix, such as a substantial increase in Job Portal volume relative to Referral, should be flagged since the model's channel-level encoding mappings were fitted on a specific distribution. Any new departments or job titles appearing in the incoming data that were not present in the training set should be identified immediately, since these records will receive the global mean as their target-encoded value rather than a group-specific estimate. Finally, any significant shift in the organisation's overall offer acceptance rate should be treated as a potential signal of genuine process change rather than simply model drift, and the two should be distinguished before triggering retraining.

18.2. Maintenance Plan

Monitoring tells you when something is wrong. The maintenance plan defines what to do about it.

18.2.1. Retraining Triggers

Retraining should be initiated when any of the following conditions are met. Model AUC-ROC falls below 0.65 on recent data. Actual versus predicted OAR diverges by more than 10% over a rolling three-month period. A significant organisational change occurs, such as the addition of new departments, a revised hiring policy, or the introduction of new sourcing channels. More than 1,000 new recruitment records have been collected since the last training run. A quarterly review flags consistent underperformance on a specific department or job title that has not been addressed through threshold recalibration alone. These triggers are designed to be specific enough to avoid unnecessary retraining while remaining sensitive enough to catch genuine degradation before it affects operational decisions.

18.2.2. Retraining Process

Table 18.3: Retraining Process

Step	Task	Notes
1	Collect & validate new data	Ensure no missing values in key columns
2	Re-run Stage 2 & 3 notebook	Feature engineering, SMOTE, hyperparameter tuning
3	Evaluate new model vs current	Must outperform current AUC-ROC (0.71) to replace
4	Save new .joblib bundle	Include updated metadata (AUC, n_train, date)
5	Deploy to Streamlit Cloud	Push updated models/ folder to GitHub
6	Validate dashboard output	Run sample predictions and verify results

The evaluation gate in step three is important. Retraining on new data does not guarantee improvement, and a new model that underperforms the current one should not be deployed simply because it is more recent. The current AUC-ROC of 0.71 sets the minimum bar for any replacement.

18.2.3. Version Control

All model versions are saved with a timestamp suffix, for example recruitment_model_2026Q2.joblib, so the training date is embedded in the filename rather than relying on file metadata. The active model in production is always named recruitment_model.joblib in the models/ folder, making it straightforward to update the live application by replacing a single file.

Previous model versions are archived rather than deleted, ensuring that a rollback to an earlier version is always possible if a newly deployed model produces unexpected behaviour in production. The GitHub repository serves as the full version history for all code and configuration changes, providing a complete audit trail of how the system has evolved over time.

Chapter XIX
Recommendations

19.1. Immediate Actions

The recommendations in this section require no additional infrastructure. Everything described here can be executed using the RePort dashboard that is already live at https://undercode.streamlit.app/. The primary deployment gap is adoption, not technology.

RePort is live and fully functional. The tool exists; the challenge now is getting the right people to use it consistently. A mandatory 30-minute onboarding session should be scheduled for all Talent Acquisition staff and hiring managers. The session doesn't need to be technical; it should focus on the two tabs, what each chart is telling the user, and how to run a prediction before extending an offer. A RePort Champion should be designated in each department to own daily use and field questions from their team. Sidebar filters should be configured to match each team's scope so that every user opens the dashboard to a view that is immediately relevant to their work. The public URL should be shared with all stakeholders; no login is required, and there is no barrier to access.

The model's performance analysis is clear on this point. Recruiter-sourced candidates produce the best predictive performance with an F1 of 0.60 and an AUC of 0.74. Job Portal follows closely. LinkedIn and Referral both exhibit systematic under-prediction and recall deficits that translate directly into missed high-efficiency hires.

At a minimum, 20% of the LinkedIn sourcing budget for high-priority roles should be shifted to Recruiter or Job Portal channels in the near term. The KPI Summary by Source chart in the Candidate Overview tab should be used to track average OAR and cost per channel on a weekly basis, so any change in channel performance is visible quickly. Referral should not be eliminated. The high false negative rate of 0.249 for this channel raises a genuine question about whether the model is under-measuring the quality that Referral produces rather than accurately reflecting it. That question deserves investigation before any decision is made to reduce Referral investment.

The Candidate Predictor tab's What-If Scenario Comparison allows HR teams to compare two sourcing or cost configurations before extending an offer in under 30 seconds. That capability should be embedded into standard practice rather than used occasionally. Use of the What-If tool should be made mandatory for all roles where cost-per-hire exceeds $7,000, since these are the hires where a declined offer is most expensive to recover from. Each scenario comparison output should be documented and logged for monthly review, creating an audit trail that also feeds into the model feedback loop described below. The five Insight Cards covering Cost Efficiency, Time to Hire, Source Quality, Pipeline Volume, and Overall Difficulty should be treated as a standard pre-offer checklist rather than supplementary information.

Applicant volume shows no statistically significant correlation with offer acceptance rate. Larger pipelines are not producing better outcomes; they are inflating cost-per-hire and time-to-hire without any yield improvement. For Engineering, Product, and Marketing roles, which show the highest applicants-per-day variance, a structured pre-screening step should be added before shortlisting. This reduces the screening burden on hiring managers and increases the proportion of the pipeline that consists of genuinely interested and qualified candidates. Job descriptions for the five roles with the highest applicant-to-offer ratios should be revised to filter for fit rather than volume. The Applicant Volume KPI card in RePort should be reviewed

monthly to track whether pipeline tightening is producing the expected reduction in cost and time without degrading acceptance rates.

The model currently has no mechanism for learning from the decisions it informs. Without a feedback loop, there is no way to measure whether predictions are accurate over time, and no trigger for retraining based on real-world outcomes. A simple shared spreadsheet should be created immediately to capture four fields for each prediction made through the dashboard: role ID, predicted OAR class, actual acceptance decision, and date. The Data and Analytics team should be assigned to compare model predictions against real acceptance decisions each month and report on prediction accuracy. A retraining trigger should be set: if monthly accuracy falls below 60% for two consecutive months, retraining is initiated using the process documented in Chapter 18. This is the minimum viable feedback infrastructure and it can be operational within a week of deployment.

19.2. Medium-Term Improvements

The subgroup analysis revealed structurally unequal model performance that cannot be ignored if the model is to be used for resourcing or budget decisions. The table below summarises the priority bias risks and the recommended action for each.

Table 19.1: Priority Bias Risks Requiring Immediate Governance

Bias Risk	Severity	Metric	Recommended Action
Marketing Specialist Inversion	Critical	AUC: 0.485	Exclude from automated scoring immediately. Requires separate model or human review.
Product Manager Failure	Critical	F1: 0.296	Block model use for PM roles. Retrain with additional PM-specific features.
HR Dept. Under-Estimation	High	FN: 27.7%	Apply lower decision threshold (< 0.5) for HR department predictions.
Referral Channel Under-Prediction	High	FN: 24.9%	Implement channel-specific threshold calibration for Referral sourcing.
Marketing Dept. Near-Chance	High	AUC: 0.629	Flag Marketing predictions as unreliable in dashboard. Add disclaimer.

Marketing Specialist and Product Manager represent the two most urgent cases. For Marketing Specialist, an AUC below 0.50 means the model is actively inverting the correct prediction for this role type. Any automated scoring for Marketing Specialist roles should be suspended immediately and replaced with human review until a reliable model can be developed for this title. For Product Manager, the combination of near-random F1 and barely-above-chance AUC means the model is providing no meaningful signal at all. Model use for PM roles should be blocked pending a targeted retraining effort that incorporates PM-specific features.

For HR and Referral, the path forward is threshold recalibration rather than exclusion. Lowering the decision threshold for HR department predictions will recover more of the true high-efficiency cases that the current threshold is missing. Applying channel-specific threshold calibration for Referral sourcing will similarly reduce the false negative rate for a channel that the model is systematically treating too conservatively.

The model uses jobtitle_oar_mean as a feature with a mean absolute SHAP value of 0.2027. For the worst-performing job titles, systematic under-prediction will degrade this feature in future training data, making those titles progressively harder to predict correctly

across retraining cycles. Three interventions should be implemented before the next retraining run.

A floor value should be applied to jobtitle_oar_mean in the feature engineering pipeline, using the global mean as the minimum permissible value to prevent progressive decay for any title. Job titles with fewer than 30 training observations should be flagged, and jobtitle_oar_mean should not be used as a feature for those titles, since the estimate is too unreliable to be useful. Finally, jobtitle_oar_mean drift should be logged quarterly as part of the standard model monitoring routine, so any deterioration is caught before it compounds.

One in three requisitions currently exceeds 60 days, which is the primary driver of the 31% benchmark gap in time-to-hire. Reducing this proportion is one of the highest-leverage operational improvements available.

A 45-day escalation trigger should be introduced: any role not filled by day 45 is automatically escalated to the TA Director, with a required action plan rather than a passive notification. For Engineering and Finance roles, which show the highest time-to-hire variance, parallel-track interview processes should replace sequential staging, so that different assessment dimensions run concurrently rather than one after another. The KPI Cards and Time-to-Hire box plots in the Candidate Overview tab should be used to segment roles exceeding 60 days by department and source, making structural bottlenecks visible rather than buried in averages.

The target is to reduce the proportion of requisitions exceeding 60 days from 33.2% to below 20% within six months. That is an achievable reduction if the escalation trigger and process changes are implemented promptly.

The current model relies entirely on process metrics: cost, time, and source. It cannot account for candidate-role fit because it has no candidate quality data. This is the most significant ceiling on future model performance, and addressing it requires a change to what gets recorded rather than how the model is built.

Three new fields should be added to the recruitment tracking CSV for all new requisitions starting from the next hiring cycle: structured interview scores on a 1 to 5 scale, pre-screening assessment results where available, and hiring manager satisfaction ratings collected at the 90-day post-hire mark. These fields are low-cost to collect and high-value for model development. The target is 1,000 enriched records within six months, which would form the basis for a meaningfully higher-quality retrained model capable of distinguishing between process efficiency and candidate fit as separate drivers of offer acceptance outcomes.

19.3. Long-Term Strategy

The current dashboard requires manual CSV uploads to update its underlying data. This creates a lag between what is happening in the recruitment pipeline and what the dashboard is showing, which reduces its value as a real-time decision support tool and places an ongoing manual burden on whoever is responsible for keeping the data current.

The long-term solution is a direct connection between RePort and the organisation's Applicant Tracking System via API or database connector, replacing the static CSV upload with an auto-refreshing data pipeline. Apache Airflow is one option for orchestrating scheduled data pulls, though simpler alternatives exist depending on the ATS and the technical infrastructure available. Alongside the pipeline itself, data validation checks should be implemented at ingestion: records missing key fields should be flagged before they enter the prediction pipeline rather than silently producing unreliable outputs. User authentication should also be added to protect candidate data, since the current dashboard URL is publicly

accessible without any login requirement, which is not appropriate for a tool handling real recruitment records.

The current model was trained on a synthetic dataset with a uniform distributional structure that limited how much genuine signal was available to learn from. Broader and richer training data will improve subgroup coverage, reduce the AUC-F1 divergence observed across departments and job titles, and move overall accuracy toward the SMART goal of 80% or above.

The target for the next major retraining should be 15,000 or more records spanning at least three annual cohorts. Prioritising collection for the job titles where current performance is below a useful threshold, specifically Product Manager, Marketing Specialist, and SEO Analyst, will address the most consequential gaps in the model's current coverage. Once 500 or more new records are available, quarterly retraining should be initiated using the same XGBoost with SMOTE pipeline and RandomizedSearchCV hyperparameter tuning that produced the current model.

The most powerful long-term improvement to the system is not a better algorithm or a larger dataset. It is a closed feedback loop where real hiring outcomes are fed back into the model on a regular basis, creating a learning system that improves continuously rather than only at scheduled retraining intervals.

A Prediction Feedback button should be added to each prediction output in the Candidate Predictor tab, allowing HR to mark each prediction as correct or incorrect after the actual hiring decision is known. Feedback should be aggregated monthly and used as a labelling signal for quarterly retraining. Prediction accuracy by department and source should be tracked over time and exposed as a model health chart in the Candidate Overview tab, giving HR leadership visibility into how the model is performing in practice rather than only at the point of initial deployment. This feedback infrastructure is also the mechanism that makes threshold calibration and subgroup bias monitoring sustainable over time rather than a one-off exercise.

The current model applies uniform weights and thresholds across all departments and job titles. The fairness analysis demonstrated why that approach is structurally inadequate: the F1 range of 0.478 across job titles and the AUC-ROC spread of 0.173 across departments are too wide for a single threshold to serve all groups equally well.

The near-term step is to train separate threshold values per department using the calibration evidence already documented in Chapter 16. Product needs a recalibrated threshold because its AUC is near-mean but its F1 is at the floor. HR needs a lower threshold because its false negative rate is the highest of any department. These changes do not require retraining and can be implemented relatively quickly.

The longer-term step is to explore department-specific models for the groups where the feature space appears fundamentally different from the rest of the dataset. Finance, with its high volume and strong signal, is a natural candidate for a standalone model that can be developed and maintained with confidence. Marketing, where the global model is performing near chance, may require a completely different feature set before any classifier can be expected to produce reliable results.

For departments and job titles that sit between these two extremes, an ensemble approach that blends the global model with department-specific corrections offers a middle path. Rather than maintaining a separate model for every group, corrections can be applied on top of a shared base model, reducing the maintenance burden while still addressing the most significant sources of subgroup bias.

References

Benjamin, A. (2024). *State of U.S. recruiting 2024–2025: Key hiring metrics [LinkedIn Pulse article]*. LinkedIn. https://www.linkedin.com/pulse/state-us-recruiting-20242025-key-hiring-metrics-pharma-alex-benjamin-neuje/

KPI Depot. (n.d.). *Job offer decline rate: KPI definition, formula, & benchmarks*. https://kpidepot.com/kpi/job-offer-acceptance-rate

Laurano, M. (2015). *The true cost of a bad hire*. Brandon Hall Group. https://www.bdo.com/getmedia/fc989309-6824-4ad6-9f8d-9ef1138e3d42/the-true-cost-of-a-badhire.pdf

LinkedIn Talent Solutions. (2024). *Global talent trends 2024.* LinkedIn Corporation. https://business.linkedin.com/talent-solutions/global-talent-trends

Society for Human Resource Management. (2022). *Talent access report* [Benchmarking report]. https://www.shrm.org/content/dam/en/shrm/research/benchmarking/Talent%20Access%20Report-TOTAL.pdf

Society for Human Resource Management. (2025). *2025 recruiting executives benchmarking: Insights to maximize recruitment*. https://www.shrm.org/topics-tools/research/2025-recruiting-benchmarking